THE UNIVERSITY OF DANANG

VN-UK INSTITUTE FOR RESEARCH AND EXECUTIVE EDUCATION





XÂY DỰNG WEBSITE CỜ CARO
ỨNG DỤNG CÁC THUẬT TOÁN TÌM KIẾM

BÁO CÁO ĐỒ ÁN CƠ SỞ

Sinh viên thực hiện: Nguyễn Tâm Đức

Mã số sinh viên: 24020005

Giảng viên hướng dẫn: Nguyễn Văn Thọ

Lớp: CSE24 Năm học: 2026–2027

TÓM TẮT

Đồ án xây dựng website cờ Caro trên bàn 15 × 15 với chế độ hai người chơi và người chơi đối đầu với bot nhiều mức độ khó. Ứng dụng hiện hành sử dụng React, TypeScript và PixiJS; engine Rust được biên dịch thành WebAssembly và thực thi qua Web Worker. Luật thắng là Freestyle Gomoku, công nhận chuỗi từ năm quân liên tiếp trở lên. Báo cáo trình bày cơ sở dữ liệu bàn cờ, bất biến đặt và hoàn tác quân, quét thắng theo nước cuối, nhận diện chuỗi, heuristic, Random, Rule-based, sinh ứng viên Chebyshev, Minimax, Alpha-Beta và các kỹ thuật tìm kiếm nâng cao đã được nghiên cứu.

Nội dung được đối chiếu với mã ứng dụng, engine game, mã TypeScript legacy, thư viện Rust reference, website trình bày và hồ sơ nghiên cứu theo mười tuần. Các kỹ thuật iterative deepening, Zobrist và bảng chuyển vị được trình bày theo đúng nơi đã hiện thực; mức Master của engine game hiện hành dùng Alpha-Beta độ sâu cố định. Báo cáo giải thích công thức, pseudocode, ví dụ tính tay, độ phức tạp, lợi ích và giới hạn của từng kỹ thuật; ảnh và biểu đồ được xử lý hoặc tái dựng bằng Python từ nguồn đã xác định.

Kết quả đánh giá gồm dữ liệu native và kiểm thử tích hợp đã lưu, cùng lần kiểm tra lại ứng dụng ngày 27/10/2026 đạt 59 test trong 9 file, typecheck và build thành công. Những số liệu lịch sử, nội dung minh họa và kết quả quan sát browser được ghi theo phạm vi của chúng. Chưa có giải đấu đủ để hiệu chỉnh sức mạnh sáu mức bot hoặc nghiệm thu trên thiết bị cảm ứng và host Wink thực. Sản phẩm của đồ án gồm ứng dụng, nền tảng trình bày, bộ thí nghiệm và hồ sơ giải thích có khả năng truy xuất nguồn.

Từ khóa: Caro, Gomoku, Minimax, Alpha-Beta, heuristic, Rust, WebAssembly, Worker.

DANH MỤC TỪ VIẾT TẮT

AI — Artificial Intelligence, trí tuệ nhân tạo

API — Application Programming Interface, giao diện lập trình ứng dụng

DFS — Depth-First Search, tìm kiếm theo chiều sâu

DOM — Document Object Model, mô hình đối tượng tài liệu

HUD — Heads-Up Display, vùng thông tin trạng thái trên giao diện

PRNG — Pseudorandom Number Generator, bộ sinh số giả ngẫu nhiên

PvP — Player versus Player, hai người chơi

PvE — Player versus Environment, người chơi đấu máy trong báo cáo này

TT — Transposition Table, bảng chuyển vị

UI — User Interface, giao diện người dùng

WASM — WebAssembly, định dạng mã thực thi được biên dịch cho nền web

Ply — Một nước đặt quân của một bên

Fixture — Trạng thái hoặc đầu vào được chuẩn bị để kiểm thử

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU 1
1.1. Bối cảnh và lý do thực hiện 1
1.2. Mục tiêu và câu hỏi của đồ án 2
1.3. Đối tượng, phạm vi và sản phẩm khảo sát 3
1.4. Phương pháp và cách sử dụng bằng chứng 4
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÁC THUẬT TOÁN 5
2.1. Phạm vi và ký hiệu 5
2.2. Biểu diễn bàn cờ và bất biến cập nhật 5
2.3. Luật thắng Freestyle và quét theo nước cuối 7
2.4. Random và rule-based: đường cơ sở 9
2.5. Hàm đánh giá heuristic và terminal utility 11
2.6. Sinh ứng viên Chebyshev và giới hạn mềm 14
2.7. Move ordering: sắp thứ tự duyệt 16
2.8. Minimax: tìm kiếm đối kháng 17
2.9. Alpha-Beta: cắt tỉa trên cùng cây 19
2.10. Iterative deepening và kiểm soát ngân sách 21
2.11. Zobrist hashing và cập nhật tăng dần 23
2.12. Transposition table và ba loại cận 24
2.13. TypeScript legacy và khác biệt với engine hiện tại 26
2.14. Sáu cấu hình và đường dữ liệu thực tế 27
2.15. Bằng chứng thực nghiệm và giới hạn suy luận 29
CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 31
3.1. Tác nhân và ca sử dụng 31
3.2. Yêu cầu phi chức năng và ranh giới 34
3.3. Kiến trúc tổng thể 35
3.4. Thiết kế dữ liệu bàn cờ và lịch sử 36
3.5. Trạng thái ván và chuyển trạng thái 37
3.6. Thiết kế bất đồng bộ và xử lý sai lệch 38
3.7. Thiết kế hiển thị bàn, tọa độ và tương tác 40
3.8. Ngôn ngữ, âm thanh và tích hợp bên ngoài 41
3.9. Các kịch bản cần kiểm chứng 42
CHƯƠNG 4. TRIỂN KHAI HỆ THỐNG 44
4.1. Môi trường và tổ chức mã 44
4.2. Triển khai nhận và cập nhật nước đi 44
4.3. Triển khai điều phối máy và kết quả 45
4.4. Triển khai bàn cờ và thành phần bao quanh 46
4.5. Ứng dụng trình bày và figure inspector 50
4.6. Pyodide và bàn tương tác của trang trình bày 52
4.7. Ranh giới giữa triển khai và kiểm chứng 55

CHƯƠNG 5. KẾT QUẢ VÀ ĐÁNH GIÁ..... 56
5.1. Phạm vi đánh giá và nguồn dữ liệu..... 56
5.2. Kết quả kiểm thử engine và luật chơi..... 56
5.3. Đối chiếu WASM, Worker và giao diện..... 57
5.4. So sánh Minimax và Alpha-Beta trên engine ứng dụng..... 58
5.5. Thực nghiệm thuật toán trên reference core..... 59
5.6. Mục tiêu và cấu trúc các tập dữ liệu..... 60
5.7. Bảng số liệu được lưu cho một fixture benchmark..... 61
5.8. Khảo sát 100 ván Rule-Based và Random..... 62
5.9. Truy nguyên biểu đồ và dữ liệu minh họa..... 63
5.10. Cách dùng evidence và bảo toàn khả năng tái lập..... 64
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN..... 65
6.1. Kết luận..... 65
6.2. Hạn chế..... 65
6.3. Hướng phát triển..... 65
TÀI LIỆU THAM KHẢO..... 67
PHỤ LỤC A. TỔNG HỢP CÔNG VIỆC THEO TUẦN..... 69
A.1. Tuần 1: mô hình và phạm vi đề tài..... 69
A.2. Tuần 2: thao tác trạng thái và kiểm tra kết thúc..... 69
A.3. Tuần 3: giao diện và vòng đời ván chơi..... 69
A.4. Tuần 4: hình thế, lượng giá và baseline..... 69
A.5. Tuần 5: Minimax và cây trò chơi..... 70
A.6. Tuần 6: Alpha-Beta..... 70
A.7. Tuần 7: ứng viên, thứ tự và cấu hình độ khó..... 70
A.8. Tuần 8: move ordering..... 70
A.9. Tuần 9: Zobrist và bảng chuyển vị..... 71
A.10. Tuần 10: iterative deepening và kế hoạch đánh giá..... 71
PHỤ LỤC B. KIỂM KÊ THƯ MỤC VÀ HỒ SƠ LIÊN QUAN..... 73
B.1. Source, sản phẩm và hồ sơ theo tuần..... 73
B.2. File gốc, mẫu và giới hạn của bản kiểm kê..... 73
PHỤ LỤC C. HƯỚNG DẪN CHẠY VÀ HỒ SƠ BÀN GIAO..... 75
C.1. Chạy ứng dụng chính..... 75
C.2. Tái chạy nghiên cứu và giới hạn dependency..... 75
C.3. Danh mục toàn bộ thư mục gốc..... 76
C.4. Hình ảnh và khả năng cập nhật..... 77
C.5. Phần còn thiếu và điều kiện chốt bản nộp..... 77

DANH MỤC HÌNH

Hình 1.1. Biểu diễn bàn 15 × 15 và chỉ số theo hàng 3
Hình 2.1. Bất biến khôi phục trạng thái sau place và undo 7
Hình 2.2. Bốn trục kiểm tra thắng quanh nước cuối 9
Hình 2.3. Các nhánh ưu tiên của bot Rule-based 11
Hình 2.4. Chuỗi liên tiếp và trọng số trong engine hiện hành 14
Hình 2.5. Ảnh hưởng bán kính và biên bàn tới tập ứng viên 15
Hình 2.6. So sánh hai thứ tự duyệt trên cùng membership 17
Hình 2.7. Ví dụ Minimax với hai tầng quyết định 19
Hình 2.8. Cây Alpha-Beta với một lá được cắt 21
Hình 2.9. Iterative deepening giữ kết quả vòng hoàn tất gần nhất 22
Hình 2.10. Cập nhật và hoàn tác Zobrist bằng XOR 24
Hình 2.11. Ba loại kết quả lưu trong bảng chuyển vị 26
Hình 2.12. Hai tuyến tactical và Rust được nhập tại controller 29
Hình 3.1. Trách nhiệm và liên hệ giữa controller, core, Pixi và AI 35
Hình 3.2. Trạng thái ván và các nhánh yêu cầu hoặc hủy AI 37
Hình 3.3. Biến đổi tọa độ và kiểm tra biên 40
Hình 4.1. Giao diện hai người chơi trên bản build local 46
Hình 4.2. Ván đấu máy sau một lượt người và một lượt bot 47
Hình 4.3. Giao diện tại viewport 390 × 844 48
Hình 4.4. Thông tin của một lần tìm kiếm Hard trong browser local 49
Hình 4.5. Chi tiết khối App và trang điều phối trong sơ đồ present 50
Hình 4.6. Chi tiết hooks và domain trong sơ đồ present 51
Hình 4.7. Tái dựng ví dụ Chebyshev năm quân của present 54
Hình 5.1. Số node của cùng cấu hình tìm kiếm trên engine game 58
Hình 5.2. So sánh số node trên reference core 59
Hình 5.3. Benchmark native và TypeScript trên một fixture midgame 61

DANH MỤC BẢNG

Bảng 2.1. Trọng số các chuỗi trong engine game 12
Bảng 2.2. Cấu hình sáu mức bot 27
Bảng 3.1. Các yêu cầu chức năng được đối chiếu với thành phần nguồn. 32
Bảng 3.2. Kịch bản và kết quả mong đợi để thiết kế kiểm thử; trạng thái chạy thực tế cần đối chiếu với hồ sơ kiểm thử riêng. 42
Bảng 5.1. Dữ liệu benchmark native và TypeScript 61
Bảng C.1. Kiểm kê các thư mục gốc theo vai trò 76

CHƯƠNG 1. GIỚI THIỆU

1.1. Bối cảnh và lý do thực hiện

Caro là một trò chơi đối kháng có luật cơ bản đơn giản nhưng tạo ra nhiều khả năng lựa chọn sau mỗi nước đi. Với bàn cờ 15 × 15, ngay trạng thái ban đầu đã có 225 vị trí có thể đặt quân. Một chương trình chơi với máy cần chọn một nước trong số các lựa chọn đó, dự đoán phản ứng của đối thủ, đồng thời giữ giao diện phản hồi trước thao tác của người chơi. Đề tài vì vậy kết hợp hai mặt: mô hình hóa bài toán tìm kiếm và xây dựng một ứng dụng tương tác có trạng thái, vòng đời và xử lý bất đồng bộ rõ ràng.

Bên cạnh ứng dụng chơi, workspace còn có engine Rust, các chương trình tham chiếu và tài liệu theo tuần, cùng ứng dụng trình bày present. Những thành phần này phục vụ các mục đích khác nhau. Ứng dụng CaroGame xử lý tương tác của ván cờ; engine cung cấp logic tính nước đi; các mô hình reference phục vụ kiểm tra và so sánh; trang trình bày kết hợp văn bản, hình, ví dụ Python và bàn chơi minh họa để giải thích quá trình phát triển. Một báo cáo đầy đủ cần mô tả quan hệ giữa chúng và giới hạn của từng nguồn, thay vì giả định mọi đoạn minh họa hoặc phép đo là hành vi hiện tại của cùng một sản phẩm.

Về bài toán kỹ thuật, nếu AI xử lý một thế cờ trong thời gian đáng kể, gọi tính toán trực tiếp trong luồng giao diện có thể làm chậm các thao tác hiển thị. Ngược lại, chuyển xử lý sang Worker tạo thêm vấn đề: người chơi có thể bắt đầu ván mới, hoàn tác hoặc nhận sự kiện tạm dừng trong lúc lời giải cũ chưa trả về. Kết quả tính đúng cho một thế cờ trước đó không còn hợp lệ nếu được áp dụng vào ván khác. Vì vậy thiết kế không chỉ cần tạo ra nước đi, mà còn cần xác định chủ sở hữu trạng thái, thứ tự sự kiện và điều kiện cho phép ghi kết quả.

Báo cáo dùng từ Caro để gọi sản phẩm, nhưng luật hiện tại được xác định cụ thể là Freestyle Gomoku: năm hoặc nhiều hơn năm quân liên tiếp cùng màu trên một trục là thắng. Việc xác định biến thể là cần thiết vì các trò chơi cùng tên có thể dùng quy định khác về chính xác năm quân hoặc hai đầu bị chặn. Kết luận về hệ thống trong báo cáo dựa trên mã và hồ sơ tại workspace được kiểm tra, không phải khẳng định về mọi phiên bản đã từng xuất hiện trong lịch sử.

1.2. Mục tiêu và câu hỏi của đồ án

Mục tiêu chung là nghiên cứu và trình bày một hệ thống Caro trên web có người chơi, bàn cờ, luật ván đấu và đối thủ máy, đồng thời giải thích được dữ liệu, thuật toán và các ranh giới giữa giao diện với engine. Mục tiêu này được cụ thể hóa theo các câu hỏi sau:

1. Bàn cờ, lịch sử nước đi và trạng thái ván được lưu thế nào để kiểm tra luật, cập nhật giao diện và hoàn tác?

2. Hệ thống nhận thao tác qua bàn vẽ, đổi sang tọa độ ô và phản hồi thắng, hòa hoặc từ chối nước không hợp lệ ra sao?

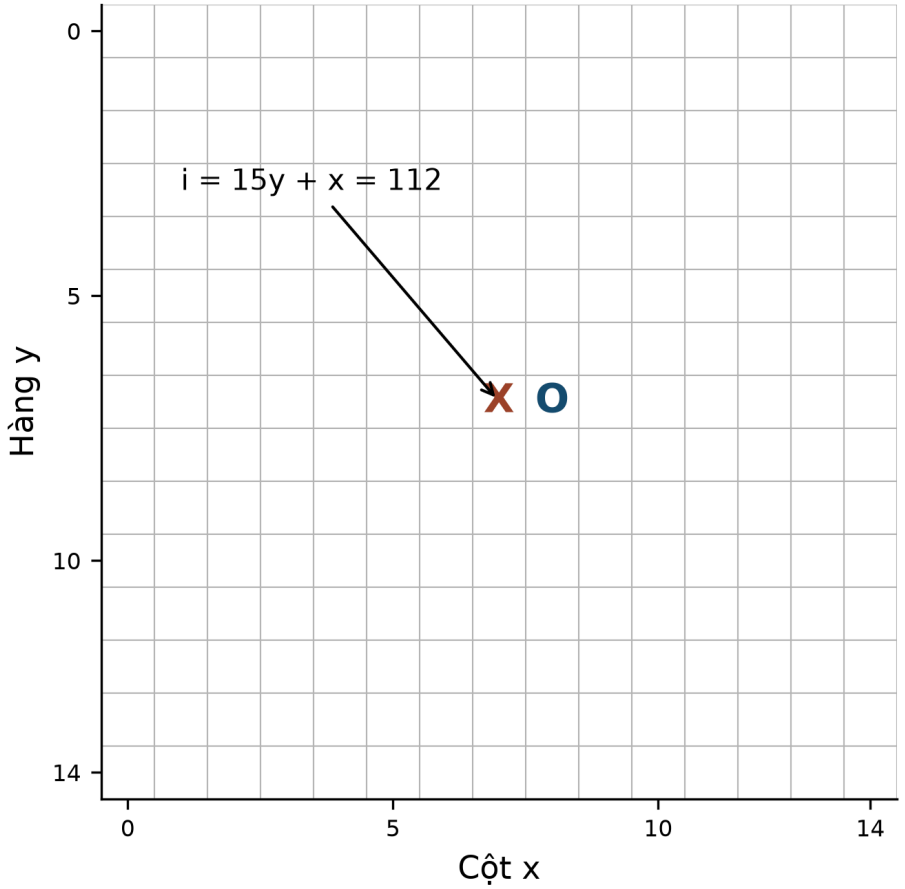
3. Máy nhận thế cờ và trả nước đi như thế nào; điều gì xảy ra nếu yêu cầu lỗi, bị hủy hoặc đã lỗi thời?

4. Các thuật toán tìm kiếm và hàm đánh giá giúp giảm phạm vi tính toán như thế nào, và đâu là giới hạn của các phép đo đã có?

5. Ứng dụng trình bày hỗ trợ diễn giải mã, hình và ví dụ thực nghiệm như thế nào; nội dung minh họa nào cần đối chiếu lại với source hiện tại?

Các mục tiêu có thể kiểm tra trên source bao gồm tái dựng luồng người chơi đặt quân, luồng máy trả kết quả, xử lý hoàn tác và các thành phần trình bày. Các mục tiêu về chất lượng trải nghiệm thực tế, sức chơi tương đối giữa các mức khó, thiết bị thật và tích hợp host đòi hỏi kiểm thử hoặc dữ liệu riêng. Không có cơ sở để mặc nhiên biến khả năng được lập trình thành kết quả đã được người dùng hoặc đơn vị triển khai chấp nhận.

1.3. Đối tượng, phạm vi và sản phẩm khảo sát



Hình 1.1. Biểu diễn bàn 15 × 15 và chỉ số theo hàng

Sơ đồ giải thích dựng từ mã nguồn. Nguồn: board.rs; core/types.ts.

Phạm vi chính gồm ứng dụng CaroGame, mã kết nối và binary WebAssembly, ứng dụng trình bày present và các nghiên cứu thuật toán. Các nguồn công khai trong báo cáo được định danh bằng commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc của tducn110/basicproject. Cây CaroGame/src tương ứng với phần source chung đã đối chiếu trong checkout 10_caro của tác giả. Rust có mã baseline và harness công khai, cùng trích đoạn và manifest của bản sửa; toàn bộ cây engine sửa dùng trong một số phép đo và thư viện caro-core-reference chưa được công bố đầy đủ. Các quan sát chỉ có ở bản làm việc và log local được ghi rõ là hồ sơ tác giả, không coi đường dẫn trên máy là nguồn GitHub mà người đọc có thể tái lập. [1] [2] [3]

Ứng dụng trò chơi có hai chế độ `1v1` và ai, với bàn hữu hạn 15 × 15. Báo cáo mô tả quy tắc Freestyle và các mức độ khó đã cấu hình. Phạm vi không gồm chứng minh

máy chơi tối ưu toàn cục, hệ thống tài khoản do đồ án tự xây dựng, xếp hạng sức chơi đã được hiệu chỉnh qua giải đấu, hay nghiệm thu thiết bị và dịch vụ production. Tích hợp Wink được trình bày như một lớp tiếp nhận sự kiện và dịch vụ bên ngoài có trong source, không mặc nhiên xem như đã kiểm thử trên host.

Đối với số liệu và hình trong tài liệu tuần, mỗi kết quả phải đi kèm bản triển khai, ca thử, dữ liệu và môi trường tương ứng. Một benchmark native của caro-core-reference không phải phép đo hiệu năng WebAssembly trong trình duyệt. Trang present cũng có các nhãn lịch sử, con số minh họa và code trích xuất của những giai đoạn trước; việc trang hiển thị một con số chỉ xác nhận nội dung trang, không xác nhận con số đó là kết quả hiện tại của ứng dụng. [4]

1.4. Phương pháp và cách sử dụng bằng chứng

Phương pháp thực hiện gồm bốn bước. Trước hết, kiểm kê các sản phẩm liên quan và đọc file cấu hình, mã nguồn, báo cáo, log hoặc dữ liệu thực nghiệm sẵn có. Tiếp theo, lần theo chuỗi đầu vào, xử lý, trạng thái, sự kiện, đầu ra và phản hồi. Sau đó, giải thích thuật toán bằng công thức, ví dụ và chỉ rõ dữ liệu đầu vào cùng giả thiết. Cuối cùng, đối chiếu yêu cầu với kiểm thử hoặc kết quả đã ghi nhận để phân biệt hành vi được mô tả trong mã với hành vi đã quan sát khi chạy.

Mỗi loại nguồn trả lời một câu hỏi khác nhau. Source xác định cấu trúc và nhánh logic của snapshot đang đọc; tài liệu chính thức giải thích khái niệm công nghệ; log và fixture xác nhận những trường hợp đã thực thi; báo cáo cũ cung cấp bối cảnh nhưng cần đối chiếu khi source thay đổi. Một phép tính giới hạn số nút là phân tích lý thuyết. Một giá trị thời gian là kết quả đo của một cấu hình và workload cụ thể. Hai loại kết luận này không thay thế nhau.

Các kịch bản quan trọng được thiết kế theo điều kiện trước, thao tác và kết quả mong đợi để tạo cơ sở kiểm thử. Bảng kịch bản trong Chương 3 là mô tả yêu cầu và điều kiện cần xác minh, không tự chứng minh chúng đã chạy thành công. Kết quả thực tế, nếu có bằng chứng, được trình bày riêng trong chương đánh giá. Các thiếu sót về danh tính, thông tin môn học, nguồn đo hoặc nghiệm thu phải được đánh dấu và bổ sung thay vì suy diễn.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÁC THUẬT TOÁN

2.1. Phạm vi và ký hiệu

Chương này trình bày từ biểu diễn bàn cờ tới cách máy lựa chọn nước đi. Nội dung được đối chiếu với ba cơ sở mã khác nhau, vì cùng một tên thuật toán không có nghĩa ba chương trình thực hiện cùng một quy trình. Thứ nhất, **engine Rust của ứng dụng** nằm ở gomoku-main/engine/src; gói WebAssembly của engine được dùng bởi ứng dụng basicproject/CaroGame. Thứ hai, **TypeScript legacy** nằm ở CaroGame/src/game/bot; mã này có các thuật toán tìm kiếm riêng nhưng AIController hiện sử dụng GomokuWasmEngine, nên không mô tả legacy là đường xử lý mặc định. Thứ ba, **Rust reference** nằm tại caro-core-reference/src, là thư viện phục vụ nghiên cứu và thử nghiệm native. Những kết quả trong reference không tự chứng minh cùng hành vi trong browser hoặc trong engine Rust được tích hợp.

Trang thuyết trình present chứa các thành phần Week5Minimax, Week6AlphaBeta, Week7CandidatesAndTiers, Week7Production, Week8To10AdvancedAI và bàn minh họa InteractiveCaroBoard. Đặc biệt, present/src/utils/caroEngine.ts có hàm chọn nước, lượng giá và luật riêng cho demo. Số hiển thị cố định trong trang hoặc biểu đồ là nội dung trình bày, cần đối chiếu với source và dữ liệu thí nghiệm gốc trước khi dùng làm bằng chứng. Ví dụ “100/100”, “60 FPS” hay bảng 8 MB được nêu ở một số thành phần thuyết trình không được coi là kết quả đã xác nhận cho engine game hiện tại.

Đặt kích thước bàn N = 15, điều kiện thắng K = 5. Tọa độ (x, y) trong Rust lần lượt là cột và hàng; TypeScript dùng (row, col) tương ứng với (y, x). Một **ply** là một lần đặt một quân, còn một cặp lượt người–máy có thể gồm hai ply. Gọi B là bàn hiện tại, p là người đang xét, A(B) là tập nước được thuật toán cho phép và d là số ply tìm kiếm còn lại. Cần phân biệt A(B) chứa toàn bộ ô trống với tập ứng viên được lọc: đổi tập này là đổi cây trò chơi được khảo sát.

2.2. Biểu diễn bàn cờ và bất biến cập nhật

2.2.1. Ma trận 15 × 15

$$i = Ny + x;\ x = i \bmod N;\ y = floor(i / N)$$

Biểu diễn mảng và cách tính offset được đối chiếu với Rust Reference [5] và source engine [1].

Engine game biểu diễn bàn bằng [[Cell; 15]; 15], trong đó mỗi Cell là Empty, Black hoặc White. Với miền tọa độ từ 0 đến 14, B[y][x] chỉ ô ở hàng y, cột x. Có 225 ô; khi mã hóa tuần tự theo hàng, chỉ số một chiều của ô là i = 15y + x. Ví dụ tâm (7,7) có chỉ số 112, góc dưới bên phải (14,14) là 224. Quy đổi ngược bằng x = i mod 15 và y = floor(i/15).

board.rs::Board sở hữu mảng ô và move_count, còn lib.rs::Game sở hữu người tới lượt, người thắng và lịch sử. Phân chia này quan trọng: bàn có thể trả lời ô nào trống, nhưng lịch sử do Game giữ mới xác định nước gần nhất để hoàn tác theo trình tự. Source dùng repr(u8) cho Cell; payload 225 ô gồm 225 giá trị một byte. Kích thước toàn bộ Board còn có bộ đếm và padding. Bản đo native ghi Cell 1 byte, Board 240 byte trong môi trường đã đo; không suy luận đây là kích thước của đối tượng trong mọi đích biên dịch hoặc tổng bộ nhớ WASM.

Bất biến đặt quân là tọa độ nằm trong bàn, quân đặt không phải Empty và ô đang trống. Đặt thành công đổi đúng một ô và tăng move_count thêm một; nếu bị từ chối, bàn không thay đổi. Ngoài điều kiện của Board, Game.play_move còn chặn trạng thái đã thắng và quyết định quân theo lượt. Một ô trống về hình học không đồng nghĩa nước đi được phép sau khi ván đã kết thúc.

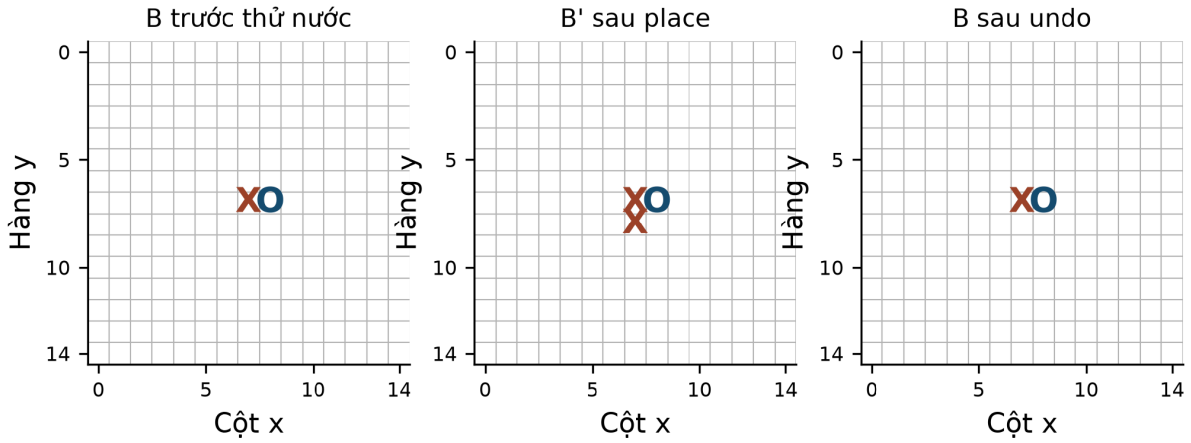
2.2.2. Apply và undo trong tìm kiếm

Đầu vào của Board.place(x,y,p) là tọa độ và quân. Đầu ra là thành công hoặc lỗi; khi thành công, bàn chuyển từ B thành B'. Board.undo(x,y) xóa một ô đang có quân và giảm số lượng; yêu cầu ngoài biên hoặc ô trống trả false và không sửa bàn. Khi dùng trong tìm kiếm, hàm đệ quy chịu trách nhiệm ghép đúng place và undo trước khi đi sang nhánh khác, kể cả khi xảy ra cắt tỉa.

Ví dụ trạng thái B có X ở (7,7) và O ở (8,7), bộ đếm bằng 2. Nhánh giả định đặt X tại (7,8) tạo B' với bộ đếm 3. Sau khi đánh giá nhánh, undo (7,8) phải trả đủ B và bộ đếm 2. Nếu chỉ xóa quân mà không giảm bộ đếm, kiểm tra bàn đầy sẽ sai; nếu chỉ giảm bộ đếm mà không xóa quân, nhánh sau sẽ nhìn thấy nước giả định. Vì vậy phép thử cần so toàn bộ trạng thái thuộc owner, không chỉ một trường.

```
THỬ_MỘT_NƯỚC(board, move, player):
    nếu place(move, player) thất bại: bỏ nhánh
    value ← xử lý trạng thái sau nước đi
    undo(move)
    trả value
```

Trong engine game, search.rs::search sao chép bàn một lần ở gốc, sau đó tìm kiếm dùng bàn làm việc với place/undo; search_in_place cho phép kiểm tra bàn được khôi phục trực tiếp. Sao chép một lần không chứng minh mức tăng tốc cụ thể khi chưa đo. Rust native corpus ghi 820 phép kiểm tra phục hồi bàn làm việc, nhưng vẫn là một tập hữu hạn. Với phép truy cập mảng đã biết tọa độ, place/undo có thời gian O(1) và bộ nhớ phụ O(1); việc tạo bản sao toàn bàn là $O(N^2)$. Source: gomoku-main/engine/src/board.rs::{place,undo} và search.rs::{search,search_in_place}.



Hình 2.1. Bất biến khôi phục trạng thái sau place và undo

Sơ đồ giải thích dựng từ mã nguồn. Nguồn: engine/src/board.rs::{place,undo}.

2.3. Luật thắng Freestyle và quét theo nước cuối

2.3.1. Điều kiện thắng

$$L_d = 1 + f_d + b_d; \quad win \Leftrightarrow \exists d \in D : L_d \geq 5$$

Trong Freestyle Gomoku, người chơi thắng khi có ít nhất năm quân liên tiếp theo chiều ngang, dọc hoặc một trong hai đường chéo. Do đó sáu quân liên tiếp cũng thỏa điều kiện; source không yêu cầu chính xác năm hay loại trừ chuỗi bị chặn hai đầu. Quy ước này cần được viết rõ vì tên “Caro” trong những bối cảnh khác có thể đi

kèm luật khác. Phần mô tả luật trong tài liệu Gomocup cũng phân biệt các biến thể; báo cáo áp dụng luật được code triển khai chứ không suy diễn từ tên trò chơi.

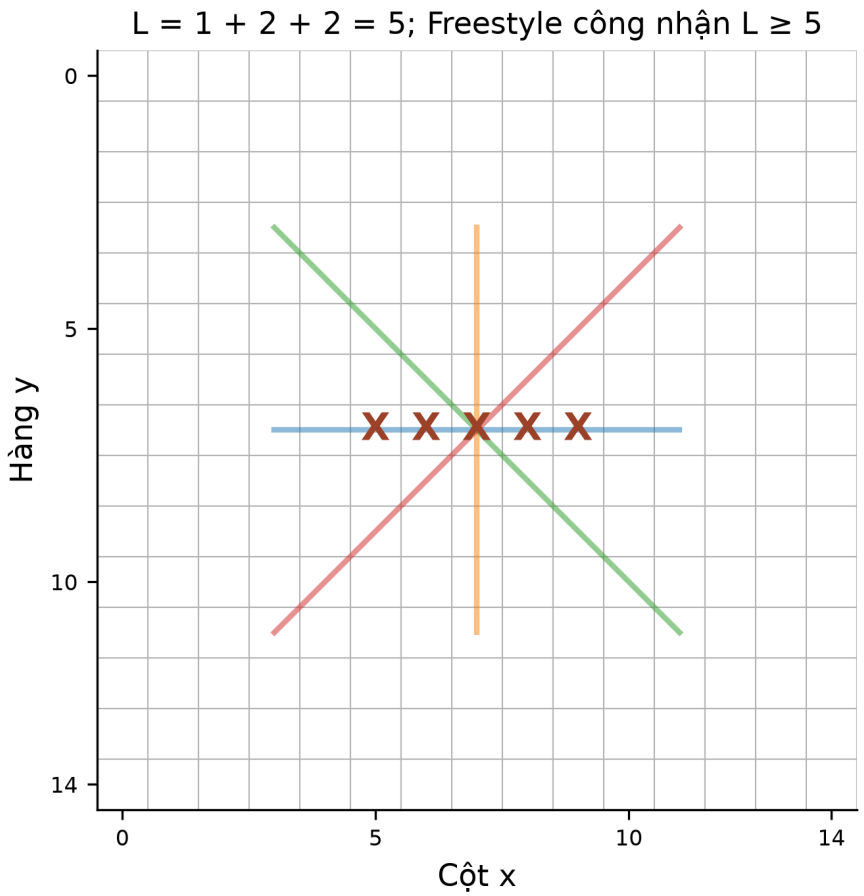
Cho nước vừa đặt m và tập bốn hướng D = {(1,0),(0,1),(1,1),(1,-1)}. Với mỗi hướng d, đếm số quân liên tiếp cùng màu từ m về hai phía, gọi là f và b. Chiều dài chuỗi đi qua m là $L_d = 1 + f + b$. Nước thắng khi tồn tại d mà $L_d \geq 5$. Ví dụ một hàng có dạng X X [X] X X, quân trong ngoặc vừa đặt, thì f = 2 và b = 2; tổng bằng 5. Với X X X [X] X X, tổng lớn hơn hoặc bằng 5, vẫn thắng trong luật này.

2.3.2. Quy trình và độ phức tạp

```
CHECK_WIN(board, last):
    nếu last ngoài bàn hoặc ô trống: trả không có người thắng
    p ← quân tại last
    với mỗi hướng trong bốn trục:
        count ← 1
        quét tối đa bốn ô liên tiếp theo hướng d
        quét tối đa bốn ô liên tiếp theo hướng -d
        nếu count ≥ 5: trả p
    trả không có người thắng
```

Ở mỗi phía, bốn ô là đủ để quyết định một chuỗi ít nhất năm qua ô vừa đặt. Nếu gặp biên hoặc quân khác, việc quét dừng sớm. Cận trên của đoạn quét là 4 trục × 2 phía × 4 ô = 32 lần đọc ô lân cận, cộng việc đọc ô gốc. Với K cố định bằng 5, độ phức tạp là O(1) theo kích thước bàn; nếu K là tham số thay đổi, cận là O(K). Đây là cận số thao tác chứ không phải số lần đọc luôn xảy ra hoặc thời gian chạy đo được.

Giả thiết quan trọng là trước nước vừa đặt, trạng thái chưa kết thúc. Nếu nhận một bàn bất kỳ từ bên ngoài, chỉ kiểm tra nước cuối không đủ chứng minh bàn không chứa chuỗi thắng cũ. Vì vậy Board.check_winner có đường quét toàn bàn tại root; các nút con sử dụng check_winner_at sau nước vừa đặt. Giao diện TypeScript dùng FreestyleGomokuWinRule.evaluate và phương thức collect theo bốn trục. Source: gomoku-main/engine/src/board.rs::{check_winner_at,count_line_through,check_winner}; CaroGame/src/game/core/WinRules.ts.



Hình 2.2. Bốn trục kiểm tra thắng quanh nước cuối

Sơ đồ giải thích dựng từ mã nguồn. Nguồn: board.rs::check_winner_at; WinRules.ts.

2.4. Random và rule-based: đường cơ sở

2.4.1. Random

Random là đường cơ sở nhằm tạo một đối thủ có thể chọn nước mà không tìm kiếm nhiều lớp tương lai. Trong ai.rs::baseline, engine kiểm tra trạng thái kết thúc; nếu ván còn tiếp tục, nó lấy toàn bộ các ô trống bằng legal_moves, sinh số giả ngẫu nhiên từ seed và lấy chỉ số R mod số_ô_trống. Từ bàn chỉ có một quân, tập chọn có 224 ô. Cùng seed và cùng trạng thái tạo cùng kết quả trong fixture, giúp tái lập kiểm thử; runtime WASM nhận seed thông qua nguồn ngẫu nhiên JavaScript.

```
RANDOM(board, seed):
    nếu kết thúc hoặc không có nước: trả không có nước
    L ← danh sách tất cả ô trống
    i ← seeded_u64(seed) mod |L|
    trả L[i]
```

Sinh danh sách cần quét O(N²) ô và tối đa O(N²) bộ nhớ cho danh sách; lấy phần tử sau đó là O(1). Phép modulo có thể tạo sai lệch xác suất nhỏ khi miền số sinh không chia hết số ô; không gọi lựa chọn là phân phối đều hoàn hảo khi chưa kiểm chứng điều đó. Sau khi chọn, source đánh giá trạng thái sau nước đã chọn để trả điểm và thống kê; phép đánh giá này không điều khiển quyết định Random. `completed_depth = 0` vì không tìm kiếm lookahead, nhưng bộ đếm nút có thể gồm gốc và trạng thái sau nước chọn. Demo ở present lại chọn ngẫu nhiên trong tập Chebyshev; hai cách chọn khác nhau và không được gộp thành một.

2.4.2. Rule-based và quét chiến thuật

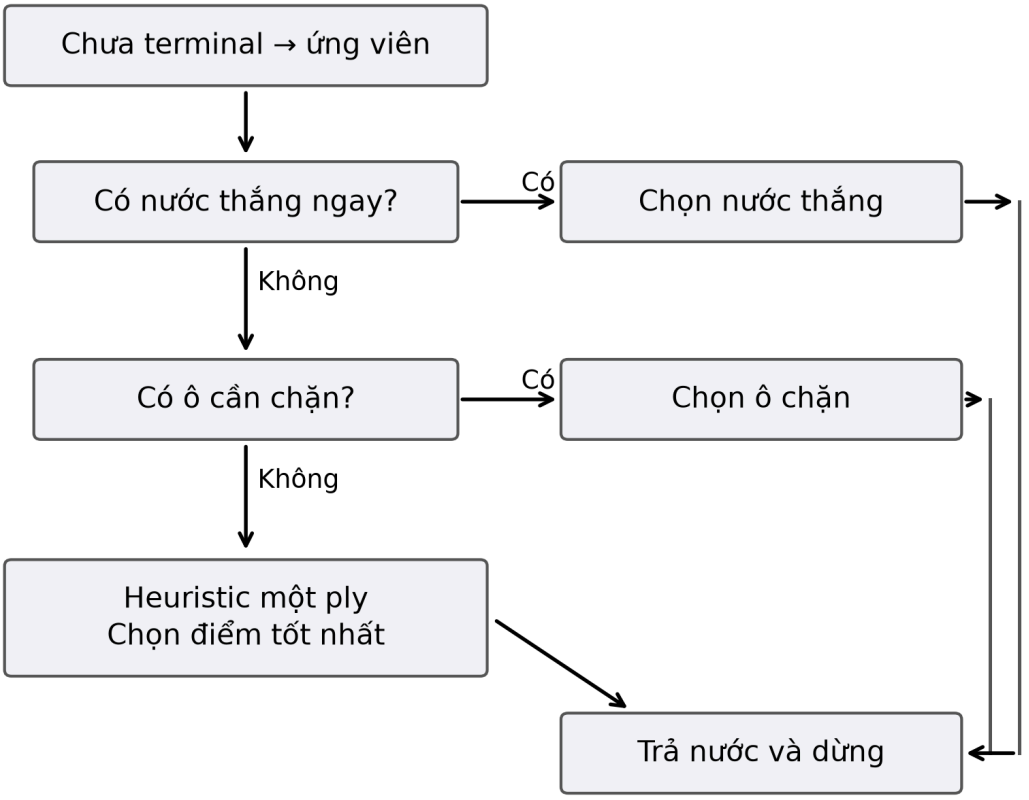
Rule-based sử dụng thứ tự ưu tiên được lập trình trước thay vì đệ quy nhiều lớp. Trong engine game, Normal gọi `select_candidates`, rồi ưu tiên nước thắng của mình; nếu không có thì chọn nước chặn một nước thắng ngay của đối thủ; nếu cả hai không có, đặt thử từng ứng viên và chọn điểm heuristic cao nhất sau một ply. Đầu vào là bàn, người đánh và cấu hình; đầu ra là một nước hợp lệ cùng điểm và bộ đếm.

```
RULE_BASED(board, player):
    C ← chọn ứng viên gần quân
    nếu C có nước thắng ngay: trả một nước thắng
    nếu C có ô đối thủ thắng ngay: trả một ô chặn
    với mỗi m trong C:
        place(m, player); s ← terminal hoặc evaluate; undo(m)
        ghi lại nước có s lớn nhất
    trả nước tốt nhất
```

Ví dụ X đang có bốn quân liên tiếp ở hàng 0, cột 0–3; cột 4 trống. Nếu O đến lượt, đặt O tại cột 4 chặn được mối đe dọa duy nhất ở phía phải vì phía trái nằm ngoài bàn. Nếu bốn quân nằm giữa bàn và cả hai đầu trống, đối thủ có hai nước hoàn thành riêng; chặn một đầu không giải quyết được đầu còn lại. Giữ cả hai tọa độ trong tập ứng viên không có nghĩa một nước có thể chặn đồng thời. Test Rust còn có ví dụ bốn quân `XX_XX` mà quân tại khoảng trống tạo thắng: logic quét hoàn thành có thể nhận diện trường hợp đó dù evaluator không phân loại đầy đủ mọi dạng gián đoạn.

Ở ứng dụng TypeScript, `AIController.requestMove` còn gọi `findForcedTacticalMove` cho Hard, Very Hard, Expert và Master trước khi gửi Worker. Easy và Normal đi thẳng tới chiến lược Rust tương ứng. Nếu prepass trả nước, kết quả ghi `source =`

tactical, độ sâu và node Rust bằng 0; đây không phải một lần Alpha-Beta hoàn tất. Độ phức tạp phụ thuộc cách sinh và thử nước; nói Rule-based là O(1) chỉ vì kiểm tra một ứng viên theo bốn trục sẽ bỏ qua việc duyệt cả tập ứng viên và lượng giá. Source: engine/src/ai.rs::{baseline,choose_move}, engine/src/candidates.rs, CaroGame/src/game/ai/{AIController,TacticalMoveResolver}.ts.



Hình 2.3. Các nhánh ưu tiên của bot Rule-based

Sơ đồ giải thích dựng từ mã nguồn. Nguồn: engine/src/ai.rs::baseline.

2.5. Hàm đánh giá heuristic và terminal utility

2.5.1. Mục đích và nguyên tắc

Tìm kiếm giới hạn độ sâu thường dừng trước khi ván có người thắng. Hàm đánh giá tĩnh ước lượng chất lượng bàn tại điểm dừng, từ góc nhìn người chơi ở gốc. Heuristic là một mô hình gần đúng: điểm lớn hơn được coi là có lợi hơn theo các trọng số đã chọn, không phải xác suất thắng hoặc chứng minh nước đi tốt nhất trong toàn bộ ván.

Engine game eval.rs::evaluate duyệt bàn theo bốn trục. Chỉ khi ô đang xét là đầu của một chuỗi tối đại cùng màu trên trục đó, nó mới đếm độ dài và số đầu mở. Việc kiểm tra ô phía trước giúp tránh tính một chuỗi nhiều lần trên cùng trục. Điểm của các chuỗi thuộc người chơi được cộng, chuỗi đối thủ được trừ; điểm ưu tiên gần tâm cũng được cộng hoặc trừ tương ứng. Ký hiệu R_p là các chuỗi của p, w là trọng số và C là chênh lệch điểm tâm:

$$A(B,p) = \sum_{r \in R_p} w(|r|, o(r))$$

$$D(B,p) = \sum_{r \in R_{opp}} w(|r|, o(r))$$

$$E_{raw}(B,p) = A(B,p) - D(B,p) + C(B,p)$$

2.5.2. Bảng trọng số và ví dụ

Bảng 2.1. Trọng số các chuỗi trong engine game

Chuỗi liên tiếp	Hai đầu mở	Một đầu mở
5 quân trở lên	1.000.000	1.000.000
4 quân	50.000	5.000
3 quân	5.000	500
2 quân	200	50

Chuỗi không nằm trong các trường hợp trên nhận 0 từ pattern_score. Điểm tâm của một quân tại (x,y) là max(0, 14 − |x−7| − |y−7|). Với một quân X ở tâm (7,7) và bàn còn trống, điểm chuỗi đơn bằng 0, điểm tâm là 14; đánh giá từ phía X là 14. Nếu quân thứ hai của X ở (8,7), hai quân tạo một chuỗi ngang hai đầu mở được cộng 200, và điểm tâm là 14 + 13; nếu bỏ qua các đóng góp khác bằng 0, tổng là 227. Các ví dụ này giải thích cách tính của source, không biểu thị một ván hợp lệ đầy đủ giữa hai người.

Evaluator này đếm chuỗi liên tục, không có một taxonomy đầy đủ cho mọi mẫu như XX_XX, fork hoặc đòn nhiều bước. Nhận diện nước thắng ngay trong candidate assessment là logic riêng, không làm cho evaluator tự trở thành bộ nhận diện đầy đủ các pattern gián đoạn. Với số hướng cố định, duyệt các chuỗi tối đại và các ô nằm

trong chúng có cận $O(N^2)$, với bộ nhớ phụ $O(1)$ ngoài bàn. Chi phí search gọi evaluator nhiều lần vẫn phụ thuộc số nút lá.

2.5.3. Phân biệt thắng thực với thế tốt

terminal_utility được kiểm tra trước giới hạn độ sâu. Đặt $M = 100.000.000$, q là số ply đã đi từ gốc, tối đa 225: thắng của gốc có giá trị $M - q$, thua có giá trị $-M + q$, hòa bằng 0. Điểm heuristic bị giới hạn trong $[-H,H]$, với $H = M - 225 - 1 = 99.999.774$. Do đó ngay cả thắng chậm nhất trong giới hạn này vẫn lớn hơn mọi điểm heuristic; thua vẫn thấp hơn mọi điểm heuristic. Trừ q ưu tiên thắng sớm, còn cộng q ở nhánh thua ưu tiên kéo dài nếu buộc phải thua.

Ví dụ chiến thắng tại q = 2 có điểm 99.999.998, thắng tại q = 4 có điểm 99.999.996, nên thuật toán từ góc nhìn gốc thích phương án đầu nếu các điều kiện khác như nhau. Những con số này là giá trị hàm utility, không phải milliseconds hay xác suất.
Source:
gomoku-main/engine/src/eval.rs::{evaluate,count_line,pattern_score,terminal_utility}.

Chuỗi minh họa	Trọng số engine game
.XXXXX.	1.000.000
.XXXX.	50.000
0XXXX.	5.000
.XXX.	5.000
0XXX.	500
.XX.	200
0XX.	50

Hình 2.4. Chuỗi liên tiếp và trọng số trong engine hiện hành

Sơ đồ giải thích dựng từ mã nguồn. Nguồn: engine/src/eval.rs::pattern_score.

2.6. Sinh ứng viên Chebyshev và giới hạn mềm

2.6.1. Miền lân cận

$$d_\infty((x_1, y_1), (x_2, y_2)) = max(|x_1 - x_2|, |y_1 - y_2|)$$

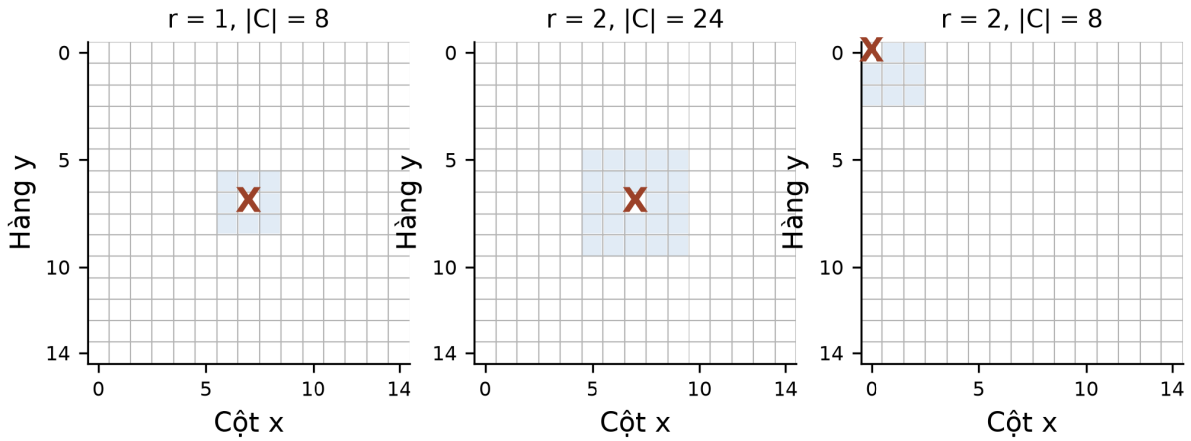
Xét hai ô a và b, khoảng cách Chebyshev là $d_\infty(a,b) = \max(|a_x - b_x|, |a_y - b_y|)$. Với bán kính r, ứng viên là ô trống nằm cách ít nhất một quân đã có không quá r. Với một quân ở tâm (7,7) và r = 1, hình vuông 3 × 3 có 9 ô; trừ ô đã chiếm còn 8 ứng viên. Với r = 2, hình vuông 5 × 5 có 25 ô, còn 24 ứng viên. Trên tổng 224 ô trống sau nước đầu, việc chỉ xét 24 ứng viên giảm số lựa chọn ở gốc khoảng $1 - 24/224 = 89{,}29\%$; đây chỉ là tỷ lệ hình học của ví dụ, không phải tỷ lệ giảm node hay thời gian của mọi ván.

Ở góc (0,0), cửa sổ r = 2 bị cắt bởi biên và gồm 3 × 3 ô, tức 8 ô trống. Nếu có nhiều quân, một ô gần nhiều quân chỉ được thêm một lần nhờ mảng seen. Bàn trống có

chính sách đặc biệt chọn tâm (7,7); bàn đầy trả tập rỗng. Source giới hạn r tối đa 14 và dùng phép cắt biên an toàn.

```
CANDIDATES(board, radius):
    nếu bàn đầy: trả []
    nếu bàn trống: trả [tâm]
    seen ← ma trận false
    C ← []
    với mỗi ô đang có quân:
        duyệt cửa sổ vuông bán kính radius, cắt tại biên
        nếu ô trống và chưa seen: đánh dấu và thêm vào C
    trả C
```

Nếu m là số quân, một cận theo thao tác duyệt là $O(N^2 + m\,r^2)$; bộ nhớ seen và danh sách có cận $O(N^2)$. Tập này là lựa chọn tiết kiệm, không phải tập mọi nước hợp lệ. Source: board.rs::get_candidates.



Hình 2.5. Ảnh hưởng bán kính và biên bàn tới tập ứng viên

Sơ đồ giải thích dựng từ mã nguồn. Nguồn: board.rs::get_candidates; minh họa một quân.

2.6.2. Giữ nước chiến thuật trước soft cap

candidates.rs::assess xét mỗi ô trong bốn trục, đếm quân hai phía của cả người chơi và đối thủ. Nếu đặt vào ô có thể nối ít nhất bốn quân sẵn có thành năm, ô đó là nước thắng; nếu đối thủ có thể hoàn thành năm tại ô đó, đây là ô chặn. Mỗi ô còn nhận điểm xếp hạng nhanh theo số quân liền kề và vị trí tâm. Các nước được sắp theo thứ tự thắng trước, chặn sau, điểm giảm dần, rồi tọa độ chuẩn để phá hòa.

Gọi T là hợp của các ô thắng ngay và ô chặn thắng ngay, c là cap và C là tập ban đầu. Chính sách giữ toàn T, sau đó lấy thêm các ô xếp hạng cao cho tới cap nếu còn chỗ. Có thể viết $C' = T \cup \text{Top_max}(0, c - |T|)(C \setminus T)$, với c được chuẩn hóa tối thiểu 1.

Nếu |T| > c, số ứng viên trả về lớn hơn c, nên đây là cap mềm. Source thực hiện truncate(max(cap,1,tactical_count)). Nếu radius 0 không tạo được lân cận khi bàn còn trống, code dùng tập ô hợp lệ làm fallback.

Ví dụ cap bằng 1 nhưng có hai ô chặn thắng ngay: cả hai được giữ để thuật toán không bỏ một tọa độ chiến thuật trước khi tìm kiếm. Tuy nhiên, như đã xét ở mục rule-based, một nước chỉ chiếm một ô và có thể không cứu được ván. Với k ứng viên thô, đánh giá chiến thuật có chi phí O(kK) cho K tổng quát, K = 5 cố định nên tuyến tính theo k; sắp xếp O(k log k), bộ nhớ O(k). Bật hoặc tắt move ordering sau chọn giữ cùng membership trong chính sách đã kiểm, còn thứ tự duyệt thay đổi.

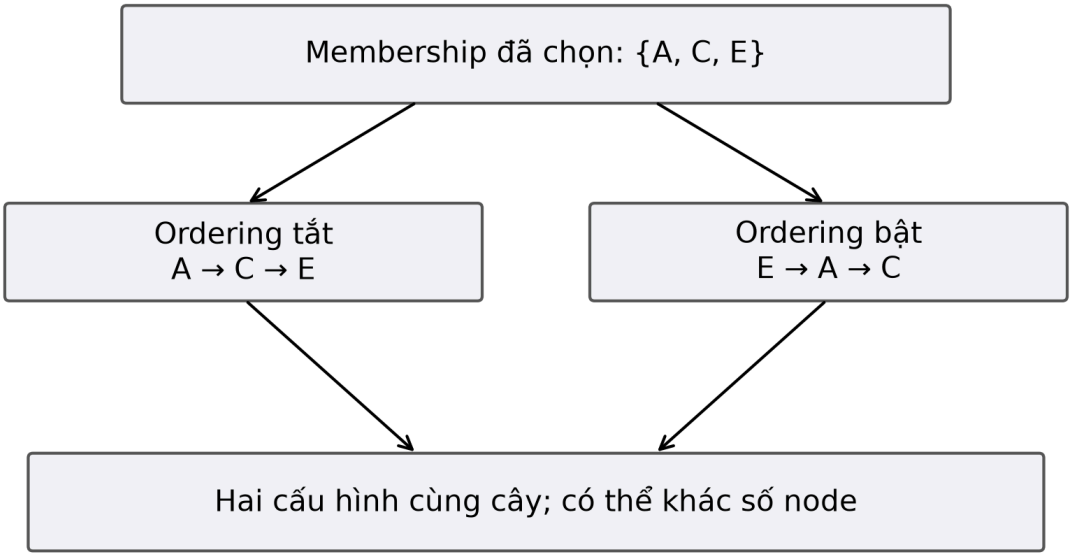
Bản reference cũ từng có cap cứng theo thứ tự khác: fixture cap_loss.json ghi cap 6 loại cả hai ô thắng [7,2], [7,7], trong khi tập không cap có 20 ứng viên. Đây là kết quả của reference riêng và minh họa rủi ro cap, không phải bằng chứng game hiện tại còn lỗi đó. Source game: candidates.rs::{assess,select_candidates}; evidence reference: research/week_05_09/results/cap_loss.json.

2.7. Move ordering: sắp thứ tự duyệt

Sắp thứ tự không tạo ra hoặc xóa nước khi membership được giữ cố định; nó thay đổi nhánh được xét trước. Đối với Minimax thuần, nếu duyệt đầy đủ cùng cây thì thứ tự không đổi giá trị, dù nước được chọn có thể khác khi nhiều nước bằng điểm. Đối với Alpha-Beta, một nhánh tốt được xem sớm có thể tạo giới hạn mạnh hơn, làm tăng khả năng cắt các nhánh về sau. Ngược lại, tính điểm và sắp xếp cũng tiêu tốn thời gian, vì vậy giảm số node không tự bảo đảm giảm latency.

Engine Rust dùng điểm nhanh từ quân liền kề và ưu tiên thắng/chặn trong candidates.rs::assess; đó không phải một lần evaluate toàn bàn cho từng ứng viên. Để tắt thứ tự ưu tiên duyệt mà giữ membership, source chọn tập sau xếp hạng/cap rồi sắp lại theo tọa độ. Vì lựa chọn membership vẫn cần đánh giá, bộ đếm ordering_evaluations có thể khác 0 ngay cả khi traversal ordering tắt. Ví dụ ba nước A, B, C có cùng membership và điểm nhanh tương ứng 10, 50, 30: bật ordering duyệt B, C, A; tắt có thể duyệt theo thứ tự tọa độ. Không thể lấy riêng sự khác biệt về thời gian để kết luận cái nào mạnh hơn về chất lượng nước đi.

Trong reference, search/ordering.rs có chính sách và chi phí đánh giá khác. Việc cùng gọi là move ordering không làm hai implementation tương đương. Corpus game ngày 23/10 có 408 cặp so giá trị khi thay thứ tự trên các cấu hình và lần lặp ghi trong dữ liệu; đây không phải 408 bàn cờ độc lập hay chứng minh mọi trạng thái. Source: engine/src/candidates.rs, engine/src/search.rs; reference: caro-core-reference/src/search/ordering.rs.



Hình 2.6. So sánh hai thứ tự duyệt trên cùng membership

Sơ đồ giải thích dựng từ mã nguồn. Nguồn: engine/src/candidates.rs.

2.8. Minimax: tìm kiếm đối kháng

2.8.1. Đầu vào, đầu ra và công thức

$$V(s,d) \;=\; max_{\mathrm{a}}\, V(T(s,a),d-1) \;\; hoặc \;\; min_{\mathrm{a}}\, V(T(s,a),d-1)$$

Cơ sở tìm trước và lượng giá thế cờ được trình bày trong nghiên cứu của Shannon [6]; hiện thực của đồ án được đối chiếu tại [1], [3].

Minimax mô hình hóa hai phía chọn nước luân phiên: MAX muốn tối đa điểm từ góc nhìn gốc, MIN muốn tối thiểu điểm đó. Đầu vào gồm bàn, người đến lượt, độ sâu d, tập ứng viên, luật và evaluator; đầu ra là điểm cùng một nước ở gốc. Nếu trạng thái kết thúc thì trả terminal utility; nếu d = 0 và chưa kết thúc thì trả heuristic. Với tập A(s) được cố định:

Giá trị được xác định theo bốn trường hợp:

$$V(s,d) = U(s) \text{ nếu trạng thái kết thúc}$$

$$V(s,0) = E(s) \text{ nếu chưa kết thúc}$$

$$V(s,d) = max\{V(T(s,a), d-1) : a \in A(s)\} \text{ khi MAX đi}$$

$$V(s,d) = min\{V(T(s,a), d-1) : a \in A(s)\} \text{ khi MIN đi}$$

T(s,a) là trạng thái sau nước a. Tại mỗi nhánh, source đặt quân, gọi đệ quy với người đối diện, rồi undo. Ở gốc, chương trình lưu tọa độ có giá trị tốt nhất. Nút lá có thể là kết quả thắng/hòa hoặc điểm heuristic khi chạm giới hạn, không nhất thiết là ván đã kết thúc.

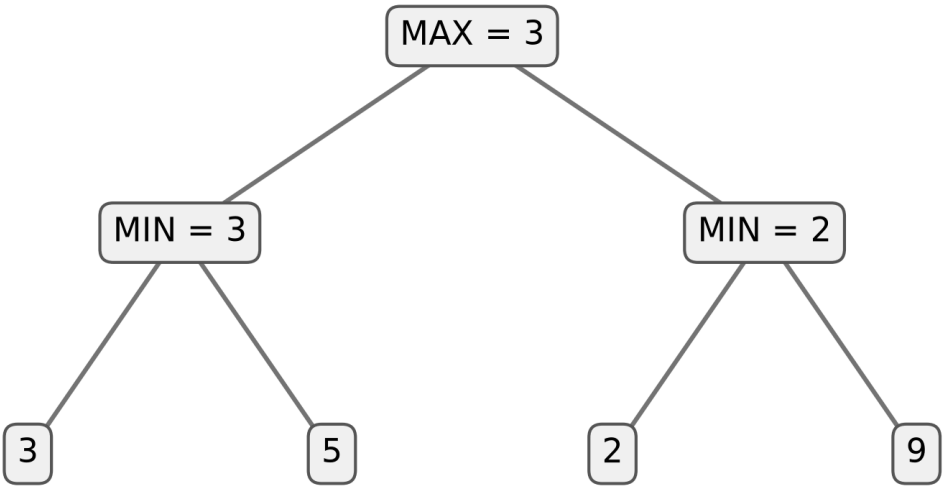
```
MINIMAX(board, player, depth, last):
   nếu trạng thái kết thúc: trả utility
   nếu depth = 0: trả heuristic
   moves ← chọn ứng viên
   best ← −∞ nếu player là MAX, ngược lại +∞
   với mỗi m trong moves:
      place(m, player)
      value ← MINIMAX(board, opponent, depth−1, m)
      undo(m)
      best ← max(best,value) nếu MAX, ngược lại min(best,value)
   trả best
```

2.8.2. Ví dụ và độ phức tạp

Xét cây minh họa có MAX ở gốc, chọn A hoặc B; sau mỗi nước là MIN. Hai lá dưới A có giá trị 3 và 5 nên MIN chọn 3. Hai lá dưới B có giá trị 2 và 9 nên MIN chọn 2. MAX so sánh 3 và 2, chọn A với giá trị 3. Đây là ví dụ cây giả định, không phải số đo từ một fixture bàn cờ. Mục đích của bước MIN là dự đoán đối thủ chọn phản ứng gây bất lợi nhất cho MAX, thay vì chỉ tính nước của máy rồi giả sử đối thủ hợp tác.

Nếu mỗi nút có tối đa b nước và tìm sâu d ply, số nút trong trường hợp đầy đủ tăng theo O(b^d). Đây là mô hình khi b được coi là cận nhánh, không phải lời hứa mỗi tầng có đúng b nước. DFS giữ stack và các vector ứng viên còn sống; nếu mỗi tầng giữ tối đa b phần tử, bộ nhớ phụ khoảng O(db) cộng bàn làm việc. Với bàn N × N, cận bảo thủ là $O(dN^2)$ cho các danh sách ứng viên đang sống. Không cần cấp phát toàn bộ cây thành các object.

Trong engine game, SearchAlgorithm::Minimax tồn tại để đối chiếu trong native search; cấu hình chơi Hard trở lên dùng AlphaBeta. Reference có search/minimax.rs độc lập. Kết quả tính đúng giá trị của cây được khảo sát còn phụ thuộc tính đúng của sinh nước, terminal, evaluator và undo. Tác phẩm của Shannon về chơi cờ bằng máy thảo luận về việc tìm trước và lượng giá trạng thái trung gian; tài liệu đó là cơ sở lý thuyết, không đo hiệu năng của project này.



Hình 2.7. Ví dụ Minimax với hai tầng quyết định

Sơ đồ giải thích dựng từ mã nguồn. Nguồn: Ví dụ toán học tác giả dựng; search.rs.

2.9. Alpha-Beta: cắt tỉa trên cùng cây

2.9.1. Ý nghĩa alpha, beta và điều kiện cắt

$$\alpha \leftarrow max(\alpha,\ v);\quad \beta \leftarrow min(\beta,\ v);\quad cắt\ khi\ \alpha \geq \beta$$

Tính chất cắt tỉa trên cùng cây và vai trò thứ tự được đối chiếu với Knuth và Moore [7].

Alpha là cận dưới của giá trị MAX có thể bảo đảm trong ngữ cảnh đang xét; beta là cận trên của giá trị MIN có thể bảo đảm. Tại nút MAX, sau mỗi nhánh cập nhật alpha = max(alpha,best). Tại nút MIN, cập nhật beta = min(beta,best). Khi alpha ≥ beta,

các anh em còn lại không thể thay đổi quyết định đã được bảo đảm ở nút tổ tiên nên có thể bỏ qua.

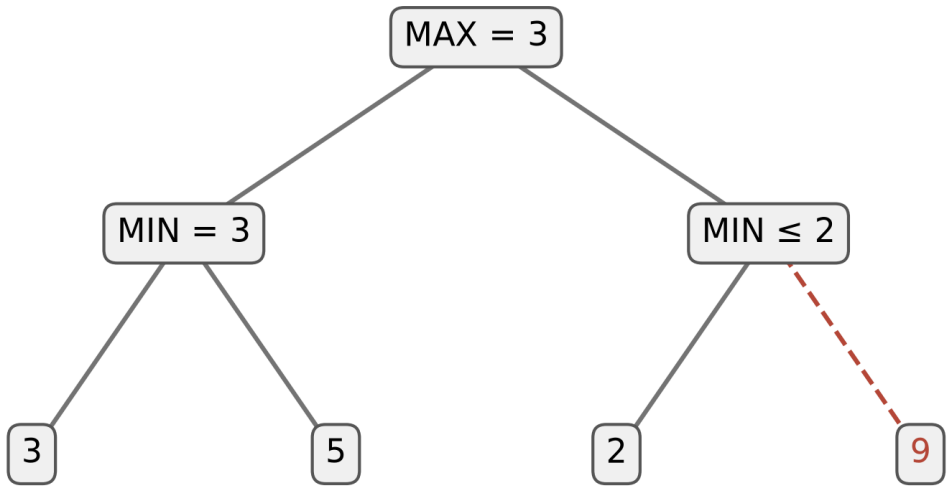
```
ALPHABETA(board, player, depth, alpha, beta):
    nếu kết thúc: trả utility
    nếu depth = 0: trả heuristic
    với mỗi nước trong cùng tập ứng viên:
        place; v ← ALPHABETA(child, đối thủ, depth−1, alpha, beta); undo
        nếu MAX: best ← max(best,v); alpha ← max(alpha,best)
        nếu MIN: best ← min(best,v); beta ← min(beta,best)
        nếu alpha ≥ beta: tăng cutoff; dừng duyệt các anh em còn lại
    trả best
```

Trên cây mục trước, duyệt A trước cho giá trị 3; MAX có alpha = 3. Sang nút MIN của B, lá đầu là 2 nên beta = 2. Vì beta ≤ alpha, MIN đã có thể chọn không lớn hơn 2, trong khi MAX đã có A đạt 3; không cần xem lá còn lại giá trị 9 để chọn giữa A và B. Kết quả gốc vẫn là 3. Nếu thay thứ tự hoặc cấu trúc cây, vị trí và số lần cắt có thể khác.

2.9.2. Điều kiện so sánh và đánh giá

Để kiểm tra Alpha-Beta so với Minimax phải giữ cùng bàn, lượt, độ sâu, luật, terminal utility, evaluator và membership ở mọi nút. So sánh khác membership không chứng minh tính tương đương của cắt tỉa vì một bên đã bỏ nước từ trước. Trong trường hợp tệ, Alpha-Beta vẫn có bậc O(b^d); với giả thiết lý tưởng về cây đều và thứ tự rất tốt, số lá có thể tăng gần O(b^(d/2)). Mô hình lý tưởng không dự báo latency trên một thiết bị.

SearchContext.value của game chia sẻ cùng logic đệ quy cho cả hai chế độ, chỉ bật điều kiện cắt khi cấu hình là AlphaBeta. Thống kê cutoffs đếm sự kiện dừng tại nút, không phải số toàn bộ hậu duệ giả định đã bị loại. Báo cáo Knuth và Moore phân tích tính chất của Alpha-Beta và ảnh hưởng của thứ tự; đây là nguồn lý thuyết, không chứng minh chính sách ứng viên hay evaluator của dự án. engine_verification.json ghi 408 cặp Minimax/Alpha-Beta trong corpus native ngày 23/10, có các lần lặp; phạm vi này không chứng minh tương đương trên mọi trạng thái.



Hình 2.8. Cây Alpha-Beta với một lá được cắt

Sơ đồ giải thích dựng từ mã nguồn. Nguồn: Ví dụ toán học tác giả dựng; search.rs.

2.10. Iterative deepening và kiểm soát ngân sách

2.10.1. Ý tưởng và ví dụ

Iterative deepening tìm lần lượt ở độ sâu 1, 2, … đến giới hạn. Sau mỗi vòng hoàn tất, chương trình lưu kết quả; nếu vòng tiếp theo bị ngắt, nó trả kết quả của vòng hoàn chỉnh gần nhất thay vì lấy một kết quả mới khảo sát dở. Ví dụ vòng sâu 1 trả A, vòng sâu 2 trả B, vòng sâu 3 chưa xong khi hết ngân sách: kết quả có thể dùng là B với độ sâu hoàn tất 2. Nếu vòng đầu chưa hoàn tất thì kết quả hoàn chỉnh có thể không tồn tại; chương trình gọi cần xử lý trường hợp đó.

```
ITERATIVE(board, max_depth, budget):
    last_complete ← none
    với depth từ 1 đến max_depth:
        nếu hết budget: dừng
        result ← tìm kiếm ở depth với budget còn lại
        nếu tìm kiếm bị ngắt: dừng
        last_complete ← result
        cập nhật budget đã sử dụng
    trả last_complete và trạng thái hết budget
```

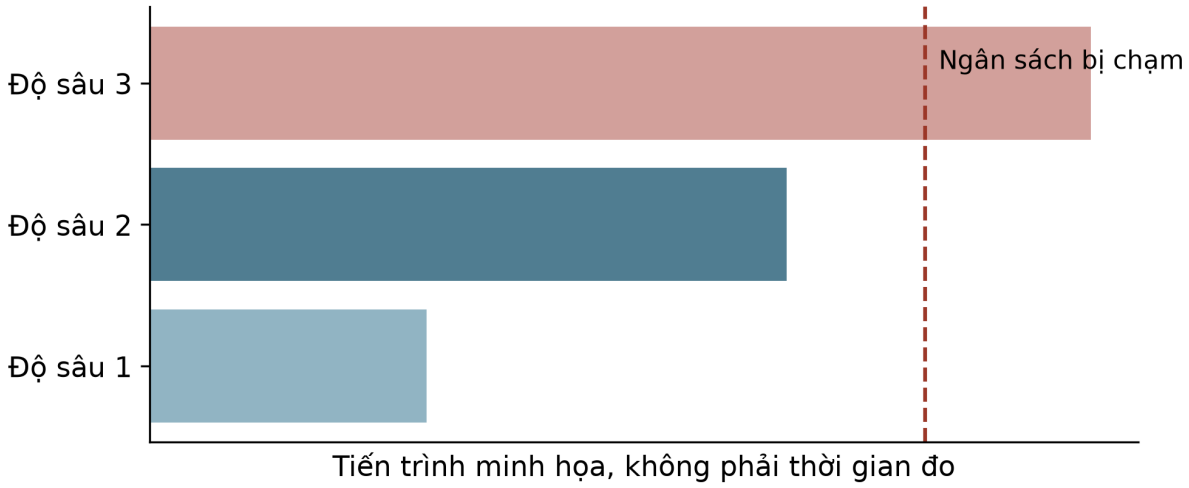
Chi phí có lặp lại các tầng nông; với hệ số nhánh lớn, tầng sâu thường chiếm nhiều công việc hơn nhưng chi phí thực vẫn phụ thuộc thứ tự, cap và TT. Giá trị của

phương pháp là có một kết quả đã hoàn thành khi bị ngắt, không phải bảo đảm chính xác tuyệt đối hay chạy dưới mọi deadline.

2.10.2. Hai implementation và trạng thái

Rust reference search/iterative.rs::iterative_deepening dùng **ngân sách số node** để giữ phép thử xác định; nó không cài sẵn deadline milliseconds của browser. Hàm khởi tạo TT riêng và thực hiện search_optimized_alpha_beta theo các độ sâu. Nếu bị NodeBudgetExhausted, nó trả last_complete và báo hết ngân sách. Thống kê tổng trong implementation ghi nhận các vòng hoàn tất; số node đã tiêu trong vòng bị ngắt không được cộng đầy đủ, nên không dùng trường aggregate như tổng mọi node đã thử.

TypeScript legacy game/bot/search.ts::findBestMove có iterative deepening với performance.now() và deadline; hàm checkAbort chỉ kiểm tra thời gian theo lô node. Vì vậy deadline là cơ chế ngắt theo thời điểm kiểm tra, không phải cam kết kết thúc đúng tuyệt đối tại từng mili giây. try/finally bảo đảm undo khi tìm kiếm bị ngắt ở các nhánh tương ứng. Đây là mã legacy không được AIController gọi làm tuyến mặc định. Engine Rust game hiện dùng cấu hình độ sâu cố định ở mỗi mức, không gắn nhãn Master đã có iterative deepening.



Hình 2.9. Iterative deepening giữ kết quả vòng hoàn tất gần nhất

Sơ đồ giải thích dựng từ mã nguồn. Nguồn: reference search/iterative.rs; sơ đồ nguyên lý.

2.11. Zobrist hashing và cập nhật tăng dần

$$H(B,s) = \bigoplus_{(x,y,p)\in B} Z(x,y,p)\ XOR\ [s = O]\ Z_{turn}$$

$$H' = H\ XOR\ Z(move)\ XOR\ \delta\ Z_{turn}$$

Trong đó δ = 1 nếu người đến lượt đổi sau nước đi; δ = 0 nếu không đổi. Ký hiệu (x,y,p) ∈ B duyệt các ô có quân cùng màu quân tại ô đó; [s = O] bằng 1 khi O đến lượt và bằng 0 khi X đến lượt.

Kỹ thuật mã hóa vị trí được đối chiếu với báo cáo gốc của Zobrist [8].

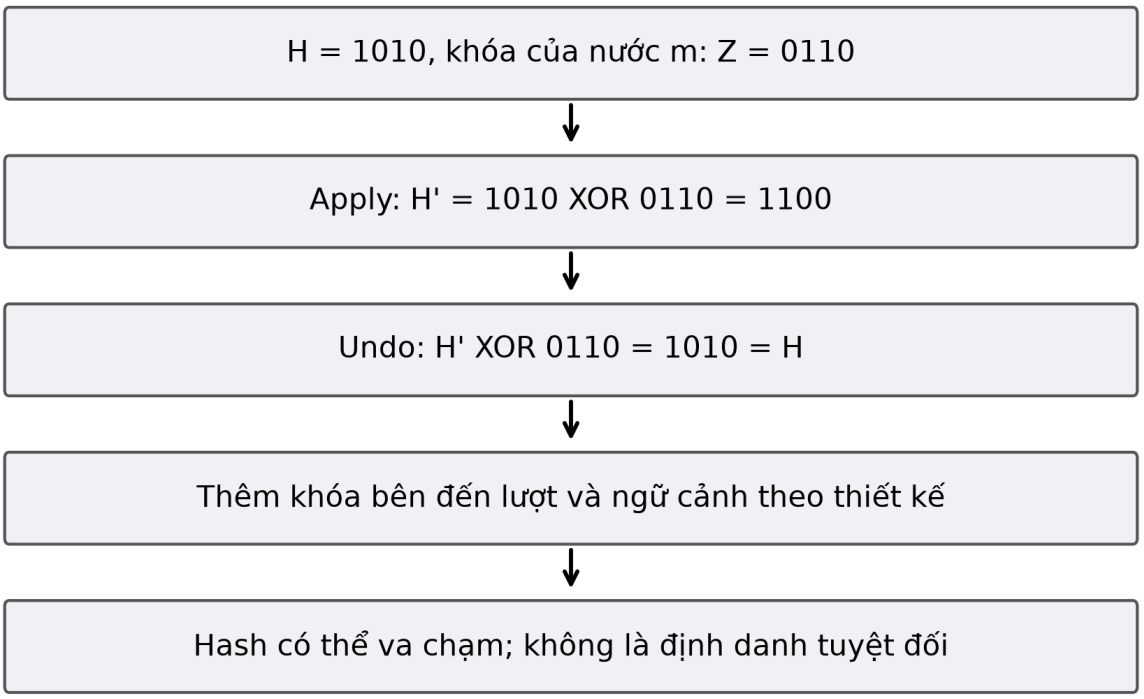
Zobrist hashing ánh xạ trạng thái bàn tới một mã số bằng cách gán mỗi cặp (ô, màu) một khóa giả ngẫu nhiên, rồi XOR các khóa của quân đang có. Có thể thêm một khóa cho người đến lượt để cùng một bàn nhưng khác lượt không bị mặc định xem là cùng trạng thái. Công thức khái quát: H(B,p) = XOR_{ô có quân} Z[ô,màu] XOR Z_lượt nếu quy ước lượt cần bật khóa đó.

Tính chất a XOR k XOR k = a cho phép cập nhật tăng dần khi đặt hoặc rút một quân. Ví dụ dùng số 4 bit chỉ để minh họa: hash ban đầu H = 1010, khóa của quân mới Z = 0110; sau đặt quân, H' = 1010 XOR 0110 = 1100. Khi hoàn tác, 1100 XOR 0110 = 1010. Nếu đổi lượt, XOR thêm khóa lượt ở cả hai thao tác thích hợp. Con số ví dụ không phải khóa thật của chương trình.

Reference zobrist.rs::hash tính hash từ bàn và lượt; after_play cập nhật theo quân mới và chỉ đổi bit khóa lượt khi trạng thái thực sự đổi to_move. Điều này quan trọng vì trong reference một nước kết thúc có thể giữ nguyên người đến lượt. Tính hash lại từ đầu cần O(N²), cập nhật một nước O(1); bảng khóa cần O(N²) phần tử. Cùng hash không chứng minh chắc chắn hai trạng thái bằng nhau vì có khả năng va chạm. Hơn nữa, hash trong reference chưa mã hóa luật, evaluator, chính sách ứng viên hoặc góc nhìn root; cần quản lý ngữ cảnh khi dùng cache.

TypeScript legacy BotBoard dùng hai mảng 32-bit để biểu diễn hai nửa hash, cập nhật khi makeMove và undoMove; đây là implementation riêng. Engine Rust game hiện không có Zobrist trong engine/src. Báo cáo gốc của Zobrist trình bày kỹ thuật mã hóa vị trí và vấn đề lỗi truy xuất; không được diễn giải hash 64 bit là danh tính

không va chạm. Source: caro-core-reference/src/zobrist.rs, CaroGame/src/game/bot/board.ts.



Hình 2.10. Cập nhật và hoàn tác Zobrist bằng XOR

Sơ đồ giải thích dựng từ mã nguồn. Nguồn: reference zobrist.rs; ví dụ 4 bit.

2.12. Transposition table và ba loại cận

2.12.1. Vấn đề lặp lại trạng thái

Trong cây trò chơi, các thứ tự nước đi khác nhau có thể dẫn đến cùng một trạng thái bàn và người tới lượt. Nếu mỗi đường đều tính lại toàn bộ cây con, chương trình lặp công việc. Transposition table (TT) lưu kết quả đã tính theo hash để các lần gặp sau có thể tận dụng. Một bản ghi thường chứa độ sâu đã tìm, điểm, loại cận và nước đi tốt nhất dùng để gợi ý thứ tự. Cần kiểm tra ngữ cảnh, độ sâu và loại cận trước khi sử dụng; giá trị của một lần tìm có cắt tỉa không nhất thiết là giá trị chính xác.

Ở caro-core-reference/src/search/transposition.rs, TTEntry gồm depth, score, bound và best_move, được lưu trong HashMap<u64, TTEntry>. Hàm with_capacity(16384) đặt dung lượng dự trữ ban đầu, không quy định bảng cố định 16.384 phần tử hoặc 8

MB. Khi lưu cùng khóa, entry mới không thay entry cũ có độ sâu lớn hơn; entry sâu hơn hoặc bằng có thể thay thế. Mức chiếm dụng thực tế còn gồm khóa, bucket, phần trống và allocator; kích thước một entry native 12 byte trong phép đo không thể nhân đơn giản để kết luận bộ nhớ toàn bảng.

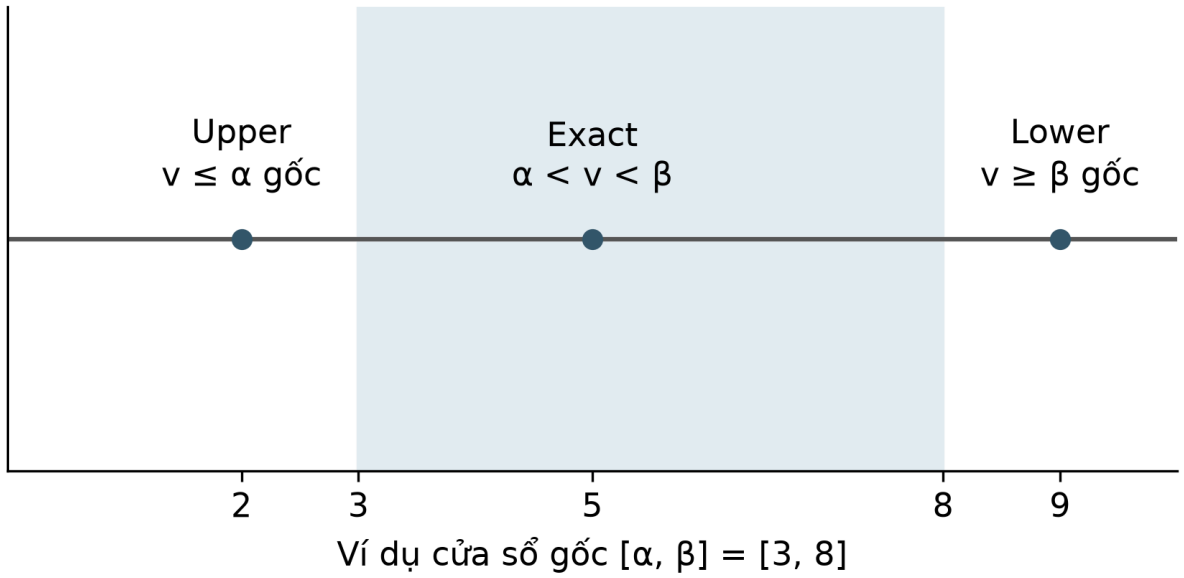
2.12.2. Exact, lower, upper

Giả sử cửa sổ ban đầu là $[\alpha_0, \beta_0]$, kết quả tìm là v. Trong implementation reference, nếu $v \le \alpha_0$, bản ghi mang cận trên Upper: giá trị thực không lớn hơn cận này trong ngữ cảnh tìm. Nếu $v \ge \beta_0$, bản ghi mang cận dưới Lower: giá trị thực không nhỏ hơn cận này. Nếu $\alpha_0 < v < \beta_0$, kết quả là Exact cho độ sâu và cấu hình đã xét. Những cận này gắn với cách cắt tỉa và cửa sổ tìm, không nên đọc mọi số đã lưu như đáp án chính xác.

Ví dụ cửa sổ $\alpha_0 = 3$, $\beta_0 = 8$. Một nhánh trả 2 thì entry Upper cho biết kết quả không vượt 2 theo ngữ cảnh; trả 9 tạo Lower, cho biết giá trị ít nhất 9; trả 5 nằm giữa cửa sổ là Exact. Khi tìm lại với độ sâu yêu cầu không lớn hơn độ sâu đã lưu, Exact có thể trả trực tiếp. Lower dùng tăng alpha; Upper dùng giảm beta. Nếu sau cập nhật alpha ≥ beta, có thể cắt. Nếu entry quá nông để trả điểm, best_move vẫn có thể làm gợi ý duyệt trước, theo code reference.

```
PROBE(key, depth, alpha, beta):
   entry ← tra bảng theo key
   nếu không có: tiếp tục tìm
   preferred ← entry.best_move
   nếu entry.depth đủ sâu:
      nếu Exact: trả entry.score
      nếu Lower: alpha ← max(alpha, entry.score)
      nếu Upper: beta ← min(beta, entry.score)
      nếu alpha ≥ beta: trả cận đã lưu
   tiếp tục tìm, có thể dùng preferred để ordering
```

search/optimized.rs::value lưu lại alpha_original và beta_original trước khi tra TT, dùng chúng để phân loại kết quả sau tìm kiếm. Trong TypeScript legacy, probeTT so hai phần hash, độ sâu và điều kiện cắt; storeTT ghi vào một slot của mảng chỉ số modulo. Đây là hai cấu trúc và quy tắc thay thế khác nhau. Source game Rust fixed-depth không dùng hai implementation đó ở tuyến AI hiện tại.



Hình 2.11. Ba loại kết quả lưu trong bảng chuyển vị

Sơ đồ giải thích dựng từ mã nguồn. Nguồn: reference search/optimized.rs; ví dụ toán.

2.12.3. Ranh giới tính đúng của cache

Một key khớp và entry đủ sâu chưa giải quyết mọi điều kiện nếu điểm phụ thuộc vào người gốc, quy tắc, trọng số, tập nước hoặc ply. Mã reference dùng điểm từ góc nhìn root; khi tái sử dụng bảng sau đổi root, cùng một hash có thể được hiểu trong ngữ cảnh khác. Thí nghiệm `cache_scope_probe.json` tại fixture `clustered` ghi cùng root cold/warm đều trả 0; sau một nước và đổi root, bảng mới trả 0 còn bảng tái sử dụng trả 19.800. Đây là lỗi ranh giới được tái hiện ở bản reference, không phải một kết quả có thể bỏ qua để tuyên bố cache dùng an toàn qua các lượt.

Tài liệu thí nghiệm cũng ghi điểm mate phụ thuộc ply tính từ root chưa được chuẩn hóa cho tái sử dụng qua lượt. Hướng xử lý là tạo bảng mới khi ngữ cảnh không tương thích hoặc thiết kế namespace và chuẩn hóa điểm, sau đó kiểm thử; đây là đề xuất, không phải bản sửa đã hoàn tất. Các phép thử hash và TT có phạm vi hữu hạn, không chứng minh không có va chạm hay đúng trên mọi thế cờ.

2.13. TypeScript legacy và khác biệt với engine hiện tại

Mã `CaroGame/src/game/bot` chứa một engine TypeScript riêng: `BotBoard` lưu bàn trong `Uint8Array` có biên đệm `WALL`, `eval.ts` xây dựng bảng mẫu theo mask nhị phân,

search.ts thực hiện tìm kiếm negamax Alpha-Beta, quiescence, TT và iterative deepening, còn worker.ts nhận yêu cầu chạy tìm kiếm. Biên đệm giúp một số phép quét nhận diện biên bằng giá trị đặc biệt. makeMove và undoMove cập nhật quân, số quân, hash, lượt và các thống kê pattern trên những đường liên quan.

evaluateLine tách một đường ở vị trí quân đối thủ rồi mã hóa những ô thuộc người đang xét thành bitmask; bảng mẫu dùng các phép thử hoàn thành năm để phân nhóm five, open four, four, open three và broken three. Đây là mô hình khác evaluator chuỗi liên tục của Rust game. Ở generateCandidates, legacy tìm vùng gần quân và kiểm các nước ép buộc bằng thử đặt/hoàn tác; không được áp bảng cap mềm Rust một cách máy móc cho mã này.

Tại đáy tìm kiếm, quiescence đánh giá trạng thái hiện tại rồi xét tiếp các nước ép buộc đến giới hạn bổ sung; ý tưởng là giảm nguy cơ dừng giữa một chuỗi chiến thuật đang biến động. Việc mở rộng này tăng lượng công việc và vẫn không bảo đảm tránh mọi sai lệch do dừng ở horizon. Trong findBestMove, deadline và SEARCH_ABORT kiểm soát việc ngắt; TT dùng ba cờ exact/lower/upper và gợi ý nước hash khi sắp xếp. Mô tả này là phân tích source, không phải kết quả benchmark hay bằng chứng rằng người dùng game đang chơi qua legacy engine.

Luồng hiện tại có thể kiểm tra từ game/ai/AIController.ts: đối tượng engine được tạo là GomokuWasmEngine, không phải GomokuEngine từ game/bot/index.ts. Vì vậy việc TS legacy có TT hoặc quiescence không đồng nghĩa sáu mức AI của game Rust hiện tại sử dụng các kỹ thuật đó. Cần duy trì nhãn implementation và đường gọi khi so sánh bất kỳ kết quả nào.

2.14. Sáu cấu hình và đường dữ liệu thực tế

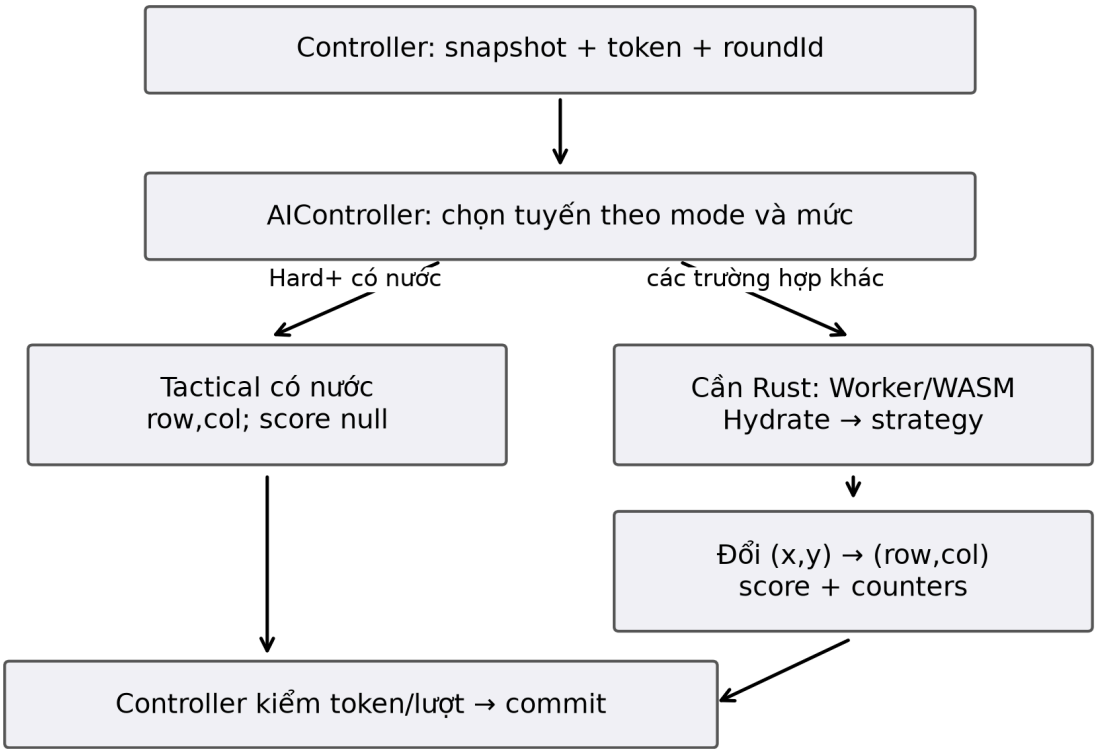
Bảng 2.2. Cấu hình sáu mức bot

Mức	Chiến lược Rust	Độ sâu	Radius / soft cap	Ghi chú
Easy	Random	0	Tất cả ô trống	Seed dùng để tái lập fixture; không có lookahead.
Normal	Rule-based	1	Lân cận, không cap cố định	Thắng, chặn, sau đó heuristic một ply.

Mức	Chiến lược Rust	Độ sâu	Radius / soft cap	Ghi chú
Hard	Alpha-Beta	2	2 / 15	Có thể được TypeScript tactical prepass xử lý.
Very Hard	Alpha-Beta	3	2 / 12	ID API là 5, thứ tự UI không theo ID số.
Expert	Alpha-Beta	4	2 / 10	Cap mềm giữ mọi ô thắng/chặn tức thời.
Master	Alpha-Beta	5	2 / 8	Độ sâu cố định; chưa tích hợp TT hay iterative deepening.

Source mã hóa độ khó ở `engine/src/ai.rs::Difficulty` và ánh xạ giao diện trong `CaroGame/src/game/wasm/contracts.ts`. Cấu hình thay đổi cả chiều sâu lẫn cap, do đó không thể suy thứ hạng sức mạnh chỉ từ số depth. Để kết luận mức này mạnh hơn mức khác cần một giao thức đánh giá riêng: tập thế cờ, đổi bên, seed, số ván, thời gian và cách xử lý hòa phải được công bố cùng kết quả.

Trên tuyến Rust, Worker nhận lịch sử và phát lại từng nước bằng `Game.play_move`; sau đó gọi `ai_move`, chuyển `(x,y)` về `(row,col)` và gửi lại kết quả. Bộ điều khiển ứng dụng xác nhận request, round, pha ván và lượt trước khi áp dụng. Phép đo `elapsedMs` trong `executeSearch.ts` bao gồm hydrate, tính và giải phóng đối tượng, nhưng không bao gồm toàn bộ thời gian tải WASM và truyền thông điệp. Đối với prepass chiến thuật TypeScript, phạm vi đo là `typescript-tactical`; không cộng hoặc so sánh hai giá trị như cùng một phép đo khi thiếu giải thích.



Hình 2.12. Hai tuyến tactical và Rust được nhập tại controller

Sơ đồ giải thích dựng từ mã nguồn. Nguồn: AIController.ts; executeSearch.ts; GameController.ts.

2.15. Bằng chứng thực nghiệm và giới hạn suy luận

Corpus ngày 23/10 của engine game được tạo tại research/current_2026_10_23. engine_verification.json ghi bốn fixture điều khiển, 16 vị trí có seed, 816 dòng chạy native và ba lần lặp thời gian cho cấu hình liên quan. Trong dữ liệu có 408 cặp so Minimax/Alpha-Beta, 408 cặp so giá trị khi thay ordering và 820 lần kiểm tra khôi phục bàn. Một cặp có thể là một lần lặp của cùng cấu hình, nên không mô tả 408 cặp là 408 vị trí độc lập. Các kiểm thử này bao phủ tập hữu hạn và chạy native, không tự chứng minh hiệu năng WASM hoặc mọi trạng thái.

Nhật ký cùng bộ evidence ghi 38 unit và 7 integration tests native, 59 frontend tests trong 9 tệp, trong đó 18 tests dùng actual WASM; tập parity gồm 8 lịch sử reachable và 291 ply được so. Kết quả browser ghi một flow UI → Pixi → Rust Worker → Undo trong HeadlessChrome bằng sự kiện pointer tổng hợp, cùng kiểm tra bố cục ở ba viewport. Những kiểm tra này có ích cho tính nhất quán của phiên bản local đã định

danh, nhưng không thay cho kiểm tra thiết bị cảm ứng thật, host Wink, giải đấu hoặc triển khai production. Log typecheck và build chạy thành công, nhưng build có cảnh báo bundle lớn và cấu hình Vite.

Thí nghiệm research/week_05_09 dùng caro-core-reference riêng. Nó có 544 dòng đo native và các phép đối chiếu, gồm cả fixture cap mất nước thắng và probe TT đổi root. Timer, toolchain, seed, fixture và các ngoại lệ được ghi ở README/JSON; không gộp trực tiếp các con số này với corpus game. Ba lần đo lặp tuần tự không ngẫu nhiên thứ tự cấu hình, không pin CPU và không được báo như kiểm định ý nghĩa thống kê. Vì vậy số thời gian là mô tả của máy và lần đo, không chứng minh mức tăng tốc phổ quát hoặc thời gian phản hồi thiết bị của người chơi.

Các thuật toán trình bày ở đây có vai trò khác nhau: quét thắng xác định luật; heuristic ước lượng trạng thái chưa kết thúc; candidate thay tập nước; ordering thay thứ tự; Alpha-Beta giảm nhánh trên cùng cây; TT và iterative deepening thuộc các implementation đã chỉ rõ. Một kết luận đáng tin cần chỉ ra source, tập nước, độ sâu, seed, môi trường và phạm vi test, tránh quy một tác dụng cho thành phần khác.

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Tác nhân và ca sử dụng

Tác nhân chính là người chơi. Người chơi chọn chế độ, đặt quân, quan sát lượt đi và kết quả, hoàn tác hoặc bắt đầu lại. Khi chơi với máy, hệ thống AI là một thành phần xử lý nội bộ thực hiện lựa chọn nước O; nó không phải người dùng độc lập thao tác trực tiếp lên giao diện. Host Wink, nếu hiện diện, là hệ thống ngoài cung cấp trạng thái pause, mute, ngôn ngữ và một số dịch vụ như bảng xếp hạng. Người đọc trang trình bày là một tác nhân khác: họ mở các phần, chạy ví dụ Python, thao tác bàn minh họa hoặc xem thông tin của hình.

Bảng 3.1. Các yêu cầu chức năng được đối chiếu với thành phần nguồn.

Mã	Yêu cầu hoặc hành vi cần có	Thành phần nguồn liên quan
F01	Chọn chơi hai người hoặc với máy, chọn mức khó cho máy	App.tsx, GameController.setMode/setDifficulty, MatchController
F02	Nhận nước trong giới hạn bàn, không cho đặt vào ô đã có quân	BoardInputController, MoveRules.validateMove, playMove
F03	Xác định thắng theo bốn trục, xác định hòa khi bàn đầy và chưa thắng	FreestyleGomokuWinRule, GameController.finishMove
F04	Giữ lịch sử, chơi lại, tạo ván mới và hoàn tác theo chế độ	GameState, GameController
F05	Tìm nước máy, khóa đầu vào khi máy tính, chỉ chấp nhận phản hồi còn phù hợp	AIController, GomokuWasmEngine, GameController.startAiTurn
F06	Hiển thị trạng thái máy đang tính, thống kê hoặc lỗi và cho thử lại	SearchFeedback, GameSnapshot, retryAiTurn
F07	Hiển thị bàn và quân, cập nhật khi kích thước vùng vẽ thay đổi	CaroGameCanvas, CaroScene, các renderer
F08	Nhận locale và pause từ host khi có tích hợp, vẫn có chế độ standalone	useWinkIntegration, i18n.ts, App.tsx
F09	Xem nội dung giải thích, figure và các ví dụ Python	present/src/App.tsx, FigureInspectorModal, PythonVisualizer

Bảng này mô tả phạm vi các chức năng quan sát được trong mã. Việc đưa F08 vào bảng không xác nhận SDK bên ngoài đang sẵn sàng hoặc các tín hiệu locale, pause đã được kiểm chứng trong mọi môi trường host. Khả năng trả dữ liệu của bảng xếp hạng là vấn đề tích hợp riêng, không được suy ra từ F08. Tương tự, việc hiện có các ví dụ Python không chứng minh kết quả ví dụ đại diện cho benchmark của engine đang được ứng dụng sử dụng.

Tiêu chí hoàn thành và kiểm chứng. Các tiêu chí sau xác định kết quả mong đợi cho từng nhóm chức năng; chúng không khẳng định mọi tình huống đã đạt. Bằng chứng cụ thể và môi trường kiểm tra được trình bày tại Chương 5.

F01: Khi đổi chế độ hai người/máy hoặc chọn mức khó, thiết lập đúng cấu hình ván và vô hiệu yêu cầu AI cũ; mức đã chọn phải được đưa vào yêu cầu tiếp theo. Kiểm tra từng lựa chọn cùng trạng thái bàn và lượt trước, sau chuyển đổi. F02: Nước hợp lệ phải có hàng và cột nguyên trong 0–14, ô trống và đúng người được đi trong ván đang hoạt động, không bị khóa. Chấp nhận đúng một lần và đổi một ô; từ chối ô đã chiếm, ngoài biên, tọa độ phân số, ván tạm dừng/kết thúc, sai lượt hoặc đang khóa AI mà không đổi bàn và lịch sử. Hàm kiểm tra biên ở core hiện kiểm phạm vi nhưng không kiểm tính nguyên, nên việc từ chối tọa độ phân số từ lời gọi trực tiếp là tiêu chí còn mở; đường pointer và adapter WASM không chứng minh mọi caller trực tiếp đều được bảo vệ.

F03: Trên cả bốn trục, kiểm tra chuỗi năm quân và overline từ sáu quân trở lên, có ví dụ sát biên; cả hai phải thắng theo Freestyle. Bàn đầy không có người thắng phải hòa; nước cuối vừa lấp bàn vừa tạo thắng phải ưu tiên kết quả thắng. F04: Kiểm tra Undo trong chế độ hai người và chế độ máy, khôi phục ô, lịch sử, lượt và phase; Replay/New Game đặt lại trạng thái tương ứng và vô hiệu công việc cũ. F05: Khi máy đang đi, khóa đầu vào và chỉ chấp nhận nước hợp lệ của request và round hiện hành; thử phản hồi đến muộn sau Undo, Reset, đổi chế độ và hủy. F06: Hiển thị trạng thái đang tính, kết quả và lỗi/thử lại khi có; gắn chiến lược, bộ đếm và phạm vi thời gian với đúng tuyến đã tạo ra chúng, không bịa số liệu thiếu.

F07: Khi trạng thái hoặc viewport đổi, vẽ đúng bàn hiện tại và giữ ánh xạ tọa độ chính xác. F08: Kiểm locale, pause/resume trong host hỗ trợ thực tế và chế độ standalone, gồm phản hồi cũ và việc khởi động lại lượt AI khi cần; không suy ra nghiệm thu host từ unit test. F09: Mở các hình và ví dụ Python trên trang trình bày độc lập, ghi rõ đây là tài liệu giảng giải thay vì số đo hiệu năng engine chạy thực tế.

Để kiểm chứng thuật toán, đối chiếu Minimax và Alpha-Beta trên cùng thế cờ, luật, góc nhìn root, độ sâu, hàm lượng giá và tập ứng viên; điểm tại root phải bằng nhau, còn nước đi có thể khác khi đồng điểm. Corpus native lưu 408 cặp so sánh có lặp cấu hình: đây là bằng chứng hữu hạn, không phải 408 bàn độc lập hay chứng minh cho mọi thế cờ. Kiểm riêng WebAssembly thực tế và adapter theo binary cùng fixture đã định danh; kết quả native không tự chứng minh hành vi browser/WASM hoặc nguồn build của binary. Các lần kiểm tra local và headless chỉ chứng minh trong môi

trường đã ghi. Kiểm tra thiết bị vật lý, host thực, vấn đề tọa độ phân số và nghiệm thu của người có thẩm quyền vẫn chờ bằng chứng riêng.

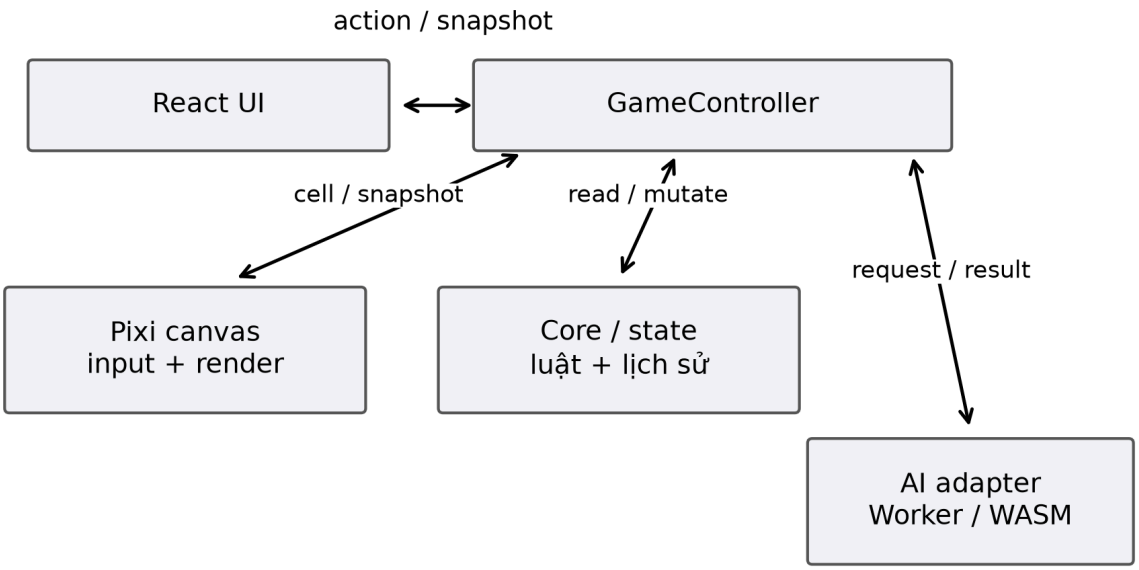
3.2. Yêu cầu phi chức năng và ranh giới

Tính đúng đắn của luật đòi hỏi mỗi nước đã chấp nhận phải nằm trong bàn, đánh vào ô trống và được ghi đúng lượt. Sau mỗi nước, kết quả thắng được xét trước khi kết luận hòa. Tính nhất quán trạng thái yêu cầu lịch sử và các ô đang có quân phản ánh cùng một chuỗi hành động; hoàn tác phải xóa quân tương ứng và khôi phục lượt. Các yêu cầu này có thể kiểm tra bằng test trường hợp biên và kiểm tra state trước–sau.

Khả năng phản hồi đòi hỏi tác vụ máy không chiếm trực tiếp luồng giao diện khi đi qua Worker. Đây là quyết định tách công việc, không phải bảo đảm thời gian phản hồi dưới một ngưỡng. Tính an toàn với bất đồng bộ yêu cầu ngăn kết quả được tính cho ván hoặc lượt trước ghi vào ván hiện tại. Khả năng phục hồi cần thể hiện khi engine trả lỗi, không trả nước, hoặc trả nước không hợp lệ; trong các trường hợp đó, giao diện cần cho người dùng biết trạng thái và có đường thử lại.

Về sử dụng trên màn hình khác nhau, mã có logic tính tỷ lệ bàn theo viewport và ResizeObserver. Điều này hỗ trợ thích ứng kích thước nhưng bản thân source chưa xác nhận khả năng dùng ở mọi thiết bị, trình duyệt hay kích thước chữ. Về bảo trì, state ván, luật, bộ vẽ, input và phần tích hợp host được tách thành các module để có thể thay đổi hoặc kiểm tra từng trách nhiệm. Về khả năng diễn giải, công cụ trình bày có registry cho hình và code minh họa; nội dung trong registry vẫn cần được xác minh lại đối với phiên bản nguồn hiện hành.

3.3. Kiến trúc tổng thể



Hình 3.1. Trách nhiệm và liên hệ giữa controller, core, Pixi và AI

Sơ đồ giải thích dựng từ mã nguồn. Nguồn: CaroGame/src/{app,game,pixi,board}.

Trách nhiệm các module được xác định từ mã ứng dụng và engine [1]; ngữ cảnh Worker tuân theo mô hình giao tiếp của HTML [9]. Định dạng và mô hình thực thi WebAssembly được tham chiếu theo đặc tả công khai [10].

Hệ thống trò chơi được tổ chức theo chuỗi trách nhiệm. App.tsx giữ giao diện và kết nối GameController; controller điều phối lượt, phase, AI và snapshot. GameState giữ quân và lịch sử, còn MatchController giữ phase, chế độ, độ khó, kết quả và mã ván. playMove kết hợp kiểm tra một ô với luật thắng. Phía hiển thị, CaroGameCanvas tạo và đồng bộ CaroScene; scene phối hợp camera, input controller và các renderer. AI có interface riêng để controller gọi; implementation hiện tại là AIController, chuyển tiếp sang GomokuWasmEngine và Worker khi cần.

Luồng người dùng cơ bản là: nhấn hoặc chạm vùng bàn → Pixi nhận pointer → bộ input phân biệt nhấn ô và kéo → chuyển tọa độ sang hàng/cột → callback gọi GameController.playCell → kiểm tra phase và lượt → playMove xác nhận ô và ghi nước → kiểm tra thắng/hòa hoặc đổi lượt → controller phát snapshot → React và

Pixi cập nhật hình cùng phần phản hồi. Khi đến lượt máy, sau nước người chơi controller bắt đầu yêu cầu AI; nhánh trả kết quả cũng đi lại qua playMove, nên vẫn phải qua kiểm tra ô trước khi thay đổi state.

Sơ đồ trách nhiệm ở mức logic:

```
Người chơi → React App → GameController → GameState + MatchController + WinRule
                  |            |
                  |            └→ AIController → Worker → Rust/WASM
                  └→ CaroGameCanvas → CaroScene → Camera/Input/Renderers
Host Wink → useWinkIntegration → App → controller và các thành phần giao diện
```

Mũi tên biểu diễn gọi hoặc trao đổi thông tin, không diễn tả một luồng đồng bộ duy nhất. Worker trả kết quả bất đồng bộ; controller có trách nhiệm cuối cùng kiểm tra kết quả còn phù hợp với state hiện thời. Màn trình bày present có ứng dụng React và state riêng, nên không đặt trong cùng sơ đồ quản lý ván của CaroGame.

3.4. Thiết kế dữ liệu bàn cờ và lịch sử

GameState dùng Map<string, Player> để lưu những ô đã có quân. Khóa được tạo bởi GameState.key từ chuỗi row,col, còn giá trị là X hoặc O. History là một mảng có thứ tự, mỗi phần tử là một Move gồm người chơi, ô và chỉ số nước. Biểu diễn này lưu trực tiếp các ô đã chiếm thay vì buộc giao diện tạo 225 đối tượng có state riêng. Logic toán trong Chương 2 vẫn có thể dùng ma trận 15 × 15 làm mô hình; ma trận lý thuyết và cấu trúc lưu ở TypeScript là hai cách biểu diễn cùng miền tọa độ.

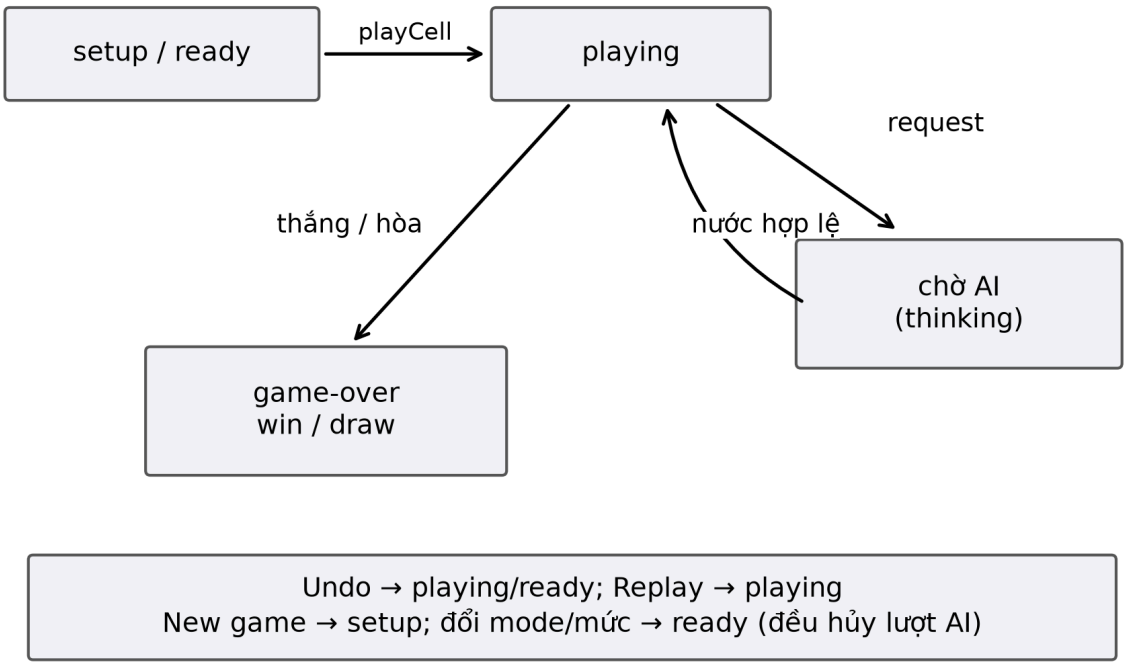
commitMove từ chối ô đã có quân, thêm quân vào Map, đẩy nước vào history và cập nhật biên nhỏ nhất chứa các quân. Việc kiểm tra biên bàn được thực hiện trước đó trong validateMove. getHistory, lastMove, moveCount và forEachStone cung cấp dữ liệu cho controller và renderer. GameController.makeSnapshot sao chép từng nước và tọa độ, duyệt quân để tạo danh sách cho giao diện, đồng thời thêm các thông tin phase, AI và lỗi. React sử dụng snapshot qua useSyncExternalStore thay vì tự sở hữu một bàn cờ cạnh tranh với controller.

Khi hoàn tác, GameState.undoLast lấy nước cuối khỏi history, xóa ô khỏi Map rồi tính lại occupied bounds bằng các nước còn lại. GameController.undo quyết định số ply cần xóa theo chế độ. Trong 1v1, một lần hoàn tác xóa một nước. Trong chế độ máy, nếu nước cuối là O thì xóa cả nước O và nước X trước đó; nếu máy chưa hoàn

thành và nước cuối là X thì xóa một nước đó. Controller hủy lượt AI đang chờ, xóa phản hồi tìm kiếm, xác định lại người đi và đưa match về playing hoặc ready tùy còn lịch sử hay không.

Một hệ quả thiết kế là history không chỉ dùng để hiển thị: nó còn là dữ liệu tái tạo bàn cờ tại ranh giới Worker. Vì vậy thay đổi cấu trúc, thứ tự hoặc hệ tọa độ của history cần được đối chiếu ở cả phía TypeScript và Rust. Quy tắc về người chơi và phase không nằm hoàn toàn trong Map; chỉ nhìn Map không đủ để biết có được đi tiếp hay không.

3.5. Trạng thái ván và chuyển trạng thái



Hình 3.2. Trạng thái ván và các nhánh yêu cầu hoặc hủy AI

Sơ đồ giải thích dựng từ mã nguồn. Nguồn: GameController.ts; GameState.ts; thinking là trạng thái chờ hiển thị.

MatchState định nghĩa bốn phase: setup, ready, playing, game-over. MatchController khởi tạo phase ready, mode 1v1, difficulty normal và roundId 0. Chọn chế độ hoặc độ khó khi không ở phase playing đưa match về ready. Khi playCell nhận thao tác trong setup hoặc ready, controller gọi startRound; hàm này tăng roundId, xóa kết quả cũ và

chuyển sang playing. Sau một nước thắng, finish ghi kết quả và chuyển game-over; nếu bàn đủ 225 quân mà chưa thắng, finishDraw ghi hòa.

newGame tăng roundId và đưa về setup; replay cũng tăng roundId nhưng đưa phase về playing khi đang chơi hoặc đã kết thúc. Controller đồng thời reset bàn, lượt hiện tại và phản hồi AI. Sau Undo, resumeAfterUndo tăng roundId, bỏ winner/draw và đưa về playing nếu còn nước, hoặc ready nếu bàn rỗng. Mã ván được dùng cùng token lượt để xác định yêu cầu bất đồng bộ còn thuộc trạng thái hiện tại hay không.

Tạm dừng là trạng thái song song với phase. setPaused(true) không kết thúc ván: controller vô hiệu hóa lượt AI đang chạy, còn scene dừng vòng render và bỏ qua input. Khi tiếp tục, nếu match còn ở playing, mode là AI và lượt hiện tại thuộc O, controller khởi động lại lượt AI. Phân biệt pause với game-over giúp hệ thống giữ bàn và vị trí lượt khi host gửi resume.

3.6. Thiết kế bất đồng bộ và xử lý sai lệch

AIController.requestMove tạo requestId và ghi thời điểm bắt đầu. Với các mức ngoài Easy và Normal, nó trước tiên thử findForcedTacticalMove trong TypeScript để thắng ngay hoặc chặn thua ngay. Nếu có nước phù hợp, kết quả được trả với nguồn tactical; nếu không, controller đóng gói difficulty, roundId và bản sao history rồi gọi engine. Easy và Normal đi tới chiến lược Rust tương ứng thay vì chạy tactical prepass này. Vì có hai nguồn, thống kê thời gian và số nút phải đọc kèm source và elapsedScope.

GomokuWasmEngine là chủ sở hữu vòng đời Worker. Mỗi yêu cầu có mã riêng và một cặp resolve/reject trong Map pending. Worker được tạo khi cần; thông điệp phản hồi được ghép với mã yêu cầu. Khi cancel hoặc destroy, các promise đang chờ bị reject, Worker được terminate và tham chiếu được xóa. Nếu Worker báo lỗi, các yêu cầu còn lại cũng được reject. Một yêu cầu sau có thể tạo Worker mới; việc có đường khởi tạo lại trong source không thay thế kiểm thử mọi lỗi tải tài nguyên thực tế.

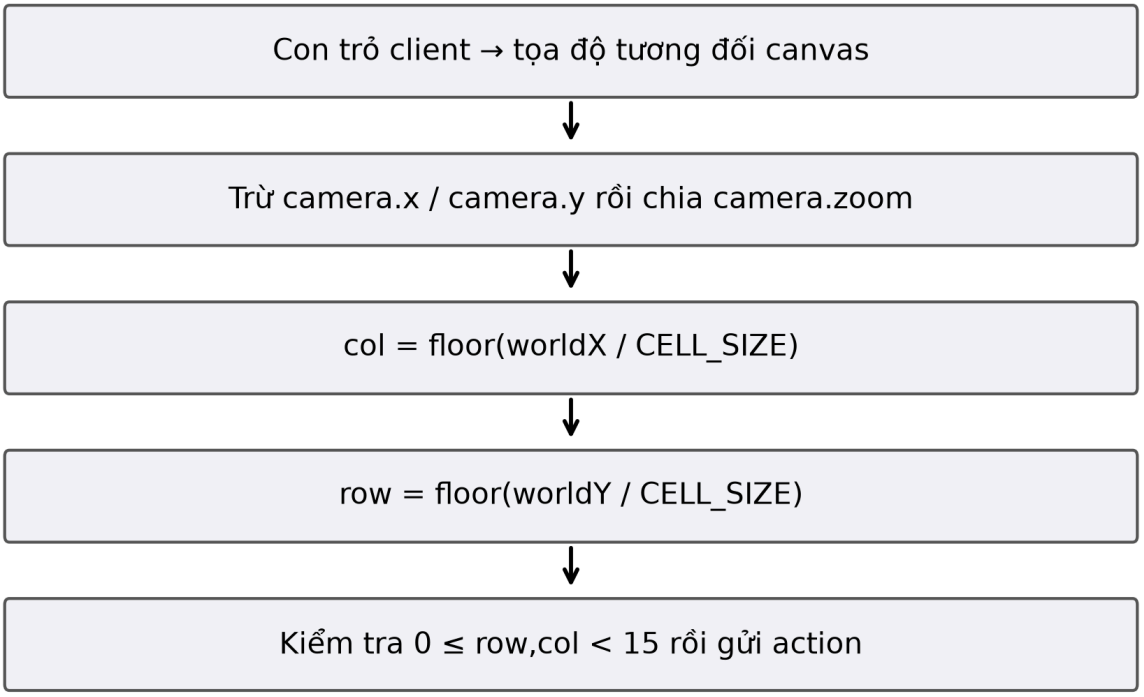
Phía Worker, initializeEngine lưu promise khởi tạo WASM để tái sử dụng trong vòng đời Worker. executeWasmSearch tạo một Game Rust, phát lại từng nước từ history, gọi ai_move, kiểm tra cấu trúc kết quả và giải phóng đối tượng trong finally. Khi tái

tạo, cột TypeScript chuyển thành x và hàng thành y; chiều ngược lại, x/y từ engine được chuyển về col/row. Nếu replay một nước không thành công hoặc telemetry không đúng contract, Worker trả lỗi thay vì tiếp tục sử dụng kết quả không hợp lệ.

Một phản hồi chỉ được commit sau khi GameController.startAiTurn kiểm tra token lượt, trạng thái pause, roundId, phase, mode và người hiện tại. Điều kiện này ngăn một kết quả của ván hoặc lượt cũ ghi vào state sau replay, Undo, đổi mode hoặc pause. Sau đó, nước được đưa qua playMove; nếu ô không hợp lệ hoặc máy không trả nước, controller ghi aiError. Nhánh catch chỉ cập nhật lỗi nếu token vẫn thuộc lượt hiện hành. SearchFeedback hiển thị trạng thái và nút thử lại; retryAiTurn chỉ bắt đầu khi còn đúng điều kiện lượt O đang chơi.

Đồng hồ elapsedMs của nhánh Rust trong executeWasmSearch bao gồm tái tạo Game, tìm kiếm và giải phóng, nhưng không bao gồm tải WASM ban đầu và chuyển thông điệp. Thời gian của tactical TypeScript có phạm vi khác. Khi phân tích hiệu năng phải nêu phạm vi này thay vì gộp hai loại thành một phép đo đồng nhất.

3.7. Thiết kế hiển thị bàn, tọa độ và tương tác



Hình 3.3. Biến đổi tọa độ và kiểm tra biên

Sơ đồ giải thích dựng từ mã nguồn. Nguồn: BoardInputController.ts; CoordinateTransform.ts.

CaroGameCanvas tạo một phần tử canvas, gắn CaroScene, đồng bộ board/lastMove/winCellSet và đăng ký ResizeObserver. Khi component bị tháo, observer được disconnect, scene được destroy và canvas được gỡ. Scene tạo Pixi Application, một world container và các lớp riêng cho lưới, quân và hiệu ứng đánh dấu. syncState nhận bản biểu diễn để vẽ; nó không phải chủ sở hữu luật hay lịch sử ván.

Camera lưu vị trí và tỷ lệ zoom. fitBoardToViewport tính kích thước bàn từ số ô và CELL_SIZE, chừa padding theo chiều rộng viewport, sau đó chọn tỷ lệ nhỏ hơn trong hai tỷ lệ theo chiều rộng và chiều cao để toàn bàn vừa vùng hiển thị. Khi camera hoặc viewport thay đổi, refreshVisibleBoard tính miền nhìn thấy, vẽ lại lưới, đồng bộ quân trong miền và đánh dấu nước cuối cùng hoặc các ô thắng. Tách world coordinate khỏi screen coordinate cho phép cùng một bàn được hiển thị ở nhiều kích thước.

BoardCoordinateTransform.screenToCell đảo biến đổi camera bằng cách lấy tọa độ màn hình trừ dịch chuyển, chia zoom và kích thước ô, rồi lấy phần nguyên. Kết quả là col và row; isBoardCell kiểm tra có thuộc bàn hữu hạn trước khi tạo thao tác nhấn. BoardInputController có trạng thái idle, pressed và dragging; nếu độ dịch chuyển giữa các sự kiện vượt ngưỡng cấu hình 6, nó chuyển sang kéo và phát PAN. Khi thả trong trạng thái pressed và tọa độ hợp lệ, nó phát CELL_TAP. Đây là chi tiết của implementation hiện tại, không đồng nghĩa mọi cử chỉ chạm hay thiết bị đều đã được xác nhận.

React hiển thị thông tin người chơi, trạng thái tìm kiếm, nút Undo, chế độ, độ khó và banner kết quả quanh vùng canvas. Kết quả luật từ controller được chuyển thành tập khóa row,col để scene tô các ô thuộc chuỗi thắng. Bằng cách này, Pixi chịu trách nhiệm vẽ còn state thắng và điều kiện thắng vẫn thuộc miền trò chơi. Việc test đúng phải bao gồm sự tương ứng giữa tọa độ pointer, ô được chọn, state đã cập nhật và vị trí quân được vẽ sau thay đổi kích thước.

3.8. Ngôn ngữ, âm thanh và tích hợp bên ngoài

i18n.ts quản lý ngôn ngữ dùng chung; App.tsx lấy bản dịch qua useTranslation và cho phép đổi VI/EN. useWinkIntegration.normalizeWinkLocale ánh xạ locale bắt đầu bằng vi thành tiếng Việt, các trường hợp khác thành tiếng Anh; locale từ host được đồng bộ vào cùng instance. Cách biểu diễn này cho phép giao diện và host dùng chung quyết định ngôn ngữ, nhưng không phải bằng chứng rằng mọi chuỗi, thiết bị hoặc vùng miền đã được kiểm thử.

playButtonClick trong utils/buttonClick.ts tạo âm click tổng hợp qua Web Audio: oscillator hình sin, gain giảm trong khoảng ngắn và tự dừng. Hàm thử khởi tạo AudioContext khi có window, thử resume nếu context đang suspended và bắt lỗi để âm thanh không chặn thao tác chính. Trong source được kiểm tra, hook Wink có hostMuted; cần đối chiếu riêng toàn bộ đường phát âm thanh với trạng thái này trước khi tuyên bố mute của host đã được áp dụng đồng nhất. Sự tồn tại của trường hostMuted hoặc lệnh resume không tự chứng minh âm thanh hoạt động đúng ở mọi trình duyệt và thiết bị.

resolveGlobalWink tìm SDK trong window, gọi init nếu có, hoặc chờ có giới hạn trước khi trả null. Khi không có SDK, hook chuyển sang trạng thái standalone và vẫn cho phép khởi tạo giao diện. Khi có SDK, hook đăng ký sự kiện pause/resume, mute/unmute và locale; cleanup hủy đăng ký và cố gắng destroy SDK. Các hàm gameplayStart/gameplayStop được gọi theo vòng đời ván. Các thao tác leaderboard và submit score kiểm tra capability, xử lý lỗi và có thể không thực hiện khi SDK không sẵn sàng. App.tsx quy đổi điểm ví dụ trong source cho thắng X, hòa hoặc kết quả còn lại; đây là chính sách của ứng dụng, không là dữ liệu thành tích thật.

Ranh giới này tách game state cục bộ khỏi các dịch vụ ngoài. Một bài kiểm thử unit với SDK giả có thể xác nhận cách hook phản ứng với sự kiện giả, nhưng không xác nhận thỏa thuận phiên bản SDK, quyền gọi API, lifecycle iframe hay tính đúng của bảng xếp hạng production. Các nội dung đó cần một phiên thử trên host và tài khoản phù hợp trước khi đưa vào kết luận nghiệm thu.

3.9. Các kịch bản cần kiểm chứng

Bảng 3.2. Kịch bản và kết quả mong đợi để thiết kế kiểm thử; trạng thái chạy thực tế cần đối chiếu với hồ sơ kiểm thử riêng.

Mã	Điều kiện và thao tác	Kết quả mong đợi
S01	Ván mới, nhấn một ô hợp lệ	Quân hiện đúng ô, history tăng một, lượt đổi nếu chưa kết thúc
S02	Khi phase đang playing, thử ô đã có quân hoặc tọa độ ngoài bàn qua controller	Nước bị từ chối, bàn và history không thay đổi
S03	Đặt quân tạo ít nhất năm liên tiếp trên mỗi trục	Ghi người thắng, chuyển game-over, đánh dấu các ô thắng
S04	Bàn đầy sau nước cuối và không có người thắng	Ghi hòa và không cho tiếp tục đặt quân
S05	Chơi 1v1, nhấn Undo	Xóa một ply, tính lại lượt và phase
S06	Chơi AI, máy đã trả lời, nhấn Undo	Xóa nước O cuối và nước X trước đó; máy không áp kết quả cũ
S07	Bắt đầu lượt AI rồi replay hoặc đổi mode	Yêu cầu cũ bị vô hiệu; phản hồi cũ không ghi vào ván mới
S08	Worker trả lỗi, không có nước hoặc tọa độ không hợp lệ	Hiển thị lỗi, không ghi nước sai; khi đủ điều kiện có thể thử lại

Mã	Điều kiện và thao tác	Kết quả mong đợi
S09	Host pause trong lúc máy đang tính, sau đó resume	Không nhận input khi pause; sau resume tìm lại nếu vẫn là lượt O
S10	Đổi kích thước vùng bàn và nhấn một ô quan sát được	Bàn được fit lại; tọa độ nhấn và quân hiển thị tương ứng
S11	Không có SDK Wink hoặc capability leaderboard	Game chuyển standalone hoặc dữ liệu dịch vụ rỗng, không giả kết quả online
S12	Mở figure và chạy ví dụ Python trong trang trình bày	Thông tin figure hoặc stdout/lỗi được hiển thị theo hành vi component

Trong playCell, nếu phase ban đầu là setup hoặc ready, startRound() được gọi trước khi kiểm tra hợp lệ của ô. Vì vậy kịch bản S02 giới hạn ở phase playing; không suy rằng phase không thay đổi khi gọi trực tiếp controller với nước đầu không hợp lệ. Trên đường Pixi, input còn lọc tọa độ ngoài bàn trước callback.

Các kịch bản S07–S09 đặc biệt cần thực thi với thời điểm phản hồi có thể kiểm soát. Chỉ chạy nước đi tuần tự nhanh không đủ phát hiện phản hồi đến sau Undo hoặc pause. Các ca S10–S12 còn phụ thuộc môi trường trình duyệt, quyền clipboard, tải tài nguyên và kết nối CDN; cần ghi rõ môi trường thực tế nếu công bố kết quả.

CHƯƠNG 4. TRIỂN KHAI HỆ THỐNG

4.1. Môi trường và tổ chức mã

Ứng dụng CaroGame là dự án web dùng React, TypeScript, Vite và PixiJS. package.json khai báo PixiJS nhánh 7.4, React nhánh 19, i18next và các lệnh phát triển, build, typecheck, Vitest. Số phiên bản trong package là phạm vi khai báo dependency của snapshot, không thay cho việc ghi lockfile và phiên bản đã cài khi tái lập. Engine Rust được biên dịch thành package JavaScript/WebAssembly dưới src/game/wasm/engine, gồm file kết nối và binary .wasm.

Cấu trúc src chia thành app điều phối, game/core giữ state và luật, game/behaviors cho hành vi nước đi và ván đấu, game/ai và game/wasm cho AI, board cho đầu vào/tọa độ/render, pixi cho scene, components cho các phần React, cùng integrations/wink và i18n. Sự phân chia này giúp phân tích chức năng theo ownership, mặc dù một số nhánh legacy dưới game/bot vẫn tồn tại và cần phân biệt với đường chạy AIController → GomokuWasmEngine hiện tại.

Ứng dụng present là dự án React/Vite riêng, có package và scripts riêng; nó sử dụng GSAP, KaTeX và các component nội dung. Không nên suy luận cùng dependency hoặc cùng cơ chế vẽ chỉ vì cả hai cùng trình bày một bàn Caro. Bàn minh họa của present là React component với state và logic TypeScript riêng; bàn của CaroGame sử dụng Pixi scene và controller như đã mô tả.

4.2. Triển khai nhận và cập nhật nước đi

Tại CaroScene.setupInteraction, sự kiện pointerdown được chuyển tới bộ input; pointermove có thể phát PAN; pointerup hoặc pointerupoutside tạo CELL_TAP nếu vẫn là thao tác nhấn hợp lệ. Callback chuyển row, col về App, sau đó gọi GameController.playCell. Controller từ chối khi pause, game-over, AI đang tính hoặc không đúng lượt người trong mode AI. Với các trường hợp còn lại, playMove gọi validateMove để kiểm tra biên và ô trống rồi tạo đối tượng Move có chỉ số bằng moveCount cộng một.

Sau khi commit, luật đánh giá nước cuối. finishMove ưu tiên xử lý thắng, sau đó mới xét bàn đầy; nếu chưa kết thúc, currentPlayer đổi từ X sang O hoặc ngược lại.

Controller tạo snapshot và thông báo cho subscriber. Trong mode AI, nếu đến O và match vẫn đang chơi, startAiTurn được gọi. Các nhánh từ chối không được trình bày là đã thử thành công chỉ vì có mã trả về rejected; mỗi nhánh cần test với state thích hợp.

Ví dụ thao tác: người chơi X nhấn hàng 7, cột 7 khi ván sẵn sàng. Bộ chuyển đổi tạo tọa độ này từ vị trí trên canvas, state ghi quân X, history có một phần tử và lượt đổi sang O. Nếu là mode 1v1, người thứ hai có thể chọn ô khác; nếu là mode AI, controller bắt đầu yêu cầu máy và đánh dấu aiThinking. Ví dụ mô tả thiết kế, không phải bản ghi của một phiên chạy cụ thể.

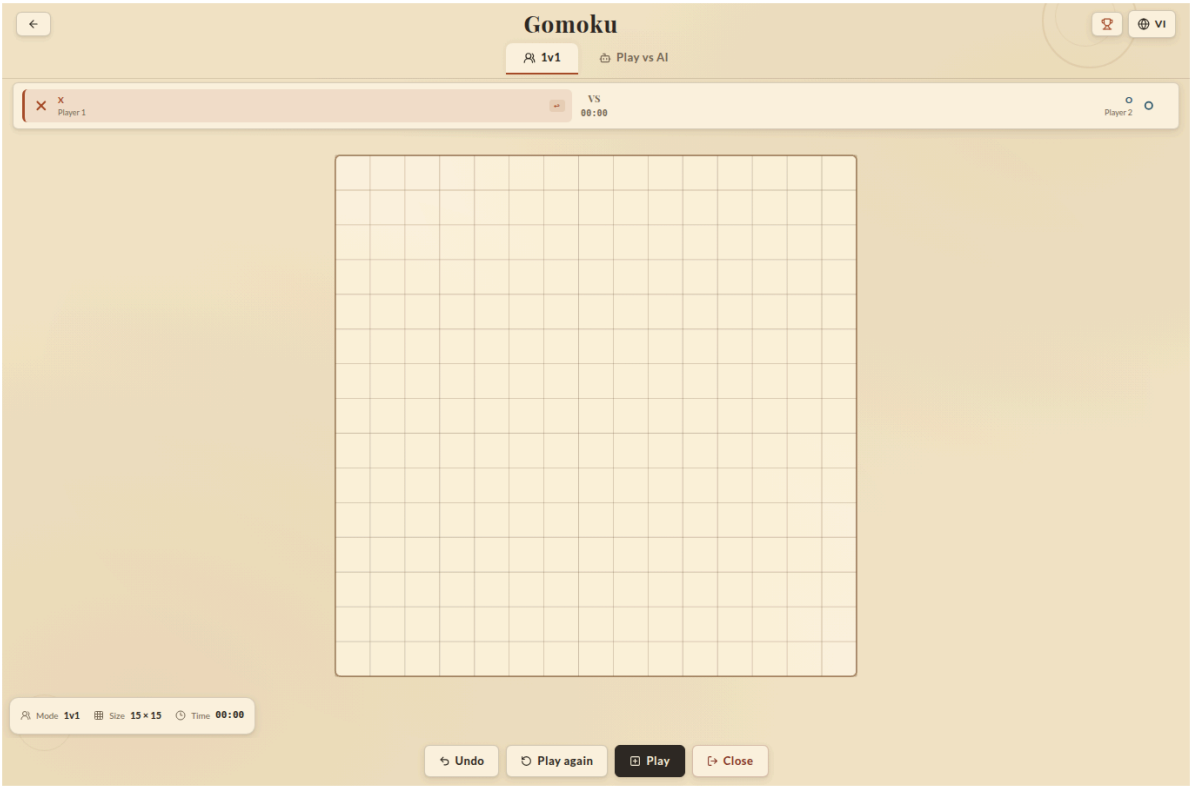
4.3. Triển khai điều phối máy và kết quả

Tầng điều phối chia làm hai trách nhiệm. GomokuWasmEngine quản lý Worker và các promise; GameController quản lý quyền áp dụng kết quả. Sự tách biệt này quan trọng vì Worker không biết người dùng đã đổi ván ở giao diện hay chưa. Controller dùng turnToken và roundId cho phép đánh dấu các yêu cầu cũ; đồng thời đối chiếu phase, mode, pause và currentPlayer khi promise hoàn thành.

Ranh giới executeWasmSearch cũng kiểm tra dữ liệu do engine trả: tọa độ nguyên trong khoảng 0–14, điểm hữu hạn, difficulty khớp, strategy thuộc danh sách mong đợi và các bộ đếm không âm hợp lệ. Việc kiểm tra contract nhằm phát hiện package cũ hoặc kết quả sai cấu trúc. Sau đó controller vẫn kiểm tra nước bằng luật trạng thái hiện tại: một tọa độ nằm trong bàn chưa đủ chứng minh ô đó còn trống hay ván còn cho phép đi.

SearchFeedback lấy các trường trạng thái như depth, nodes, cutoffs, leaves, nguồn và lỗi từ snapshot. Số liệu là telemetry của một lời gọi, không nhất thiết là đo toàn bộ thời gian trải nghiệm từ lúc người chơi nhấn đến lúc quân xuất hiện. Cũng có các nước tactical trả trực tiếp từ TypeScript với số nút bằng không; không được đưa chúng vào cùng tập so sánh search Rust mà không tách nguồn.

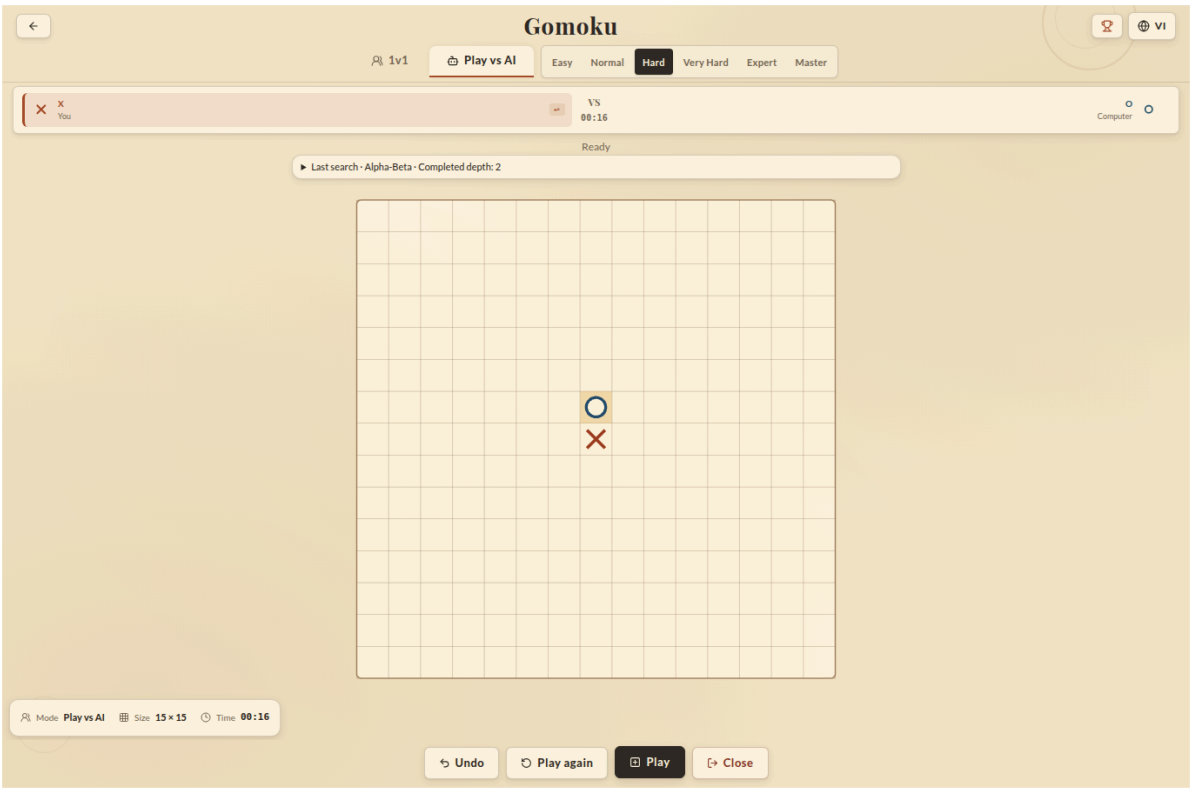
4.4. Triển khai bàn cờ và thành phần bao quanh



Hình 4.1. Giao diện hai người chơi trên bản build local

Ảnh chụp trình duyệt local. Nguồn: Ảnh chụp Chrome ngày 27/10/2026, 1365 × 900.

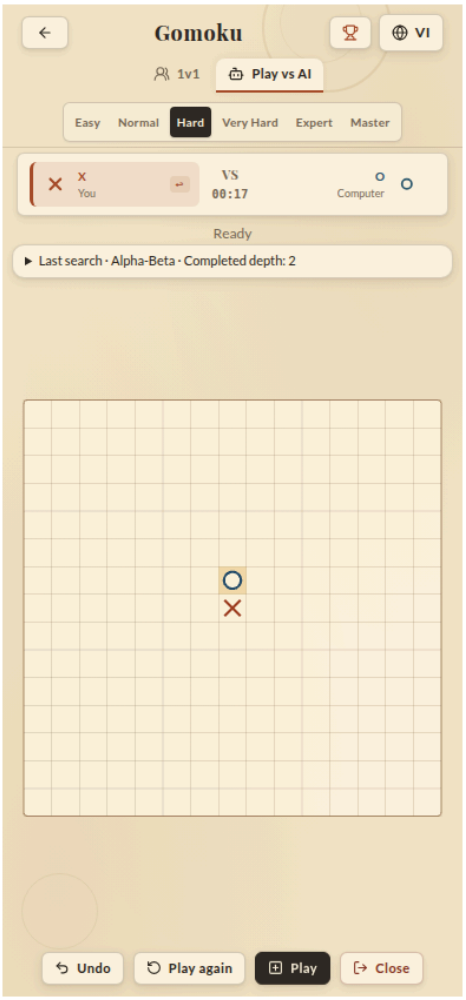
Ở Hình 4.1, bản build hiện hành hiển thị bàn trống với lưới 15 × 15 ở chế độ hai người chơi. Các nút chọn chế độ và điều khiển đặt ngoài canvas, còn trạng thái ván do controller cung cấp. Vị trí quân X/O sau khi đặt được minh họa ở Hình 4.2.



Hình 4.2. Ván đấu máy sau một lượt người và một lượt bot

Ảnh chụp trình duyệt local. Nguồn: Ảnh chụp Chrome ngày 27/10/2026; Hard, completed depth 2.

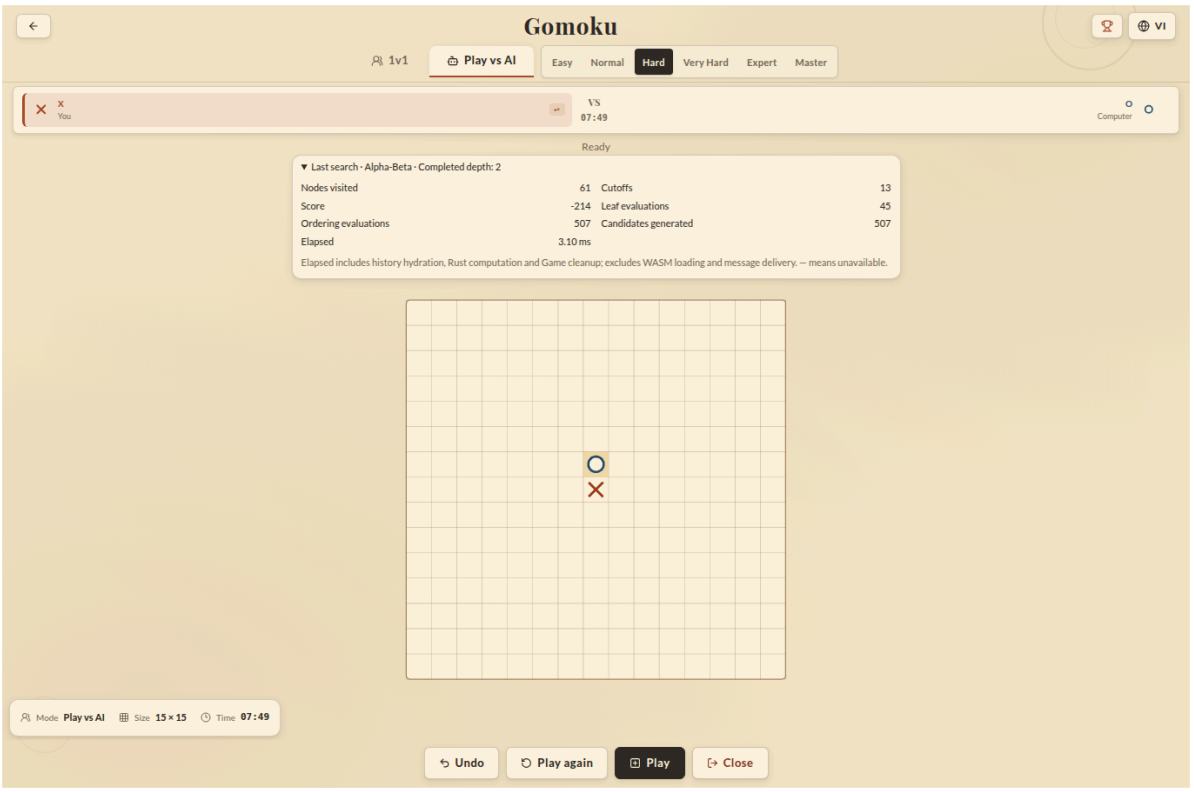
Ở Hình 4.2, ảnh được chụp sau một thao tác đặt quân ở giữa bàn và phản hồi của bot Hard. Nhãn độ sâu hoàn tất bằng 2 khớp cấu hình của mức Hard trong lần quan sát này.



Hình 4.3. Giao diện tại viewport 390 × 844

Ảnh chụp trình duyệt local. Nguồn: Ảnh chụp Chrome ngày 27/10/2026; viewport giả lập.

Ở Hình 4.3, ảnh tại viewport 390 × 844 minh họa bố cục trong vùng hiển thị hẹp. Đây là giả lập kích thước cửa sổ trên trình duyệt desktop, không là phép thử cảm ứng vật lý.



Hình 4.4. Thông tin của một lần tìm kiếm Hard trong browser local

Ảnh chụp trình duyệt local. Nguồn: Ảnh chụp Chrome ngày 27/10/2026; một quan sát, không phải benchmark.

Ở Hình 4.4, bảng chi tiết hiển thị số node, lá, cutoff, lượng giá thứ tự và thời gian của một yêu cầu. Ví dụ ghi 61 node và 13 cutoff; thời gian hiển thị thuộc phạm vi hydrate–tính–cleanup của adapter. Một quan sát đơn lẻ chưa đại diện phân phối độ trễ.

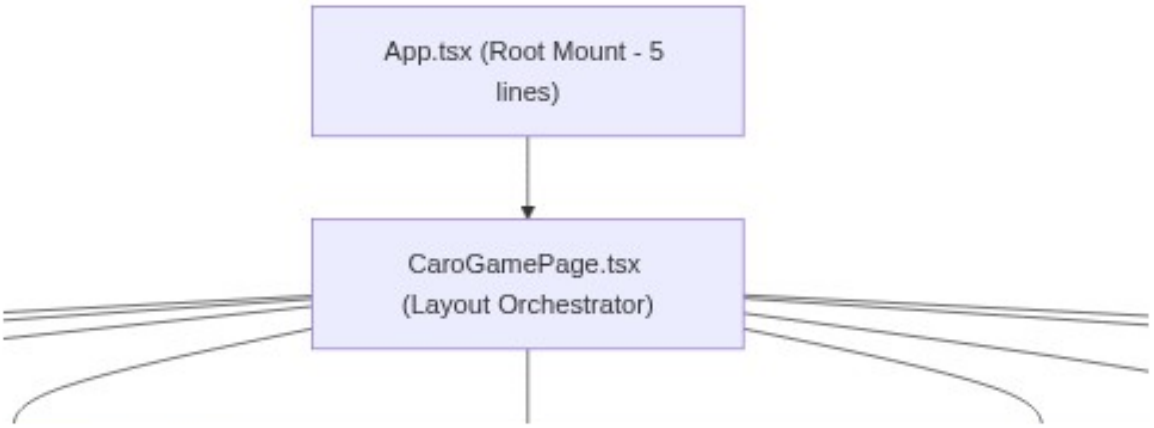
CaroScene xây dựng scene graph gồm các layer Grid, Stone và Fx trong world container. Bộ vẽ lưới tạo cấu trúc bàn; bộ vẽ quân đồng bộ các quân cần hiển thị; bộ đánh dấu thể hiện nước cuối và chuỗi thắng. Camera và phép chuyển đổi tọa độ giữ cho thao tác và hình vẽ dùng cùng hệ quy chiếu. CaroGameCanvas là cầu nối vòng đời React–Pixi: nó khởi tạo scene một lần, đồng bộ dữ liệu mới và dọn dẹp khi tháo component.

Tính responsive được thể hiện qua kích thước bao bởi 100dvh ở shell, ResizeObserver trên container và hàm fit bàn. Kích thước canvas được điều chỉnh khi vùng chứa đổi; scene tính zoom từ không gian hiện có. Điều này giải quyết việc biến đổi bố cục ở mức mã. Muốn kết luận sử dụng tốt trên màn hình nhỏ vẫn phải xem

kích thước ô, khả năng thao tác touch, vùng thông tin cạnh bàn và khả năng đọc khi người dùng thay đổi tỷ lệ chữ.

Các nút React bao quanh canvas gọi controller, còn ngôn ngữ do i18n cung cấp. App lưu đồng hồ hiển thị riêng và chỉ tăng trong lúc có ván, chưa game-over và host chưa pause. Điều này tách đồng hồ hiển thị khỏi lịch sử nước và không biến tick mỗi giây thành nước đi. Kết luận về số lần render hoặc hiệu năng của tách state cần phép đo cụ thể; không suy số render bằng không từ việc có hai state riêng.

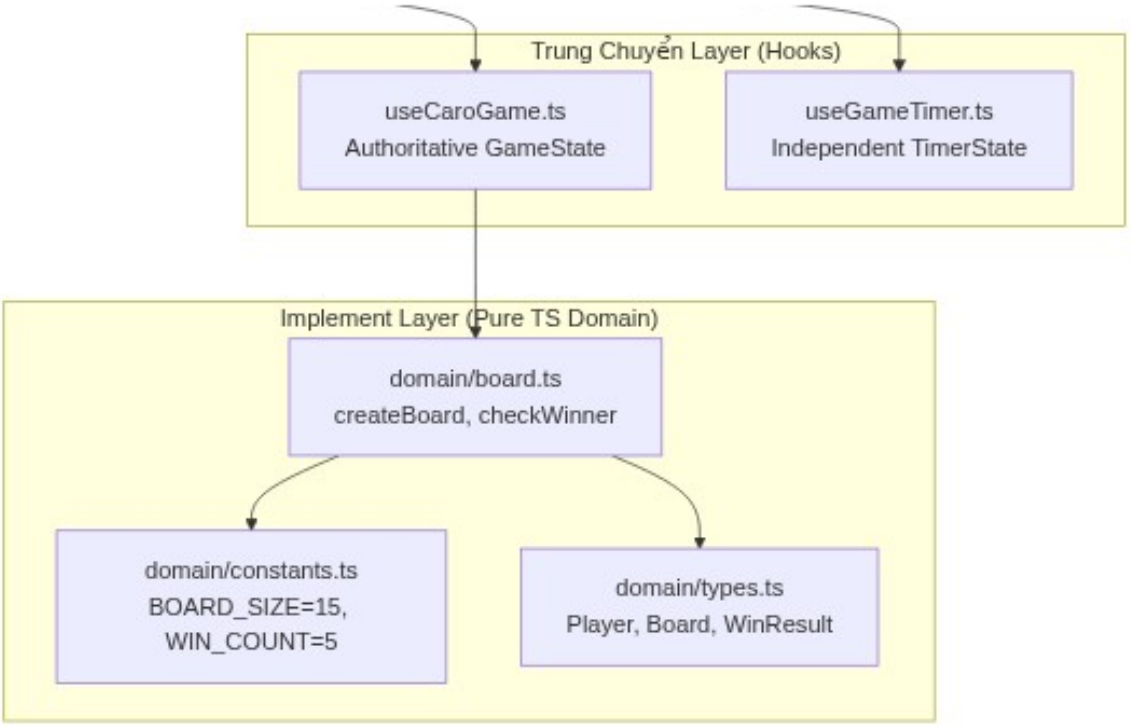
4.5. Ứng dụng trình bày và figure inspector



Hình 4.5. Chi tiết khối App và trang điều phối trong sơ đồ present

Ảnh tài liệu lịch sử của frontend DOM. Nguồn: present/public/assets/component_hierarchy.png; crop bằng Pillow.

Ở Hình 4.5, sơ đồ nguồn trong present mô tả frontend DOM lịch sử với App và CaroGamePage. Ảnh được crop bằng Pillow để đọc được tên khối; cấu trúc này cần phân biệt với controller và canvas Pixi của ứng dụng hiện hành.



Hình 4.6. Chi tiết hooks và domain trong sơ đồ present

Ảnh tài liệu lịch sử của frontend DOM. Nguồn: present/public/assets/component_hierarchy.png; crop bằng Pillow.

Ở Hình 4.6, phần phóng cắt từ sơ đồ cho thấy game state, timer và domain ở các nhóm khác nhau. Đây là cách tổ chức được tài liệu trình bày mô tả; các khẳng định về số lần render vẫn cần phép đo riêng.

Nội dung và hình của website trình bày được đối chiếu trực tiếp từ thư mục present [11].

present/src/App.tsx lắp ghép các section về tổng quan, tuần 3–10, visualizer, bàn tương tác, benchmark và roadmap. GSAP/ScrollTrigger được dùng trong effect để tạo chuyển động khi section xuất hiện. Mỗi section là nội dung trình bày; thứ tự hoặc nhãn tuần phản ánh cách trang tổ chức thông tin, không tự xác nhận chronology của mọi commit trong project. Các dữ kiện về trạng thái hiện tại phải tiếp tục đối chiếu source và hồ sơ gate.

App giữ activeFigureId; khi người dùng mở hình, id được tra trong FIGURE_REGISTRY và truyền cho FigureInspectorModal. Một mục registry chứa tên, đường dẫn hình, caption, file nguồn được nêu, công thức, diễn giải, bước thuật toán,

code và phần lợi/hại. Modal hỗ trợ hai cách bố trí, copy code, phóng to/thu nhỏ và kéo hình. Đây là cơ chế tập trung dữ liệu giải thích để tái sử dụng trong giao diện. Tuy vậy, sourceFile và scriptCode trong registry là nội dung đã ghi vào ứng dụng trình bày; chúng không tự cập nhật khi mã thật thay đổi và cần kiểm tra độc lập trước khi dùng làm trích dẫn source.

Một số phần trong README và figure của present mô tả phiên bản lịch sử hoặc đưa ra những con số như tỷ lệ giảm, thời gian hoặc tỉ số tốc độ. Các con số đó chỉ được đưa vào đánh giá khoa học khi xác định được script, fixture, môi trường và dữ liệu gốc tương ứng. Nếu chưa đối chiếu được, báo cáo chỉ mô tả việc trang hiển thị hoặc minh họa, không lặp lại con số như kết quả mới của bản game hiện tại.

4.6. Pyodide và bàn tương tác của trang trình bày

present/index.html tải script Pyodide từ CDN, còn pyodideRunner.ts giữ instance dùng chung sau khi khởi tạo. executePythonCode thiết lập stdout và stderr, thử tải package được nhận diện từ import, chạy mã bằng runPythonAsync rồi trả chuỗi kết quả hoặc thông báo lỗi cho giao diện. PythonVisualizer có các script mẫu cho quét mẫu, heuristic, ứng viên Chebyshev và phân tích số liệu. Kết quả chạy là output của chính script trong trang tại lần đó; nó không tự gọi engine Rust production hoặc chứng minh benchmark của engine.

Việc tải script và package phụ thuộc tài nguyên có sẵn, nên cần mô tả kết quả lỗi khi mạng hoặc CDN không đáp ứng. Trong mã hiện tại, thực thi được gọi thông qua API Pyodide từ trang; không có bằng chứng trong đoạn này cho thấy mọi script đều chạy ở một Worker riêng hoặc có giới hạn thời gian cho tác vụ dài. Cần kiểm thử cụ thể trước khi khẳng định tính cô lập, phục hồi hoặc khả năng đáp ứng khi chạy mã bất kỳ.

InteractiveCaroBoard có state bàn, lượt, mode PvP/PvE, bot Rule hoặc Random, lựa chọn hiện tập Chebyshev và giá trị heuristic. Nó tạo bản sao mảng khi thay đổi bàn, kiểm tra thắng quanh nước mới và đặt nước máy qua callback trì hoãn. Component cũng có lựa chọn hiển thị nguồn bàn nhúng bên ngoài qua iframe. Bàn tự chứa này là công cụ minh họa với logic riêng; kết quả thao tác trên đó không tự chứng minh luồng Worker, Undo, Wink hay telemetry của CaroGame.

4.6.1. Ví dụ quét hình thế bằng Python

Script mẫu pattern_scanner.py tạo ma trận 15 × 15, đặt bốn quân X liên tiếp ở hàng 7 và ba quân O theo cột 7. Hàm detect_patterns duyệt từng hàng bằng vòng while, nhận diện đầu của đoạn liên tiếp rồi đếm chiều dài và hai đầu mở. Với bốn X ở cột 4–7, hai đầu đều trống, kết quả open_four bằng 1. Đây là một ví dụ dễ quan sát cách tránh đếm lặp trong cùng hàng.

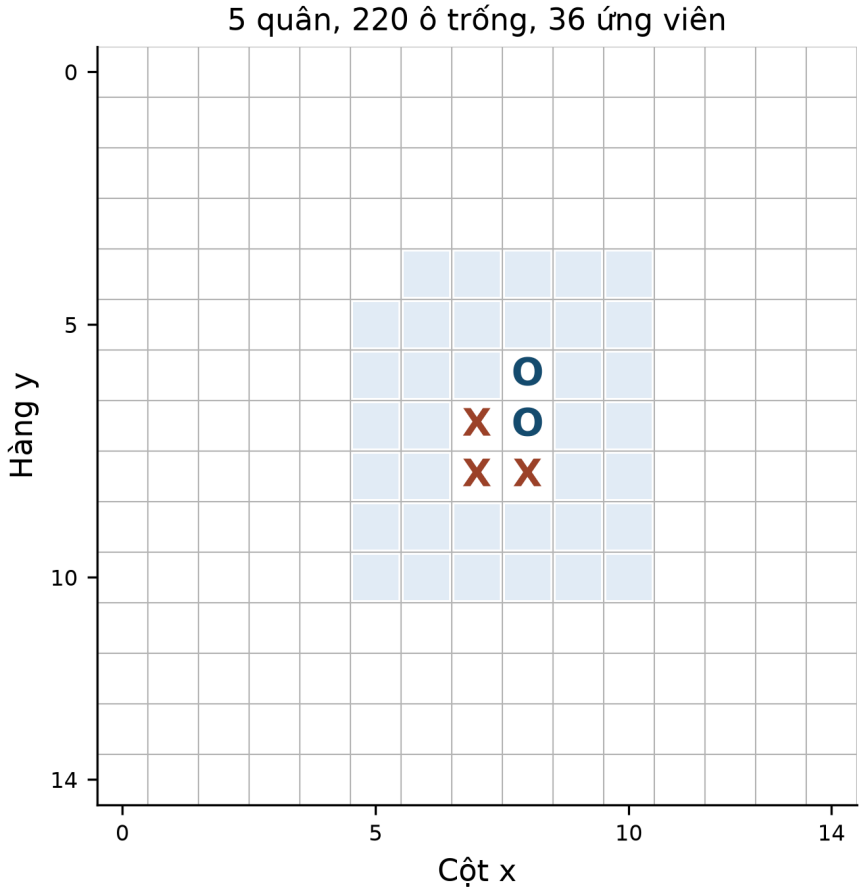
Phạm vi cài đặt thực tế hẹp hơn phần mô tả của trang. Hàm mẫu chỉ quét hàng ngang; dictionary trả về gồm năm loại five, open_four, closed_four, open_three, closed_three. Nó chưa quét dọc và chéo, chưa trả open_two và closed_two. Vì vậy dòng mô tả “bốn trục, bảy mẫu” của giao diện chưa được script mẫu này thực hiện đầy đủ. Ba O đặt dọc là dữ liệu khởi tạo; việc in tên thế đó ra màn hình không có nghĩa thuật toán đã quét và phát hiện nó. Ngoài ra, open four là một đe dọa mạnh nhưng kết luận thắng ở lượt sau còn phụ thuộc lượt đi và các đe dọa thắng hiện có của đối thủ.

4.6.2. Ví dụ cộng điểm heuristic

Script heuristic_evaluator.py nhận các dictionary số lượng mẫu đã biết. Nó không nhận ma trận rồi tự phát hiện mẫu. Hàm evaluate_position nhân số lượng với trọng số, dùng hệ số phòng thủ 1,2 và chặn điểm trong miền đã chọn. Ví dụ một open four của bot đối đầu một open three cho 100.000 − 1,2 × 5.000 = 94.000 điểm; một open three đối đầu một closed four cho 5.000 − 1,2 × 10.000 = −7.000 điểm. Hai phép tính minh họa cách cộng trọng số và cách một mối đe dọa làm điểm âm.

Bảng trọng số của script này khác eval.rs trong engine game hiện hành. Chẳng hạn open four của ví dụ là 100.000, trong khi engine game dùng 50.000. Đầu vào các mẫu được đặt thủ công và script không lựa chọn nước đi từ bàn cờ, nên điểm âm chưa tự thực hiện hành động chặn. Khi giải thích kết quả, cần gắn nhãn đây là bộ tính điểm mẫu của present. Các công thức về terminal dominance trong chương 2 mô tả thiết kế engine thật với miền điểm riêng.

4.6.3. Ví dụ tập ứng viên và mẫu số của tỷ lệ giảm



Hình 4.7. Tái dựng ví dụ Chebyshev năm quân của present

Ví dụ hình học tính lại bằng Python. Nguồn: PythonVisualizer.tsx::chebyshev_pruner.py; 36/220, giảm 83,64%.

Script chebyshev_pruner.py tạo tập occupied từ năm tọa độ ở giữa bàn, duyệt vùng bán kính hai quanh từng quân và dùng set để loại trùng các ô ứng viên. Chạy đoạn mã mẫu bằng CPython trong lượt biên soạn ngày 27/10 cho 36 ứng viên và 220 ô trống. Tỷ lệ giảm so với ô trống là (220 − 36) / 220 × 100% = 83,64%. Đây là tỷ lệ của đúng vị trí năm quân đó. Dùng 225 làm mẫu số sẽ trộn các ô đã có quân vào tập nước hợp lệ; dùng 72,39% của thí nghiệm 100 ván lại trộn một cách tổng hợp khác vào ví dụ này.

Ví dụ cho thấy vai trò của cấu trúc set, điều kiện biên và việc trừ ô occupied. Nó cũng cho thấy một nội dung trình bày có thể có tiêu đề không khớp output của chính script. Kết quả chạy mẫu chỉ kiểm tra phép tính của đoạn Python được trích từ

source; không xác nhận Pyodide tải thành công trong browser hoặc khẳng định chính sách giữ chiến thuật của engine đã được thực hiện bởi đoạn mã đó.

4.6.4. Ví dụ thống kê và giới hạn của số được nhập sẵn

Script empirical_analysis.py gán trực tiếp total_games, số ván thắng, số ply trung bình và avg_time_per_move_us. Nó không mở week4_bot_battle.json và không điều khiển bot chơi 100 ván. Chạy script chỉ tính lại phần trăm từ các hằng số. Giá trị 42,24 µs nhập sẵn khác giá trị trung bình 251,26 µs của JSON. Do đó output là trình bày một bộ số được nhập vào mã, không phải lần đo mới.

So sánh một thời gian native với 16,67 ms của chu kỳ 60 khung hình mỗi giây cũng chưa đo rendering, main thread, truyền thông điệp hay thiết bị. Khẳng định không rớt frame trong dòng print của ví dụ chưa được phép tính đó chứng minh. Báo cáo giữ nguyên vai trò giáo dục của visualizer và dùng raw JSON/CSV đúng phạm vi khi đánh giá. Bốn đoạn mã mẫu đã được chạy cục bộ để đối chiếu tính toán; bản kiểm tra lưu cả stdout và nhãn môi trường, không biến chuỗi tự in “WebAssembly” thành bằng chứng chạy WebAssembly.

4.7. Ranh giới giữa triển khai và kiểm chứng

Phần triển khai cho thấy các chức năng được viết ở đâu và thông tin đi qua các thành phần nào. Chương đánh giá cần bổ sung kết quả chạy phù hợp cho từng yêu cầu, gồm phiên bản source, lệnh, fixture, môi trường và kết quả. Đặc biệt, logic xử lý lỗi và hủy trong source cần các tình huống trả lỗi và phản hồi chậm; logic responsive cần quan sát màn hình; tích hợp Wink cần host thực hoặc môi trường giả được ghi rõ.

Các công thức, ảnh và đoạn code trong trang trình bày là tài sản giải thích hữu ích, nhưng phải đối chiếu với engine và ứng dụng hiện tại trước khi dùng làm nguồn cho các kết luận về hiệu năng hay behavior. Những thông tin chưa tìm được nguồn gốc, như thông số thiết bị đo, dữ liệu thô hoặc phương pháp lặp, được liệt kê là thiếu. Điều này giúp báo cáo vừa trình bày được toàn bộ sản phẩm đã tạo, vừa giữ đúng giới hạn bằng chứng.

CHƯƠNG 5. KẾT QUẢ VÀ ĐÁNH GIÁ

5.1. Phạm vi đánh giá và nguồn dữ liệu

Các hồ sơ được dùng để đánh giá gồm snapshot game [2], reference [3] và benchmark lịch sử [12].

Việc đánh giá được tổ chức theo ba tầng: kiểm tra engine Rust ở môi trường native; kiểm tra gói WebAssembly (WASM) và adapter; kiểm tra luồng giao diện trong trình duyệt. Một nhóm thực nghiệm riêng sử dụng thư viện caro-core-reference để nghiên cứu thuật toán. Thư viện này không đồng nhất với engine Rust được đóng gói trong ứng dụng; vì vậy kết quả của hai nhóm được trình bày tách biệt. Bằng chứng chính ghi ngày 23/10/2026; reference ghi ngày 17/10/2026. Đây là kết quả đã lưu, không phải phép đo mới.

5.2. Kết quả kiểm thử engine và luật chơi

Theo G3_native_tests.log, lệnh kiểm thử Rust ngày 23/10 ghi nhận 38 unit test và 7 integration test thành công. Các nhóm kiểm tra bao gồm khởi tạo bàn cờ, từ chối ô đã có quân hoặc tọa độ ngoài biên, đặt và hoàn tác quân, nhận diện thắng theo bốn trục và xử lý bàn cờ hòa. Các test xác nhận Freestyle: chuỗi từ năm quân, kể cả dài hơn năm hoặc bị chặn hai đầu, được tính thắng; điều này khác luật đúng năm hoặc không bị chặn hai đầu.

Tại lớp ứng dụng, G5_frontend_tests.log ghi 59 test thành công trong 9 file; typecheck và bản build Vite cũng kết thúc với mã thoát 0. Bộ 59 test này đã bao gồm 18 trường hợp actual-WASM trong G4_actual_wasm_tests.log, nên không cộng 18 thành một nhóm độc lập nữa. Build ghi cảnh báo chunk JavaScript sau minify lớn hơn 500 kB và cảnh báo về cấu hình loader; các cảnh báo đó không làm lệnh build thất bại nhưng vẫn là điểm cần xem xét khi tối ưu tải trang. Log build ghi riêng binary WASM khoảng 111,58 kB trước gzip và 34,14 kB sau gzip trong build đó; đây là kích thước file, không phải lượng RAM hay bộ nhớ GPU khi chạy.

Trong lần kiểm tra lại ứng dụng CaroGame ngày 27/10/2026, bộ test frontend ghi nhận 59 test đạt trong 9 file và lệnh build Vite thành công. Build vẫn có cảnh báo liên quan __dirname/loader cấu hình và chunk sau minify lớn hơn 500 kB. Đây là kiểm

tra lại của lớp ứng dụng trong phiên hiện tại; không phải phép chạy lại các thí nghiệm native hay các phép đo lịch sử được nêu ở các mục sau. Kết quả test và build không thay thế kiểm thử browser, thiết bị cảm ứng hoặc Wink host.

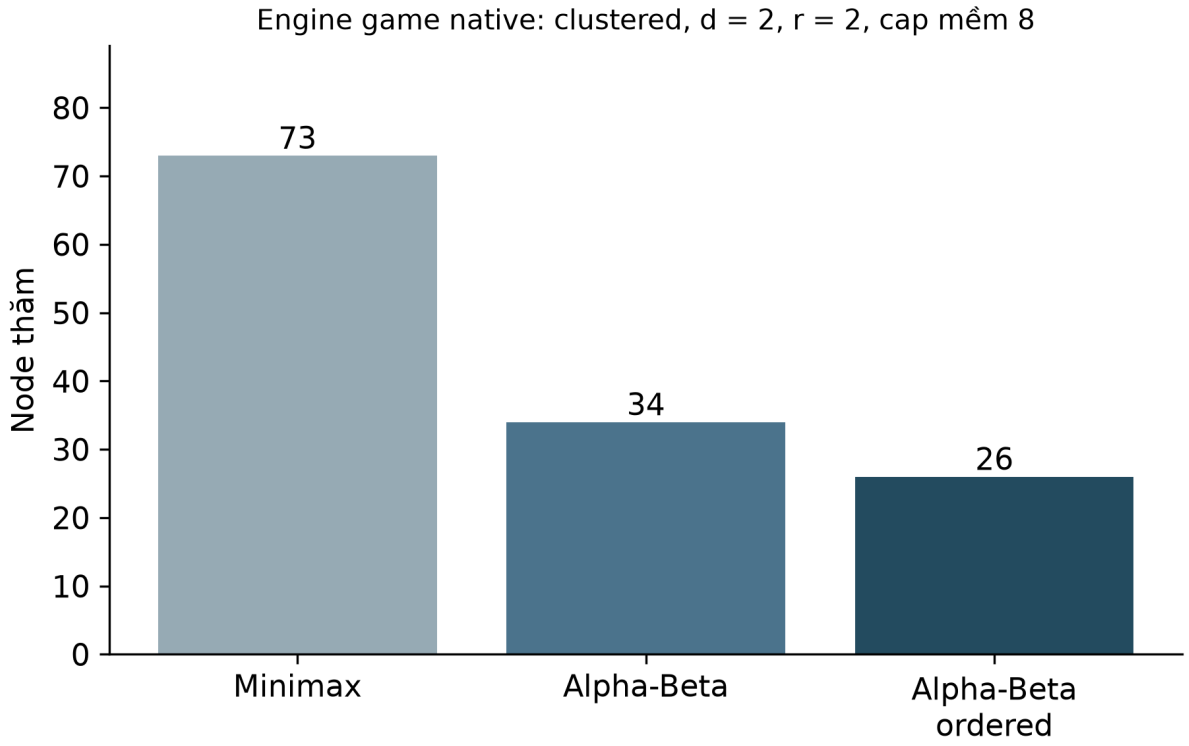
5.3. Đối chiếu WASM, Worker và giao diện

Theo wasm_package.json, binary WASM có SHA-256 1ff854e6493b39727d07cfcf69b0af06e349a748249d85d688741a4f85017989. File binary trong CaroGame của checkout báo cáo hiện có cùng hash. Khớp hash không chứng minh toàn bộ hành vi ứng dụng.

Bộ wasm_ts_parity.json ghi nhận so sánh thông qua adapter với WASM thật trong Node trên tám lịch sử ván cờ có thể đạt được, gồm 291 lượt đặt quân được chấp nhận, cùng các trường hợp đầu vào bị từ chối, Undo và Reset. Quy ước người chơi sau nước kết thúc khác nhau giữa hai phía: Rust chuyển lượt còn TypeScript giữ người vừa đi. Do vậy phép so sánh lượt tiếp theo chỉ áp dụng khi ván vẫn đang diễn ra. Đây là so sánh trên corpus xác định, không chứng minh hai cách biểu diễn tương đương ở mọi trạng thái.

Kiểm tra bằng Headless Chrome 155 đã gọi Worker thực và binary có hash nêu trên cho cả sáu mức Easy, Normal, Hard, Very Hard, Expert và Master; từng kết quả được ghi là nước hợp lệ trên vị trí đã chọn. Ở Hard trở lên, lớp TypeScript có thể trả nước chiến thuật trước khi gọi WASM; counter tactical không phải counter tìm kiếm Rust. Một phép kiểm giao diện khác thực hiện chuỗi React/Pixi input → Worker Rust → thông tin tìm kiếm → Undo. Sau Undo, thông tin chi tiết của nước bot được gỡ và trạng thái nút thay đổi như ghi trong G5_browser_ui.json. Kiểm tra kích thước 1280×800, 390×844 và 320×568 không phát hiện tràn ngang trong các trường hợp được ghi. Tuy nhiên ở 320×568, vùng bàn cờ còn thấp khi bảng chi tiết đang mở, nên kích thước thực tế của vùng tương tác cần được đánh giá thêm.

5.4. So sánh Minimax và Alpha-Beta trên engine ứng dụng



Hình 5.1. Số node của cùng cấu hình tìm kiếm trên engine game

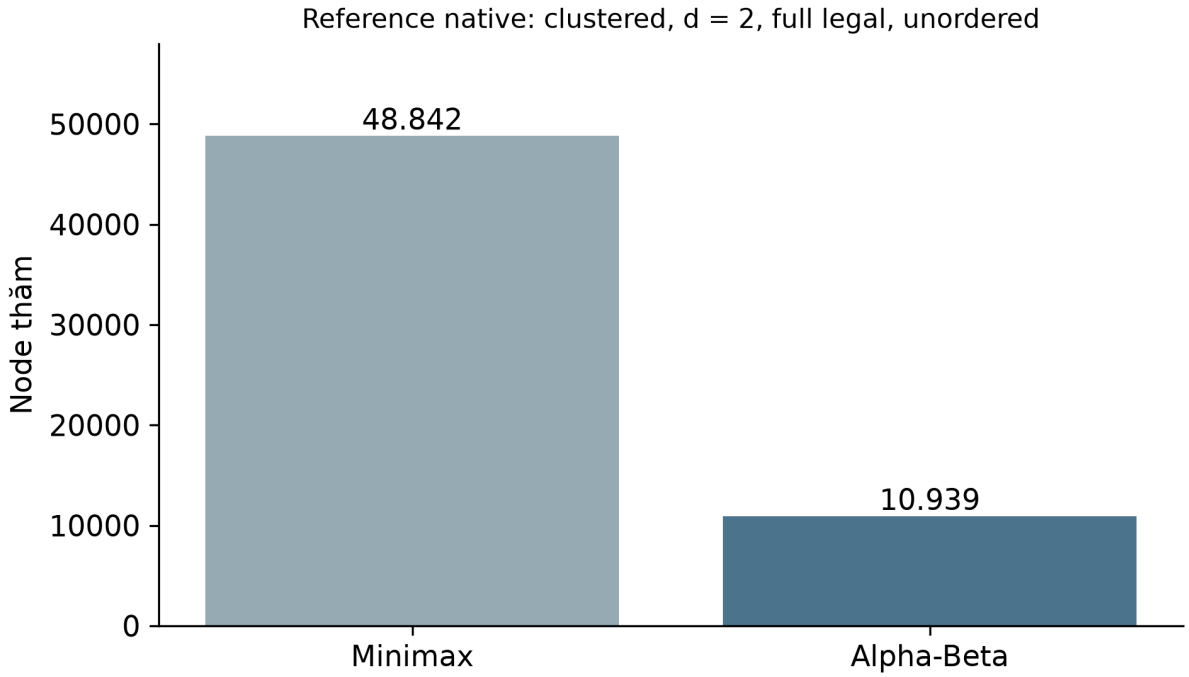
Biểu đồ từ dữ liệu native đã lưu. Nguồn: research/current_2026_10_23/engine_search_summary.csv.

Bộ thực nghiệm native của engine ứng dụng gồm 816 dòng đo trong engine_search_runs.csv; engine_verification.json ghi 408 cặp Minimax–Alpha-Beta, 408 cặp kiểm tra giá trị khi thay thứ tự duyệt và 820 lần kiểm tra khôi phục trạng thái gốc. Các cặp bao gồm những lần lặp, không phải 408 vị trí độc lập. Dữ liệu gồm bốn fixture có kiểm soát và 16 vị trí sinh theo seed; mỗi cấu hình kiểm soát được lặp ba lần liên tiếp. Đồng hồ không tính clone bàn cờ ở gốc; không ghim CPU hay ngẫu nhiên hóa thứ tự, do đó thời gian chỉ mô tả host đã đo.

Ví dụ, tại fixture clustered, độ sâu 2, bán kính ứng viên 2, soft cap 8 và tắt sắp thứ tự duyệt, Minimax thăm 73 node; Alpha-Beta thăm 34 node với 6 cutoff. Cả hai trả điểm 3 và cùng tọa độ được ghi (7,6). Như vậy số node trong ví dụ này giảm khoảng 53,42%, tính bằng $(73 - 34) / 73 \times 100\%$. Trung vị thời gian native tương ứng là 65.040 ns và 43.210 ns, tức khoảng 65,04 µs và 43,21 µs. Với cùng fixture và cấu hình thành viên ứng viên, khi bật sắp thứ tự, Alpha-Beta ghi 26 node, 7 cutoff và

trung vị 38.846 ns. Các số này lấy từ engine_search_summary.csv; chúng minh họa một cấu hình cụ thể, không phải mức tăng tốc chung cho mọi ván.

5.5. Thực nghiệm thuật toán trên reference core



Hình 5.2. So sánh số node trên reference core

Biểu đồ từ dữ liệu native đã lưu. Nguồn: research/week_05_09/results/search_summary.csv.

Nhóm nghiên cứu ngày 17/10 dùng caro-core-reference, với 544 dòng đo trong research/week_05_09/results/search_runs.csv, 112 cặp so sánh Minimax–Alpha-Beta và 112 cặp so sánh thứ tự duyệt. Log ghi 6 unit test và 3 integration test thành công. Tập này cũng gồm bốn fixture kiểm soát và 16 vị trí sinh theo seed; một số fixture chiến thuật là trạng thái tổng hợp để kiểm tra hành vi, không nhất thiết có thể đạt bằng chuỗi lượt đi xen kẽ thông thường. Kết quả không chạy browser, Worker hay binary WASM của trò chơi.

Ở fixture clustered của reference, độ sâu 2, xét toàn bộ nước hợp lệ và không sắp thứ tự, Minimax thăm 48.842 node; Alpha-Beta thăm 10.939 node. Hai kết quả có cùng điểm 0 và cùng nước (6,7), tức số node giảm khoảng 77,60% trong cấu hình đó. Trung vị thời gian của ba lần chạy lần lượt là 69.252 µs và 16.442 µs. Không thể

trộn trực tiếp số này với thí nghiệm ở mục 5.4: thư viện, chính sách sinh nước đi và cấu hình đều khác.

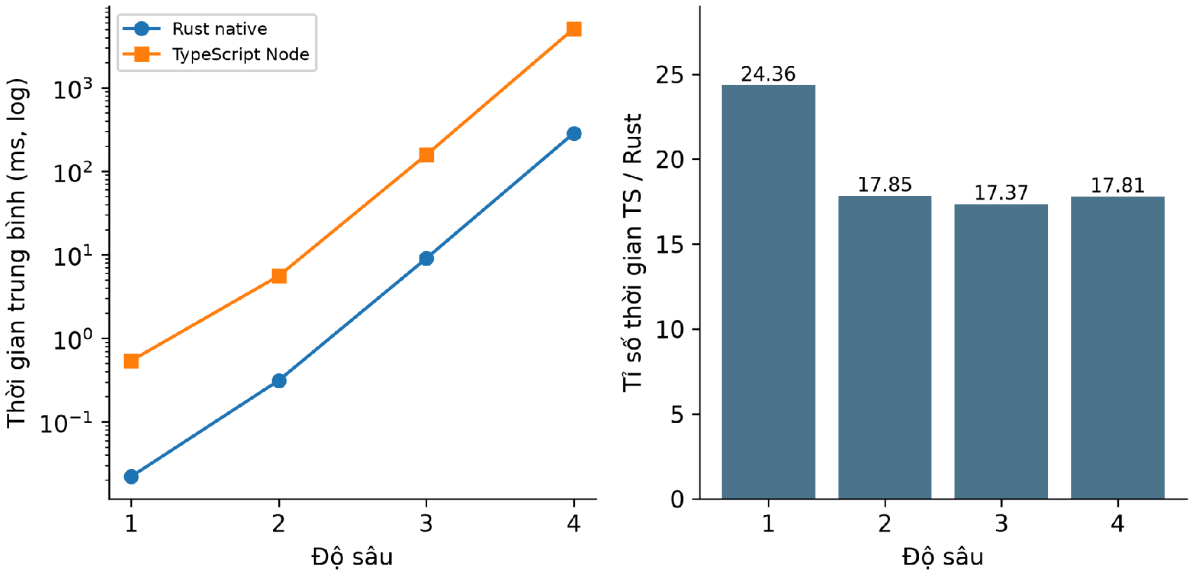
Reference còn cho thấy giới hạn của tối ưu hóa nếu điều kiện sử dụng không phù hợp. Trong cap_loss.json, một cap cứng bằng 6 trên fixture win đã loại cả hai ô thắng ngay, tọa độ (7,2) và (7,7). Đây là ví dụ đối với chính sách của reference; engine ứng dụng có chính sách soft cap ưu tiên nước chiến thuật và được kiểm tra riêng, do đó không thể chuyển nguyên kết luận lỗi sang engine ứng dụng. cache_scope_probe.json cũng ghi trường hợp tái sử dụng bảng chuyển vị sau khi đổi người chơi gốc cho điểm 19.800, trong khi bảng mới cho 0. Kết quả cho thấy cache cần được giới hạn theo ngữ cảnh đánh giá; nó không chứng minh lỗi đang xuất hiện trong đường đi WASM hiện hành.

5.6. Mục tiêu và cấu trúc các tập dữ liệu

File benchmark/results/benchmark_data.json chứa bốn kết quả Rust và bốn kết quả TypeScript cho cùng nhãn fixture midgame, tại độ sâu một đến bốn. benchmark/rust_bench/src/main.rs dùng thư viện caro-core-reference và fixture gồm tám quân đặt sẵn; file benchmark/ts_bench/benchmark.ts cài một bản TypeScript riêng cho phép đối chiếu. scripts/benchmark_rust_vs_ts.py gọi hai chương trình với tham số midgame rồi ghi JSON. Gói Rust chạy native, chương trình TypeScript chạy trong Node; không có phép đo WASM trên browser trong chuỗi này. Manifest Rust tại checkout này trỏ dependency tương đối tới một vị trí ngoài checkout của caro-core-reference, hiện không tồn tại; vì vậy không xác nhận có thể tái chạy nguyên trạng tại đây.

Mã runner có một lần khởi động trước khi đo, sau đó mỗi kết quả là thời gian trung bình chia theo số lần lặp ở từng độ sâu. Bên Rust chạy 100, 50, 20 và 5 lần tương ứng độ sâu 1–4; TypeScript chạy 50, 20, 5 và 2 lần. Cả hai tạo lại trạng thái fixture trong đoạn được tính giờ. Số vòng chạy không giống nhau; JSON không lưu từng mẫu thời gian, phân phối, hash nguồn, điều kiện máy hoặc thời điểm cho lần đo cụ thể. Mặc dù có cùng điểm và số node được ghi, điều đó mới đối chiếu trên fixture và các cấu hình này; nó không chứng minh hai hiện thực tương đương với mọi trạng thái.

5.7. Bảng số liệu được lưu cho một fixture benchmark



Hình 5.3. Benchmark native và TypeScript trên một fixture midgame

Biểu đồ tái dựng dữ liệu benchmark lịch sử. Nguồn: benchmark/results/benchmark_data.json; một fixture, lịch sử.

Bảng 5.1. Dữ liệu benchmark native và TypeScript

Độ sâu	Node Rust / TS	Cutoff Rust / TS	Rust native (ms)	TypeScript Node (ms)	Điểm Rust / TS
1	45 / 45	0 / 0	0,0221	0,5383	99.990 / 99.990
2	609 / 609	39 / 39	0,3125	5,5795	19.990 / 19.990
3	16.184 / 16.184	454 / 454	9,0248	156,7846	9.999.997 / 9.999.997
4	496.790 / 496.790	16.350 / 16.350	285,4517	5.083,4518	9.999.997 / 9.999.997

Trang present/src/components/BenchmarkSection.tsx sao chép những giá trị số này vào mảng mã nguồn nhưng mô tả phép đo trên mười thế cờ; runner và JSON chỉ cho thấy một fixture midgame. Week6AlphaBeta.tsx lại ghi hệ số ở độ sâu 1 và 2 không phù hợp phép chia thời gian từ JSON. Không dùng mô tả mười thế cờ hoặc các hệ số mâu thuẫn làm kết quả đã xác thực. Muốn so sánh tốc độ một cách khái quát cần nhiều vị trí, dữ liệu per-run, cấu hình compiler và runtime, cùng protocol warmup và thứ tự đo được lưu.

Một phép so sánh Rust native với TypeScript Node cũng không thể trực tiếp giải thích độ trễ người chơi gặp trên Web. Đường thực tế còn có khởi tạo và truyền thông điệp đến Worker, tái dựng state, gọi WASM và cập nhật giao diện. Cần đo riêng từng khoảng cùng phân phối độ trễ trên thiết bị đích. Vì vậy các biểu đồ so sánh ở website được xem là phần trình bày dữ liệu lưu của một fixture, không là bằng chứng rằng ứng dụng đạt một mức khung hình cụ thể hay mức tăng tốc phổ quát.

5.8. Khảo sát 100 ván Rule-Based và Random

File benchmark/results/week4_bot_battle.json ghi 100 ván trên Rust native, trong đó Rule-Based thắng 100, Random thắng 0 và hòa 0; trung bình 9,90 ply mỗi ván. “Ply” là một lượt đặt quân của một bên, do đó hai ply thường tương ứng một lượt của mỗi bên. File ghi trung bình 251,26 micro giây cho mỗi nước Rule-Based được tính giờ, tổng thời gian toàn bộ thí nghiệm 135,544 ms, số ô trống trung bình 220,21 và số ứng viên trung bình 60,80. Tỷ lệ tính từ hai trung bình là $1 - 60{,}80 / 220{,}21 \approx 72{,}39\%$; đây là tỷ lệ giảm số ô so với ô trống theo cách tổng hợp của thí nghiệm, không phải tỷ lệ cắt nhánh Alpha-Beta hoặc bằng chứng giữ được mọi nước tốt.

Theo benchmark/rust_bench/src/bin/week4_experiment.rs, Rule-Based đánh X trong 50 ván đầu và O trong 50 ván sau; Random sử dụng seed xác định từ số ván và ply. Thế khởi đầu và chính sách của đối thủ do chương trình quyết định. Seed của hai nhóm đổi bên không được dùng theo cặp giống nhau, nên đổi bên không tự loại bỏ hoàn toàn ảnh hưởng của lượt đi đầu. JSON chỉ có số tổng hợp, không có diễn biến từng ván hoặc khoảng tin cậy. Kết quả 100/100 mô tả một bộ thí nghiệm xác định với baseline Random, không suy ra xác suất thắng phổ quát hay sức mạnh trước con người hoặc đối thủ khác.

Đồng hồ trong mã bao gồm thao tác sinh ứng viên và gọi choose_rule_based_move, vì vậy chỉ số 251,26 µs cần được hiểu theo phạm vi đó. Tệp JSON không ghi giá trị cực đại cho mỗi nước. Ngược lại, present/src/components/Week4RustEngine.tsx hiển thị 42,24 µs trung bình và gọi 251,26 µs là cực đại; cả hai nhãn đều không khớp file dữ liệu đang lưu. PythonVisualizer.tsx và một số tài liệu Week4 cũng nhúng 42,24. Vì thiếu bản ghi gốc tương ứng với con số này, báo cáo sử dụng 251,26 µs

như giá trị trung bình được ghi trong JSON và không diễn giải nó thành tốc độ khung hình browser.

Bộ JSON đồng thời liệt kê bảy fixture phân loại chuỗi liên tiếp đều PASS: năm quân, bốn mở, bốn một đầu, ba mở, ba một đầu, hai mở và hai một đầu. Chương trình kiểm tra các fixture trước khi xuất kết quả; điều này mô tả bảy trường hợp cụ thể, không chứng minh đầy đủ mọi mẫu gián đoạn, chồng lấn, fork hoặc mọi vị trí cạnh biên. Trung bình ứng viên khai cuộc và trung cuộc được ghi lần lượt 45,19 và 92,95; trường tàn cuộc có 0,00 nhưng file không ghi số mẫu từng giai đoạn, nên không đủ căn cứ diễn giải 0 là một phép đo tàn cuộc có mẫu.

5.9. Truy nguyên biểu đồ và dữ liệu minh họa

present/public/data/benchmark_data.json và present/public/data/week4_bot_battle.json khớp byte với hai tệp tương ứng trong benchmark/results/ tại thời điểm kiểm kê. Điều đó xác nhận hai bản sao cùng nội dung, nhưng không chứng minh mọi component đang đọc trực tiếp file JSON: nhiều component chứa giá trị hard-code. Khi thay dữ liệu gốc, một mảng hard-code hoặc ảnh đã xuất có thể không đổi theo, tạo mâu thuẫn giữa bảng, biểu đồ và nguồn.

present/scripts/generate_charts.py nhúng các dãy thời gian cho biểu đồ Rust–TypeScript, tạo present/public/assets/rust_vs_ts_speedup.png. Do script dùng mảng số cố định, ảnh chỉ phản ánh các giá trị được chép vào script tại lúc xuất; cần so lại với JSON trước khi trình bày. Cùng script tạo chebyshev_heatmap.png từ năm quân đặt thủ công; việc đếm tập hợp trong đoạn script cho 36 ô ứng viên. Đây là ví dụ hình học minh họa một vị trí và quy tắc bán kính 2, không phải phép đo trung bình của tập ván.

scripts/export_visuals.py dùng mảng số node viết sẵn cho hình assets/charts/search_scaling_comparison.png; trong phạm vi tệp đã đọc chưa thấy bản ghi từng fixture hoặc nguồn tính trung bình tương ứng. Nên dùng hình này như minh họa hoặc thay bằng hình sinh từ CSV có provenance, không dùng để báo cáo số đo. assets/debug/board_state_example.png do script vẽ các quân và đường thắng mẫu, không phải ảnh chụp một ván hoặc minh chứng tương tác thực tế.

scripts/generate_week4_visuals.py đọc một phần JSON Week4, nhưng biểu đồ donut gắn trực tiếp 100/0/0 và các giá trị ô trống 222/212 của từng giai đoạn được đặt sẵn trong script. Số tổng hợp và ví dụ suy diễn được đặt trên cùng một biểu đồ nên chú thích phải chỉ rõ nguồn từng thành phần. Các hình taxonomy và flow thể hiện khái niệm, không tự kiểm chứng code; sơ đồ flow còn gắn con số độ trễ không khớp JSON. Không mặc định một ảnh chất lượng cao là bằng chứng đo lường hoặc xác nhận mã hiện hành.

present/src/components/Week8To10AdvancedAI.tsx chứa bảng move ordering 148/52 đến 382.100/78.450 và một kịch bản giải đấu, nhưng trong các tập raw đã kiểm kê chưa tìm thấy provenance liên kết những số này với một lần chạy cụ thể. Chúng được giữ là nội dung trình bày chưa xác thực, không dùng để thay số từ research/week_05_09/results/search_summary.csv. Tương tự, kịch bản 500 ván trong trang trình bày là kế hoạch; không xem đó là giải đấu đã diễn ra khi chưa có kết quả từng ván và cách sinh seed.

5.10. Cách dùng evidence và bảo toàn khả năng tái lập

Các tệp công khai trong research/current_2026_10_23 ghi dữ liệu và provenance của snapshot engine/game; research/week_05_09 ghi nghiên cứu native reference. Runner có thể ghi đè kết quả, vì vậy phải lưu bản sao và kiểm tra dependency trước khi chạy lại. Thư viện caro-core-reference đầy đủ chưa có trong snapshot công khai. Các log .log nêu ở mục 5.2 và hồ sơ kiểm lại ngày 27/10 là hồ sơ local của tác giả; chúng không được trình bày như tệp đã công bố trên GitHub. JSON, CSV và trích đoạn công khai hỗ trợ đúng các quan sát được ghi trong từng tệp, không thay thế toàn bộ source và log còn thiếu. [2] [3]

Mỗi con số nên được lưu cùng fixture hoặc seed, thuật toán, config candidate, độ sâu, số lần lặp, định nghĩa counter, môi trường và đường dẫn raw. Hình ảnh chỉ nên được coi là phép biến đổi của dữ liệu nếu script lấy đúng nguồn, không chèn số thủ công và có đủ thông tin để chạy lại. Các bước tiếp theo cần tách tập dành cho kiểm chứng correctness khỏi tập đánh giá performance; sự khớp điểm trên vài trường hợp là bằng chứng hữu hạn, không phải chứng minh thống kê hay chứng minh mọi trường hợp.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết luận

Đồ án đã xây dựng một hệ thống chơi cờ với bàn cờ 15×15, luật Freestyle và các chiến lược bot có thể lựa chọn, đồng thời tổ chức xử lý tìm nước đi qua Rust/WASM và Worker. Từ mô hình trạng thái và kiểm tra thắng theo bốn trục, đồ án triển khai và khảo sát heuristic, Random, Rule-Based, Minimax, Alpha-Beta, sinh ứng viên và sắp thứ tự. Phần giao diện thể hiện bàn cờ, trạng thái và thông tin tìm kiếm trong những luồng đã được kiểm tra.

6.2. Hạn chế

Corpus thực nghiệm còn nhỏ so với không gian trạng thái của bàn cờ; các phép kiểm không chứng minh tính đúng đắn toàn diện hoặc khả năng chơi tối ưu. Thời gian chạy được thu thập trên một host với số lần lặp hạn chế, chưa kiểm soát đầy đủ tải hệ thống; vì vậy kết quả mang tính minh họa và tái lập theo cấu hình đã lưu. Các phép kiểm browser là tự động, dùng pointer tổng hợp; chưa thay thế đánh giá trên thiết bị, Wink host hay kiểm thử với người dùng. Chưa có dữ liệu thi đấu bot đủ lớn hoặc quy trình đánh giá sức mạnh, nên không thể suy ra xác suất thắng hay thứ hạng thực tế từ tên difficulty hoặc độ sâu.

6.3. Hướng phát triển

Trước hết, có thể mở rộng corpus kiểm thử với các trạng thái đạt được từ lịch sử hợp lệ, thêm fixture cho trường hợp hai mối đe dọa, nước thắng có khoảng trống và thế hòa. Mỗi tối ưu nên giữ bộ so sánh cùng luật, cùng hàm lượng giá, cùng độ sâu và cùng tập nước đi; kiểm tra khôi phục state sau cả đường bình thường lẫn đường bị hủy. Đối với cache, khóa hoặc phạm vi sống cần phản ánh những yếu tố ảnh hưởng đến giá trị như người chơi gốc, luật và cấu hình; cần xử lý đúng bound và điểm thắng theo độ sâu trước khi gắn vào ứng dụng.

Tiếp theo, nếu triển khai iterative deepening và giới hạn thời gian trong Worker, hệ thống cần trả nước đi từ độ sâu hoàn tất gần nhất, khai báo rõ phạm vi bộ đếm, đồng thời bảo đảm Undo, Reset và đổi ván không áp dụng kết quả cũ. Sau tích hợp

cần kiểm lại parity, tính hợp lệ của nước đi và khả năng khôi phục khi Worker lỗi hoặc bị hủy. Việc chọn cấu hình difficulty nên dựa trên thực nghiệm độc lập, không chỉ tăng tên gọi hay độ sâu.

Cuối cùng, một protocol đánh giá sức mạnh có thể sử dụng nhiều vị trí mở đầu, đổi bên, seed được công bố và số ván đủ lớn; báo tỷ lệ thắng–hòa–thua cùng độ bất định. Đánh giá thời gian cần ghi thiết bị, trình duyệt, phiên bản, điều kiện tải và phân phối thay vì một con số đơn lẻ. Kiểm thử trên màn hình cảm ứng thật, nhiều viewport và môi trường host sẽ bổ sung bằng chứng trải nghiệm mà các phép kiểm headless hiện chưa cung cấp.

TÀI LIỆU THAM KHẢO

[1] tducn110, “CaroGame/src” và “SOURCE_EXCERPTS.md,” basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/tree/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src ; https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/current_2026_10_23/SOURCE_EXCERPTS.md. [Truy cập: 27/10/2026].

[2] tducn110, “Current engine and browser evidence, 23 October 2026,” basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/tree/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/current_2026_10_23. [Truy cập: 27/10/2026].

[3] tducn110, “Week 05–09 reference experiments and results,” basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/tree/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/week_05_09. Thư viện caro-core-reference đầy đủ không nằm trong snapshot công khai này. [Truy cập: 27/10/2026].

[4] tducn110, “Project plan” và “Week 05–09 report index,” basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/PLAN.md ; https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/Week_05_09_Report_Index.md. [Truy cập: 27/10/2026].

[5] The Rust Project Developers, “Type layout” và “Array types,” The Rust Reference. [Online]. Available: https://doc.rust-lang.org/reference/type-layout.html ; https://doc.rust-lang.org/reference/types/array.html. [Truy cập: 27/10/2026].

[6] C. E. Shannon, “Programming a computer for playing chess,” Philosophical Magazine, vol. 41, no. 314, pp. 256–275, 1950, doi: 10.1080/14786445008521796. [Online]. Available: https://doi.org/10.1080/14786445008521796.

[7] D. E. Knuth and R. W. Moore, “An analysis of alpha-beta pruning,” Artificial Intelligence, vol. 6, no. 4, pp. 293–326, 1975. https://doi.org/10.1016/0004-3702(75)90019-3

[8] A. L. Zobrist, “A New Hashing Method With Application for Game Playing,” University of Wisconsin–Madison, Technical Report 88, 1970. https://research.cs.wisc.edu/techreports/1970/TR88.pdf

[9] WHATWG, “Workers,” HTML Living Standard. [Online]. Available: https://html.spec.whatwg.org/multipage/workers.html. [Truy cập: 27/10/2026].

[10] WebAssembly Community Group, “WebAssembly specifications.” [Online]. Available: https://webassembly.org/specs/. [Truy cập: 27/10/2026].

[11] tducn110, “Presentation source, assets and demonstrations,” basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/tree/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/present. [Truy cập: 27/10/2026].

[12] tducn110, “Native Rust/TypeScript benchmark harnesses and results,” basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/tree/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/benchmark. [Truy cập: 27/10/2026].

PHỤ LỤC A. TỔNG HỢP CÔNG VIỆC THEO TUẦN

A.1. Tuần 1: mô hình và phạm vi đề tài

Week1/Tuần 1.md cùng đề cương PDF xác lập bài toán website cờ, bảng 15×15, trạng thái và các mục tiêu bot. Nghiên cứu luật phải làm rõ tên biến thể vì cùng tên Caro có nhiều quy ước; mã hiện hành dùng Freestyle, thắng khi một trục có ít nhất năm quân. Đề cương đề xuất các chức năng và kỹ thuật, còn mã nguồn cùng bằng chứng mới quyết định mức độ đã hiện thực. Các hình Mermaid ở Week1 là tài liệu minh họa, không phải phép chạy hệ thống.

A.2. Tuần 2: thao tác trạng thái và kiểm tra kết thúc

Week2/Tuan 2.md mô tả ma trận, kiểm tra nước hợp lệ, cập nhật người chơi và bốn hướng kiểm tra. Bàn cờ hữu hạn cho phép quét cục bộ từ nước cuối, tránh phải tìm lại mọi chuỗi sau mỗi thao tác. Các test hiện hành trong CaroGame/tests/game/core.test.ts và engine Rust cần được đọc theo phạm vi của chúng; các con số test trong một bản tuần cũ không thay cho log chạy của phiên bản khác. Quy tắc thắng và hòa phải kiểm trước khi chấp nhận nước tiếp theo.

A.3. Tuần 3: giao diện và vòng đời ván chơi

Week3/Tuan 3.md, Week3/assets và source CaroGame ghi nhận công việc tổ chức giao diện, bàn cờ, sự kiện và trạng thái. Các sơ đồ UI cần được xem như bản thiết kế của thời điểm xuất; source hiện tại cho thấy controller điều phối game, React hiển thị và Pixi xử lý cảnh bàn cờ. Chức năng Undo, Reset và hủy yêu cầu bot cần được kiểm tra dưới điều kiện bất đồng bộ vì kết quả từ một ván cũ có thể đến sau khi trạng thái đã thay đổi. Các fixture controller và Worker giúp kiểm phần này, song không thay thế kiểm thử thao tác vật lý.

A.4. Tuần 4: hình thế, lượng giá và baseline

Các file riêng trong Week4 giải thích nhận diện chuỗi, hàm lượng giá, Random Bot và Rule-Based Bot. Phân biệt các trọng số trong thư viện reference với trọng số của engine game: hai module có thể cùng tên chiến lược nhưng không cùng tham số. Thí nghiệm Rule-Based–Random và bảy fixture được bàn ở mục 5.8 là bằng chứng hữu hạn; chúng không chứng minh chính xác toàn bộ taxonomy. Ảnh biểu đồ và

trang present có các giá trị chưa đồng bộ, do đó phải đối chiếu với JSON và script trước khi trích dẫn.

A.5. Tuần 5: Minimax và cây trò chơi

Week5/Week_05_Rust_Minimax_Search_Tree_Analysis.md giải thích cây trạng thái, lượt MAX/MIN, độ sâu và hoàn tác. Giá trị của một nước phụ thuộc hàm đánh giá tại lá và tập ứng viên; giảm độ sâu tiết kiệm công việc nhưng có thể bỏ sót hậu quả xa hơn. research/week_05_09 lưu fixture và các phép đối chiếu reference; engine game cũng có Minimax làm baseline trong corpus riêng. Không nên coi các số trong một corpus như số đo của đường Worker.

A.6. Tuần 6: Alpha-Beta

Week6/Week_06_Rust_Alpha_Beta_Pruning_and_Transposition_Tables.md nghiên cứu việc duy trì cận alpha và beta, dừng duyệt khi cận cho thấy nhánh không thể cải thiện quyết định. Với cùng cây và cùng hàm đánh giá, dữ liệu corpus ghi nhận các cặp so sánh giá trị; số lượng node được giảm phụ thuộc vị trí và thứ tự. Một cutoff là sự kiện dừng xét nhánh, còn số node tiết kiệm không được suy trực tiếp từ số cutoff. Các con số trong Chương 5 là ví dụ có cấu hình và nguồn CSV rõ ràng.

A.7. Tuần 7: ứng viên, thứ tự và cấu hình độ khó

Week7/Week_07_Candidate_Move_Generation.md xét tập ô gần quân đã có theo khoảng cách Chebyshev và chi phí của giới hạn ứng viên. Ở engine game hiện tại, chính sách soft cap giữ nước hoàn tất thắng hoặc chặn thắng ngay trước khi lấy thêm ứng viên thường. Điều đó bảo vệ một nhóm tình huống chiến thuật trong corpus, nhưng không bảo đảm mọi nước phát triển hoặc fork được chọn. Sáu nhãn giao diện là cấu hình chiến lược và độ sâu, chưa phải thang sức mạnh đã calibration bằng giải đấu.

A.8. Tuần 8: move ordering

Week8/Week_08_Move_Ordering_Optimization.md tập trung vào thứ tự xét các nước mà không thay tập thành viên. Nếu cùng cây, thay thứ tự không làm thay giá trị Minimax lý tưởng, nhưng có thể thay số nhánh cắt và nước được chọn khi hòa điểm. Chi phí xếp hạng cần được tính, vì ít node hơn không chắc làm tổng thời gian giảm.

Corpus reference ghi cả ordering_evaluations để phân biệt công việc xếp hạng với số lá tìm kiếm; bảng minh họa hard-code trong present chưa có truy xuất raw phù hợp.

A.9. Tuần 9: Zobrist và bảng chuyển vị

Week9/Week_09_Zobrist_Hashing_and_Transposition_Table.md bàn về khóa tăng dần, lưu giá trị và kiểu cận trong reference. verification.json ghi 3.998 kiểm tra hash khi đi và 3.998 khi hoàn tác; tt_semantics.json ghi các lớp exact, lower và upper trong fixture đã xem. Probe cho thấy việc dùng lại bảng khi thay root có thể sai điểm, nên cần xác định namespace và xử lý điểm mate trước khi tích hợp. Đây là kết quả nghiên cứu reference, không xác nhận Master trong engine game hiện tại dùng bảng đó.

A.10. Tuần 10: iterative deepening và kế hoạch đánh giá

Week10/Week_10_Iterative_Deepening_Time_Budget_and_Evaluation.md mô tả tìm kiếm tăng dần độ sâu và đề xuất protocol đánh giá bot. research/week_05_09/results/budget_abort.json ghi một probe ngân sách node trong reference: state được khôi phục và độ sâu hoàn tất cuối cùng là 2. Tổng node báo cáo trong probe không tính vòng lặp bị ngắt, nên không gọi 40 là toàn bộ node đã thử. Nội dung giải đấu, time budget thực và các chức năng tích hợp vào game cần bằng chứng thực thi riêng trước khi ghi hoàn thành.

Mốc bàn giao và điều kiện hoàn thành theo tuần. Đây là lịch tương đối để đối chiếu sản phẩm; không phải biên bản xác nhận mỗi tuần đã đạt. Ngày bắt đầu, kết thúc và lịch duyệt cần xác nhận riêng. W09–W10 thuộc nghiên cứu reference; việc tích hợp vào engine game cần bằng chứng riêng.

W01: Mô hình hóa bàn cờ, tác nhân, phạm vi Freestyle và ranh giới nguồn; bàn giao mô hình bài toán cùng giả thiết.

W02: Đặc tả tọa độ và thao tác trạng thái; minh họa nước hợp lệ/không hợp lệ, thắng và hòa bằng fixture có tên.

W03: Mô tả ownership giao diện và vòng đời ván; truy vết nước đi, Undo, Reset và request cũ theo từng thành phần.

W04: Mô tả chuỗi, lượng giá và baseline Random/Rule-based; có ví dụ tính lại được và xác định đúng implementation.

W05: Đặc tả đệ quy Minimax, độ sâu, bất biến apply/undo và ví dụ truy vết; xác định tập ứng viên được kiểm.

W06: Đặc tả cận và cắt tỉa Alpha-Beta; so điểm root với Minimax trong cùng ngữ cảnh tìm kiếm cố định.

W07: Đặc tả radius, soft cap, giữ nước chiến thuật tức thời và sáu cấu hình; nêu giới hạn lọc ứng viên.

W08: So sánh thứ tự trên cùng tập nước; ghi rõ điểm, node và chi phí xếp hạng thay vì mặc định tăng tốc.

W09: Mô tả cập nhật Zobrist và các cận TT; kiểm khôi phục và nêu rủi ro va chạm, đổi root và dùng lại điểm mate.

W10: Đặc tả iterative deepening và ngắt theo ngân sách; thể hiện kết quả hoàn tất gần nhất cùng khôi phục, ghi tích hợp hoặc đánh giá giải đấu là chờ nếu thiếu bằng chứng thực thi.

PHỤ LỤC B. KIỂM KÊ THƯ MỤC VÀ HỒ SƠ LIÊN QUAN

B.1. Source, sản phẩm và hồ sơ theo tuần

CaroGame/ là ứng dụng web có package riêng, source React/TypeScript, lớp bàn cờ Pixi, game core, controller, adapter Worker/WASM, i18n, tích hợp host và các file test. present/ là website trình bày nghiên cứu riêng, có component theo tuần, mô phỏng và dữ liệu/ảnh tĩnh; nó không phải bằng chứng rằng mọi thành phần được minh họa đang chạy trong game. Week1/ đến Week10/ là tài liệu và hình theo chủ đề từng tuần như Phụ lục A; một số tuần có bản DOCX/PDF được tạo trước snapshot Markdown cập nhật, nên không mặc nhiên xem mọi bản xuất là đồng bộ.

benchmark/ chứa Rust native harness, TypeScript harness và hai JSON tổng hợp. research/ chứa hai nhóm thí nghiệm có ngày, fixture, raw CSV, JSON và snapshot baseline. assets/ chứa hình, sơ đồ và ảnh trạng thái dựng; mỗi ảnh phải xác định là sơ đồ khái niệm, ví dụ thủ công hay biểu đồ từ dữ liệu đo. scripts/ chứa công cụ sinh ảnh, điều phối benchmark và cập nhật roadmap; sự tồn tại của script cho phép xem quy trình dự kiến nhưng không tự chứng minh script đã được chạy ở trạng thái hiện tại.

rules/ chứa hướng dẫn quản trị layout, roadmap và hình ảnh của project; archived/ giữ draft và roadmap cũ để tham khảo lịch sử. .gemini/ và .obsidian/ chứa cấu hình hỗ trợ môi trường tài liệu, còn .git/ chứa metadata của repository. Các thư mục cấu hình và lịch sử không phải tính năng trò chơi; nội dung trong đó không được xem là kết quả đo hoặc tự động có hiệu lực như yêu cầu hiện tại.

B.2. File gốc, mẫu và giới hạn của bản kiểm kê

Ở root, README.md, LAYOUT.md, PLAN.md, ROADMAP.md, Week_05_09_Report_Index.md, Caro_Rules_Heuristic_Rust_WASM_Review.md và BENCHMARK_RUST_VS_TYPESCRIPT.md là các tài liệu định hướng hoặc mô tả. Hai file Mau bao cao do an co so.docx và templatereport.docx là mẫu người dùng cung cấp; nội dung mẫu cần được phân biệt với yêu cầu trực tiếp. Các file cấu hình .gitattributes, .gitignore, .mise.toml phục vụ repository/môi trường. Báo cáo DOCX được tạo trong lượt làm việc là output, không phải nguồn chứng minh lịch sử implementation.

Danh sách máy đọc trong tệp report_caro_2026/evidence_inventory.json liệt kê từng file đã thấy dưới các thư mục, kèm vai trò root, số lượng và SHA-256 các dataset chính; danh sách bỏ qua nội bộ .git, node_modules, target và dist. Kiểm kê tên file giúp bảo đảm không bỏ thư mục, nhưng không có nghĩa mọi claim trong mọi file đã được xác thực. Nguồn có mức độ tin cậy cao hơn cho hành vi hiện hành là code cùng log và raw gắn đúng phiên bản; kế hoạch, roadmap, biểu đồ hard-code và bản trình bày cần được ghi theo vai trò tương ứng.

PHỤ LỤC C. HƯỚNG DẪN CHẠY VÀ HỒ SƠ BÀN GIAO

C.1. Chạy ứng dụng chính

Ứng dụng chính có package riêng trong CaroGame. Sau khi chuẩn bị Node.js tương thích và npm, cài dependency theo lockfile, chạy dev server hoặc kiểm thử/build bằng các script của package. Các lệnh dưới đây mô tả quy trình, không thay thế kết quả kiểm tra môi trường cụ thể.

```
git clone https://github.com/tducn110/basicproject.git
cd basicproject
git checkout 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc
cd CaroGame
npm ci
npm run typecheck
npm test
npm run build
```

Để mở dev server, chạy npm run dev ở terminal riêng; lệnh này duy trì tiến trình ở foreground cho tới khi dừng. Các lệnh kiểm thử và build phía trên chạy độc lập với dev server.

Trong lần biên soạn này, Node.js 24.22.1 được dùng cho test, typecheck và build. Vite 8.3.1 đóng gói thành công, ghi WASM 111,58 kB trước gzip và 34,14 kB sau gzip. Cảnh báo __dirname trong config và chunk lớn hơn 500 kB vẫn xuất hiện. Không tự sửa dependency hoặc cấu hình project để làm thay đổi phạm vi của báo cáo.

C.2. Tái chạy nghiên cứu và giới hạn dependency

Các lệnh dưới giả định một shell mới tại thư mục gốc basicproject ở commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc. Bản công khai có Rust baseline, harness và trích đoạn, nhưng chưa chứa đầy đủ engine sửa và caro-core-reference của phép đo lịch sử. Chỉ chạy các lệnh nghiên cứu sau khi có đúng dependency và xác minh hash; không thay một thư viện tùy ý rồi gọi là tái lập. Lưu kết quả mới ở thư mục riêng để giữ hồ sơ gốc. [1] [2] [3]

```
# Shell mới tại thư mục gốc basicproject
cargo test --manifest-path gomoku-main/engine/Cargo.toml
cargo test --manifest-path caro-core-reference/Cargo.toml
```

Hai lệnh Cargo trên là hướng dẫn theo vị trí source đã quan sát; báo cáo này không ghi chúng như lệnh vừa chạy ngày 27/10. Các kết quả native trong chương 5 có ngày và nguồn riêng.

C.3. Danh mục toàn bộ thư mục gốc

Bảng C.1. Kiểm kê các thư mục gốc theo vai trò

Thư mục	Số file trong kiểm kê	Vai trò
.gemini	3	Cấu hình hướng dẫn cục bộ
.git	metadata	Metadata và lịch sử repository
.obsidian	4	Cấu hình môi trường Obsidian
CaroGame	83	Ứng dụng React/Pixi, core TypeScript, Worker/WASM và kiểm thử
Week1	6	Đề cương, luật, mô hình trạng thái và hình
Week10	3	Iterative deepening và kế hoạch đánh giá
Week2	2	Core TypeScript và ghi chú kiểm chứng
Week3	4	Báo cáo UI, vòng đời và sơ đồ
Week4	10	Pattern, heuristic, bot cơ sở và hình
Week5	4	Nghiên cứu Minimax và thí nghiệm reference
Week6	4	Nghiên cứu Alpha-Beta
Week7	3	Sinh ứng viên và cấu hình mức khó
Week8	3	Nghiên cứu thứ tự duyệt
Week9	3	Zobrist và bảng chuyển vị
archived	7	Bản nháp và roadmap lịch sử
assets	8	Biểu đồ, sơ đồ và bàn mẫu
benchmark	8	Harness Rust native, TypeScript Node và JSON tổng hợp
present	46	Website giải thích, mô phỏng và bản sao dữ liệu

Thư mục	Số file trong kiểm kê	Vai trò
research	66	Dữ liệu engine/reference, snapshot và provenance
rules	4	Hướng dẫn layout, roadmap và hình ảnh
scripts	7	Script sinh hình, benchmark và roadmap

Danh sách chi tiết từng file và hash dataset được đóng gói tại report_caro_2026/evidence_inventory.json. Phạm vi kiểm kê loại nội bộ Git, node_modules, target và dist; các thư mục output báo cáo tạo sau thời điểm kiểm kê được ghi trong manifest bàn giao.

C.4. Hình ảnh và khả năng cập nhật

Bộ hình của báo cáo được tạo bằng Python với matplotlib và Pillow. Script dựng sơ đồ thuật toán, vẽ biểu đồ trực tiếp từ CSV/JSON, crop các vùng ảnh tài liệu lịch sử và chuẩn hóa ảnh chụp giao diện. Mỗi hình có caption, loại nội dung và nguồn. Ảnh browser ngày 27/10 minh họa bản build local và một lượt bot Hard; thao tác trên canvas dùng sự kiện pointer tổng hợp. Đây không phải kiểm thử ngón tay trên thiết bị thật hoặc kết quả benchmark tổng quát.

C.5. Phần còn thiếu và điều kiện chốt bản nộp

Thông tin sinh viên, MSSV, lớp, giảng viên và năm học đã được cung cấp. Ngày bắt đầu, ngày kết thúc thực hiện và ngày nộp cụ thể chưa được xác nhận. Tên mở rộng trên bìa được xây dựng từ nội dung đề cương, cần khớp tên đề tài đã được giảng viên phê duyệt. Báo cáo đã trình bày những phần còn nghiên cứu, các claim của present chưa có provenance và giới hạn nghiệm thu. Nếu môn học có quy định riêng về bìa, phông hoặc dung lượng, áp dụng quy định chính thức đó trước khi nộp.

THE UNIVERSITY OF DANANG

VN-UK INSTITUTE FOR RESEARCH AND EXECUTIVE EDUCATION





DEVELOPMENT OF A CARO WEB APPLICATION USING ADVERSARIAL SEARCH ALGORITHMS

BASIC PROJECT REPORT — ENGLISH VERSION

Student: Nguyễn Tâm Đức

Student ID: 24020005

Supervisor: Nguyễn Văn Thọ

Class: CSE24

Academic year: 2026–2027

ABSTRACT

This project studies the design and implementation of a Caro web application together with a set of adversarial-search experiments and teaching materials. The report specifies a 15 × 15 Freestyle game, the board and move-history contracts, and the algorithms used to select and evaluate moves. It explains random and rule-based baselines, contiguous-run heuristics, tactical checks, Chebyshev candidate generation, soft candidate caps, Minimax and Alpha-Beta. It also treats iterative deepening, Zobrist hashing, transposition-table bounds, quiescence and incremental pattern counting in their corresponding reference or legacy implementations. Each algorithm is presented with its inputs, state, decision procedure, invariants, complexity, worked examples and limitations.

The application combines React interface components, a Pixi board, an authoritative controller and a Worker-based Rust/WebAssembly route. Request identity, round identity, cancellation, coordinate conversion and telemetry validation preserve the connection between a proposed AI move and the round in which it may be committed. The separate presentation website provides interactive explanations and Python/Pyodide examples. Its demonstrations are analysed individually rather than treated as measurements of the game engine.

Evaluation distinguishes published native experiments, binary/adapter parity, actual browser Worker observations, selected UI interactions and historical cross-language benchmarks. GitHub references identify immutable source and evidence snapshots. The analysis includes negative findings such as tactical moves discarded by a reference candidate cap, increased timing despite fewer searched nodes, and unsafe table reuse across a changed context. The resulting report connects algorithmic principles to concrete implementation while stating the boundaries of finite tests, source availability and reproducibility.

Keywords: Caro; Gomoku; Minimax; Alpha-Beta; candidate generation; Rust; WebAssembly; game-state consistency.

CONTENTS

CHAPTER 1. INTRODUCTION … 1
1.1 Background and motivation … 1
1.2 Objectives and research questions … 1
1.3 Scope, versions and source identity … 2
1.4 Method and report structure … 3
CHAPTER 2. ALGORITHMS AND DATA STRUCTURES … 4
2.1 Scope, evidence, and notation … 4
2.2 Board representation and legal updates … 5
2.3 Freestyle win detection … 8
2.4 Baselines: random play and rule-based play … 10
2.5 Static evaluation and terminal utility … 13
2.6 Candidate generation and a soft tactical cap … 15
2.7 Move ordering versus move selection … 18
2.8 Minimax search … 19
2.9 Alpha-beta pruning … 21
2.10 Iterative deepening and budgets … 23
2.11 Zobrist hashing and incremental state … 25
2.12 Transposition table: entries, bounds and context … 27
2.13 Quiescence search in the legacy TypeScript bot … 29
2.14 Legacy bitmask patterns and incremental line counts … 31
2.15 Difficulty configurations and the active request path … 33
2.16 Synthesis and limits of the algorithmic claims … 34
CHAPTER 3. SYSTEM ANALYSIS AND DESIGN … 36
3.1 Actors, use cases and functional requirements … 36
3.2 Non-functional requirements and acceptance boundaries … 38
3.3 Architecture and ownership … 39
3.4 Board state, history and legal transition contract … 40
3.5 Match phases and state transitions … 41
3.6 Asynchronous request protocol and cancellation … 42
3.7 Coordinate transformation and gesture interpretation … 43
3.8 Search-result contract and presentation of telemetry … 44
3.9 Verification scenarios derived from the design … 45
CHAPTER 4. IMPLEMENTATION … 47
4.1 Toolchain and project organization … 47
4.2 Accepted move processing … 47
4.3 AI dispatch, Worker execution and WASM hydration … 48
4.4 Board rendering and interface synchronization … 50
4.5 Presentation website and interactive explanation … 51
4.6 Python/Pyodide execution and four instructional programs … 51
4.7 Implementation traceability and limits … 54
CHAPTER 5. RESULTS AND EVALUATION … 55
5.1 Evaluation questions, evidence and boundaries … 55
5.2 Native board and search checks of the 23 October engine … 55

5.3 WebAssembly parity and browser observations … 57
5.3.1 Follow-up local application validation, 27 October … 58
5.4 Separate native reference experiments … 59
5.5 Historical cross-language and Week 4 datasets … 62
5.6 Presentation examples and evidence quality … 64
5.7 Threats to validity and interpretation … 64
CHAPTER 6. CONCLUSION AND FUTURE WORK … 66
6.1 Work completed and contribution … 66
6.2 Limitations … 66
6.3 Future development … 67
REFERENCES … 69
APPENDIX A. REPOSITORY INVENTORY AND EVIDENCE MAP … 74
APPENDIX B. PUBLIC SOURCE GAPS AND CLAIM REGISTER … 77
APPENDIX C. READING AND REPRODUCIBILITY PROCEDURE … 78

LIST OF FIGURES

Figure 2.1. Row-major board representation … 6
Figure 2.2. Search must restore its caller state … 8
Figure 2.3. Last-move scan along four axes … 10
Figure 2.4. Rule-based tactical priorities … 12
Figure 2.5. Contiguous-run weights of the archived Rust evaluator … 14
Figure 2.6. Radius and board edges affect candidate membership … 16
Figure 2.7. Alternative traversal orders on identical membership … 19
Figure 2.8. A two-level Minimax example … 21
Figure 2.9. Alpha-Beta cutoff on an unchanged tree … 23
Figure 2.10. Iterative deepening and incomplete work … 25
Figure 2.11. Incremental XOR and turn-key handling … 27
Figure 2.12. Bound semantics in a transposition table … 29
Figure 3.1. Responsibilities in the playable application … 39
Figure 3.2. Match phases and the separate AI waiting flag … 41
Figure 3.3. Inverse camera transform before board bounds checking … 43
Figure 4.1. Tactical and Rust routes converge at the guarded commit … 49
Figure 4.2. Union of clipped Chebyshev neighbourhoods in a five-stone example … 53
Figure 5.1. Recorded native engine nodes on the centre fixture … 57
Figure 5.2. Native reference node counts, with ranking cost excluded from bars … 61
Figure 5.3. Historical native-versus-Node benchmark on one midgame fixture … 63

LIST OF TABLES

Table 2.1. Difficulty configurations and recorded search behaviour … 33
Table 3.1. Functional requirements F01–F09 … 36
Table 3.2. Verification scenarios and expected outcomes … 45
Table 5.1. Native engine search on the centre fixture at depth two … 56
Table 5.2. Native reference search on the clustered fixture at depth two … 60
Table 5.3. Historical native Rust and TypeScript/Node benchmark … 63
Table A.1. Repository inventory and roles … 74
Table B.1. Public evidence and remaining verification gaps … 77

ABBREVIATIONS AND READING CONVENTIONS

AI — Artificial Intelligence.

DSA — Data structures and algorithms.

DFS — Depth-first search.

DOM — Document Object Model.

PRNG — Pseudorandom number generator.

PvP — Two people playing on the same application.

PvE — A person playing against the computer in this report.

TT — Transposition table.

UI — User interface.

WASM — WebAssembly.

Ply — One stone-placement move by one side.

Fixture — A deliberately prepared input position for a test or measurement.

Coordinates — Frontend (row,col) equals Rust (y,x); indices run from 0 to 14.

Evidence — Published source, a recorded experiment and an author observation are identified separately.

CHAPTER 1. INTRODUCTION

1.1 Background and motivation

Caro is a two-player placement game in which a small set of rules produces a large adversarial decision space. A player places one stone on an empty cell, attempts to complete a winning line and must anticipate the opponent's reply. The board representation is simple enough to implement explicitly, while the number of possible move sequences makes exhaustive search impractical beyond shallow depths. This combination makes the project suitable for studying data structures, recursive search, evaluation functions, pruning and the relationship between algorithmic results and an interactive application.

The project develops both a playable web application and material for explaining its algorithms. In the application, React supplies the surrounding interface, Pixi renders the board and a controller coordinates accepted moves, match state and AI requests. The AI path combines a TypeScript tactical check with a Worker-based WebAssembly engine. The repository also preserves a TypeScript search implementation, native Rust benchmark programs, baseline Rust engine code, weekly research reports and a separate presentation website. These parts are related, but they are distinct execution paths with different evidence. This report explains how they fit together rather than treating every algorithm mentioned in a slide as a feature of the active engine. [1] [2] [3]

The practical problem is to return a legal and useful move while maintaining coherent application state. A search that produces a high score is insufficient if it changes the caller's board, uses an incompatible win rule, blocks interaction unnecessarily or commits a result after the user has reset the round. The report therefore considers both the mathematical search problem and the application boundary at which a result becomes a move. It also distinguishes an observed counter from a configured limit and a native experiment from a browser measurement.

1.2 Objectives and research questions

The first objective is to specify the actual game rules and state transitions. A 15 × 15 board and a five-stone target do not uniquely define a Caro variant. The application's Freestyle rule accepts a contiguous line of at least five stones, including longer lines. Its implementation is explained from the last-move scan, with the assumptions under

which that local test is valid. The second objective is to specify the AI algorithms precisely: their inputs, outputs, recurrence or decision policy, candidate membership, state mutation and restoration, stopping conditions and computational cost. [4]

The third objective is to connect those algorithms to the implementation. The report identifies which component owns the board, who may commit a move, how history is replayed into the Rust/WASM object and how stale responses are rejected. The fourth objective is to evaluate the published experiment records without turning limited measurements into universal claims. In particular, it asks whether Minimax and Alpha-Beta agree on the same selected tree, whether ordering reduces total work or only visited nodes, and whether a cap can discard an immediate tactical completion.

These objectives lead to four research questions. RQ1 asks how legal board state and the Freestyle win condition can be represented and checked deterministically. RQ2 asks how evaluation, candidate generation and adversarial recursion produce a move, and which assumptions preserve correctness. RQ3 asks how pruning, ordering and caching affect the measured search, including counterexamples and context hazards. RQ4 asks how the native and browser components preserve a consistent game when requests are asynchronous. The evaluation chapter answers each question using the appropriate source or artifact; it does not infer human playing strength from a search counter.

1.3 Scope, versions and source identity

The public project reference used throughout this English report is the immutable tducn110/basicproject revision 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc. The application is under CaroGame; the report also covers present, the weekly folders, benchmark and research. The local working project named 10_caro has a different repository layout and an origin named tducn110/caro. A pathname or a repository name alone therefore does not identify an implementation. Where local work is discussed, the report distinguishes it from the public source snapshot and uses the public mirror for claims that can actually be verified there. [1] [5]

The public tree contains Rust programs and baseline engine source as well as a compiled Rust WebAssembly package. It must not be described as a JavaScript-only project. At the same time, the complete modified engine used for a particular research run is not necessarily the archived baseline. The research provenance and

selected excerpts identify a measured modified implementation; the reference-library experiments have their own dependency and results. The report uses the words application, baseline, archived excerpt, legacy TypeScript and native reference to preserve these distinctions. [6] [7] [8]

The work excludes an online multiplayer server, a training pipeline, a learned neural policy and a demonstrated tournament ranking. These are not required to explain the implemented algorithms. Browser and host behaviour are discussed only to the extent supported by source and recorded observations. A rule recommendation, a future experiment and an implemented feature are labelled separately. This scope allows the report to be detailed without claiming capabilities absent from the project.

1.4 Method and report structure

The analysis follows the path from an input to its visible consequence: pointer event, cell coordinate, legal move check, board mutation, win or draw evaluation, next-turn decision, AI request, validated response, committed move and updated display. Source inspection establishes the mechanism. Published test and measurement records establish what was exercised on their particular corpus and environment. Primary literature explains general search concepts; it is not used as proof that the project implements a feature. All project citations in the reference list resolve to GitHub resources rather than machine-specific file paths.

Chapter 2 specifies the data structures and algorithms. Chapter 3 gives the requirements, ownership boundaries and state/data flows. Chapter 4 explains the implementation of the playable application and presentation project. Chapter 5 evaluates the published native, WASM, Worker, UI and benchmark records with explicit limits. Chapter 6 states the achieved outcomes and development directions. The appendices inventory the repository and explain how to inspect or reproduce the work without silently substituting a different engine.

CHAPTER 2. ALGORITHMS AND DATA STRUCTURES

2.1 Scope, evidence, and notation

This chapter explains how a Caro position can be represented, tested for a result, evaluated and searched. It connects the concepts to evidence from the project while distinguishing three implementation contexts. The current application controller and frontend rules can be inspected directly in the application source. An archived document contains excerpts of the modified Rust engine: those excerpts are source evidence for the functions shown, but its header states that the underlying files included local changes, not a committed engine revision. The author also inspected the locally available modified Rust engine; the archived excerpts record source hashes, but do not establish how the vendored binary was built [6]. The public repository also contains Rust benchmark, baseline and harness source as well as the compiled WebAssembly package. It does not follow that the complete modified Rust engine corresponding to the vendored binary can be independently rebuilt from the application source tree alone. The available baseline Rust files and the documented modified-engine excerpts must therefore not be treated as interchangeable [6], [7], [9].

The repository additionally contains a TypeScript bot implementation under the older bot module. It is called the legacy bot here because the current AIController constructs GomokuWasmEngine and sends the normal AI request through that adapter, with a separate tactical prepass for higher difficulties. Its quiescence, iterative deepening and transposition table are useful algorithms to study; their presence does not show that the current WASM engine uses them [10], [11]. A separate Rust reference engine has historical native research reports and test outputs. Results from that reference are evidence for the recorded reference experiment, not for the browser game, the vendored WASM binary or the current application [8].

The notation throughout uses an N × N board with N = 15 and winning length K = 5 unless stated otherwise. Coordinates (x,y) mean column and row, respectively, with both in 0…14. In frontend code the equivalent coordinate is (row,col) = (y,x). A ply is one placed stone. Let B denote a board, p the side to move, opp(p) the other side, and m a legal move. The transition T(B,p,m) places p at m; it is defined only if the cell is empty and the game permits a move. Let A(B,p) be the moves the selected search

policy actually generates, which may be a strict subset of the legal empty cells. A root player R is the player whose advantage is represented by a larger score. These distinctions matter because a proof about the generated search tree cannot establish a claim about every legal move if the generation policy has already excluded some moves.

The mathematical pseudocode is explanatory and names the relevant state explicitly. It is not claimed to be a line-for-line reproduction of any unavailable full source. All numerical examples are constructed to illustrate the definitions; they are not measurements of execution time or winning strength. In particular, a theoretical bound, a recorded local fixture, a browser observation and a production result require different evidence. Shannon's treatment of machine chess motivates look-ahead and evaluation of nonterminal positions; it does not validate the application code or establish its strength [12].

2.2 Board representation and legal updates

2.2.1 Matrix, index conversion, and state ownership

A position consists of 225 cells, each containing empty, X or O. In an array indexed as B[y][x], row-major conversion to a flat logical index is i = 15y + x; the inverse is x = i mod 15, y = floor(i/15). Hence the center (7,7) has index 112, and (14,14) has index 224. These are logical indices for an unpadded board. They are not the internal padded indices of the legacy TypeScript bot. Rust arrays are laid out in element order according to the language reference, but a statement about array layout alone does not establish the byte size or field arrangement of an entire board object on a particular target [13].

The archived Rust place excerpt validates bounds, rejects an empty stone, rejects an occupied cell, writes a cell and increments a move counter. The companion undo rejects an out-of-range or already-empty cell and otherwise clears it and decrements that counter [6]. Such local board checks do not by themselves establish that a position is reachable, that turns alternate correctly, or that play has stopped after a winner. Those conditions belong to the game and call-site contract. For example, an empty cell is geometrically available after a win, yet the game may no longer allow placing a stone there.

A board and a game are different state owners. Cells and the count describe the position; turn, winner, and move history describe the match's progression. A history

provides the information needed to restore actual game moves in reverse order. A recursive search temporarily changes a working board to explore alternatives, which must not silently become the player's actual move. The documented Rust search makes a copy at its outer entry, while its in-place search relies on restoration of the working board [6]. A separate copy prevents exploratory mutations from directly modifying the input; it does not prove that every internal bookkeeping field is correct.

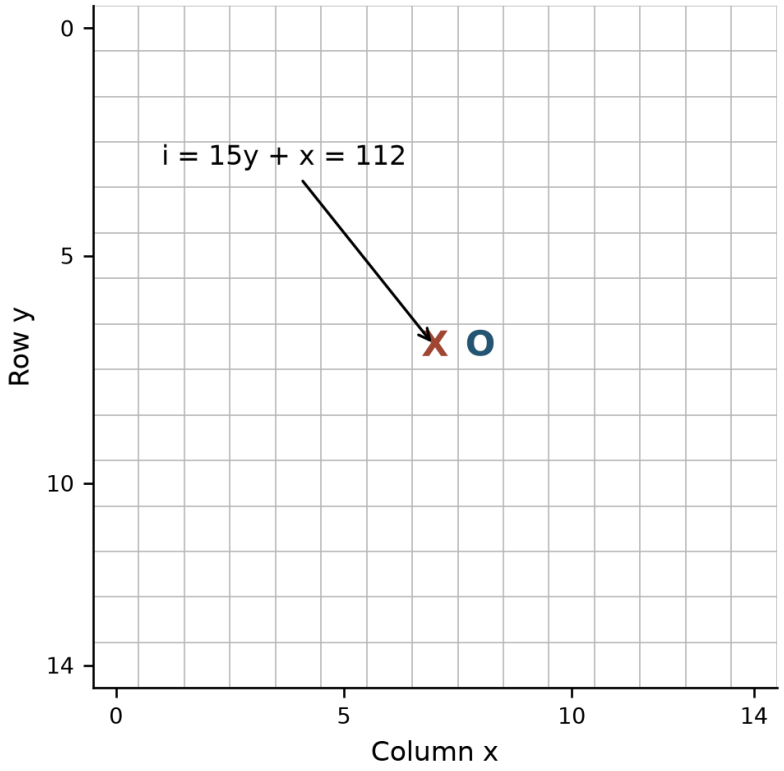


Figure 2.1. Row-major board representation

Author-drawn explanation from the cited source. Source: [14].

2.2.2 Apply and undo

Inputs and output. APPLY(B,m,p) takes the working board, a coordinate and a nonempty player; it returns success or failure. UNDO(B,m) takes the coordinate of a previously applied move and returns whether removal was possible. The stronger search precondition is that undo is paired with the most recent successful hypothetical move on that branch. An individual undo that merely removes an occupied cell does not enforce the whole history discipline.

```
APPLY(B, (x,y), p):
    if x or y is outside the board, or p is empty: return failure
    if B[y][x] is not empty: return failure
    B[y][x] ← p
    B.move_count ← B.move_count + 1
    return success

UNDO(B, (x,y)):
    if (x,y) is outside the board or B[y][x] is empty: return failure
    B[y][x] ← empty
    B.move_count ← B.move_count - 1
    return success

VISIT_CHILD(B, p, m):
    require APPLY(B, m, p) succeeds
    try: value ← VISIT(B, opp(p), m)
    finally: require UNDO(B, m) succeeds
    return value
```

The finally in this explanatory version means every normal return, cutoff and supported abort must pass through restoration. In languages without such a construct, equivalent structured cleanup is necessary. Additional state such as a hash, side to move, cached pattern counts, last move or winner must also be restored by its actual owner. Restoring the two fields shown in the board excerpt is not a proof that an unrelated implementation's extra fields were restored.

Invariant and argument. Immediately before exploring each sibling, the working state must equal the state at the parent node. A successful apply changes exactly the selected empty cell and increments the count. Its matching undo clears that same cell and decrements the count; therefore these fields equal their former values. By induction over nested calls, assuming each recursive child restores every state field it owns, the parent sees the same state for the next sibling. The induction depends on legal matching calls; it is not guaranteed if an exception bypasses undo or a different move is removed.

For example, let X occupy (7,7), O occupy (8,7), and the count be two. Trying X at (7,8) changes that cell and makes the count three. Undoing (7,8) returns both the cells and count to their initial values. Clearing the cell but leaving count three makes fullness checks unreliable; decrementing without clearing contaminates the next branch. With direct indexing, each operation is O(1) time and O(1) auxiliary space; copying an N × N board costs $O(N^2)$. Rejecting an occupied or out-of-bounds move must leave the board unchanged. A finite restoration test is useful evidence for the tested cases, but not a formal proof for every possible state.

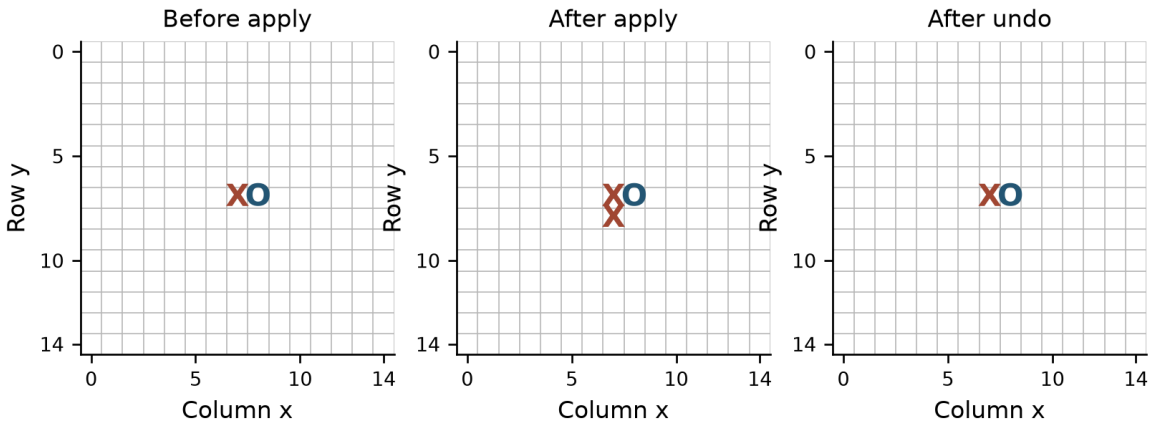


Figure 2.2. Search must restore its caller state

Author-drawn explanation from the cited source. Source: [6].

2.3 Freestyle win detection

The frontend win rule treats a contiguous line of at least five stones as a win, including an overline of six or more. It checks four axes through the last move, with both directions on each axis. It does not require exactly five or impose an additional two-blocked-ends condition [4]. The term Caro is used for the application, but this particular rule is specified explicitly because games with similar names may use other winning conditions.

Inputs, outputs and preconditions. The procedure takes a board and the last placed coordinate m, and returns the winner and a collection of winning cells when a qualifying line is found, or no result otherwise. It assumes m refers to the move that may have just caused the transition. If the board was already won before that move, testing only a line through m cannot detect every earlier win. A root-level validation of an arbitrary imported board needs a separate full-board check. The current frontend rule reads the player at m; an empty or invalid location produces no winner.

For directions D = {(1,0), (0,1), (1,1), (1,-1)}, let f_d and b_d count consecutive stones equal to the stone at m in the positive and negative direction. A win through the last move occurs exactly when some 1 + f_d + b_d ≥ K. To decide whether the threshold is reached, scanning at most K−1 matching cells in each direction is sufficient: any additional cells cannot change a negative result to a positive result after K−1 matching cells already exist on one side.

```
WIN_AT(B, m, K):
    if m is out of bounds or B[m] is empty: return NONE
    p ← B[m]
    for d in [(1,0),(0,1),(1,1),(1,-1)]:
        cells ← [m]
        for sign in [+1,-1]:
            for step from 1 to K-1:
                q ← m + sign * step * d
                if q is outside or B[q] ≠ p: break
                append q to cells
        if length(cells) ≥ K: return (p, cells)
    return NONE
```

The scan invariant is that every collected cell is on the selected axis, adjacent to the preceding segment and occupied by p. Stopping at the first boundary or different cell prevents joining disconnected sequences. If a contiguous run through m has length at least K, either its collected positive and negative portions together reach K−1 neighbors or a single portion does, so the check succeeds. Conversely, K collected stones form a contiguous same-player segment through m, establishing the stated rule. The returned display collection may be truncated to K−1 stones per side; it is sufficient to prove a win but is not necessarily the full maximal overline.

Consider X X [X] X X on one row with the bracketed center as the last move. Each side contributes two; 1+2+2=5. For X X X [X] X X, the value exceeds five and remains a win. An empty last cell returns no winner; a gap interrupts the run; edge and corner scans stop safely. At K = 5, there are at most 4 × 2 × 4 = 32 neighboring-cell inspections plus the origin, so the per-move rule is O(1) for fixed K and O(K) when K varies. The frontend's returned cell list takes O(K) space. A full-board validation has a different cost and must not be confused with this incremental check.

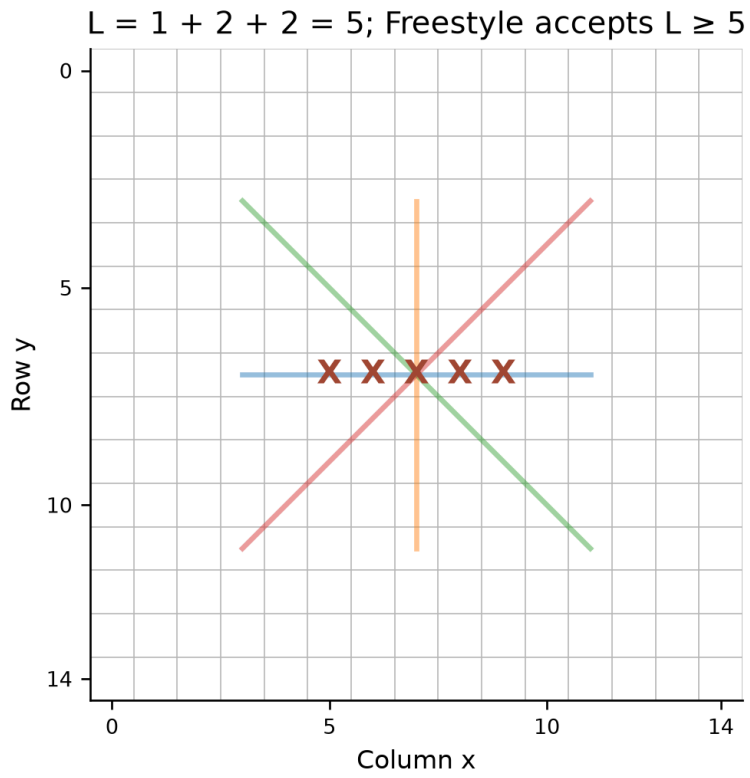


Figure 2.3. Last-move scan along four axes

Author-drawn explanation from the cited source. Source: [4].

2.4 Baselines: random play and rule-based play

2.4.1 Random baseline

A random baseline makes the meaning of added search explicit: it selects without comparing a tree of future replies. The documented Rust baseline first checks for a terminal position or an invalid empty player; for Easy it gathers legal empty cells, uses a seeded pseudo-random value modulo the number of cells, and evaluates the resulting move for the returned result. That evaluation occurs after selection and does not choose the Easy move [6].

```
RANDOM_MOVE(B, p, seed):
    if p is empty or B is terminal: return NONE
    L ← all legal empty cells in B
    if L is empty: return NONE
    i ← SEEDED_INTEGER(seed) mod length(L)
    return L[i]
```

The input includes the seed because pseudo-random generation is deterministic for a particular generator state and input. The output is either an element of L or no move. With a board containing only one stone, |L|=224; a one-element list always returns its sole entry. A full or terminal board must not perform modulo zero. The legality argument is immediate from construction of L, provided the game-level preconditions

and list are correct. Modulo reduction does not necessarily give a perfectly uniform distribution when the generator's integer range is not divisible by |L|; no such claim is required to explain the baseline.

Gathering all cells takes $O(N^2)$ time and $O(N^2)$ space in the worst case; selection after that is O(1). A post-selection evaluation has its own cost. A fixed seed makes the selection reproducible under the same generator and ordering, not necessarily across different implementations or changed inputs. A random policy does not promise to notice an immediate win or threat.

2.4.2 Immediate tactics and one-ply heuristic

The documented Normal baseline first obtains candidates, then chooses an immediate win when available, otherwise an immediate blocking coordinate, and otherwise compares the evaluations of candidate moves. These are priorities, not a multi-ply adversarial proof. The candidate assessment is the source of the immediate-win and block markers [6]. The current frontend also has a separate tactical resolver for Hard, Very Hard, Expert and Master before it requests the WASM engine. Easy and Normal skip that frontend prepass. A tactical response reports its own source and zero Rust search work; it must not be presented as a completed alpha-beta search [10].

```
RULE_BASED(B, p, policy):
    if B is terminal or p is invalid: return NONE
    C ← SELECT_CANDIDATES(B, p, policy)
    if an m in C wins immediately for p: return m
    if an m in C is an immediate winning cell for opp(p): return m
    best_move ← NONE; best_score ← -infinity
    for m in C:
        APPLY(B,m,p)
        try:
            score ← TERMINAL_OR_EVALUATE(B,p,m)
        finally:
            UNDO(B,m)
        if best_move is NONE or score > best_score:
            best_move ← m; best_score ← score
    return best_move, best_score
```

The decision invariant after processing the first j non-tactical candidates is that best_score is the greatest observed value among those j moves and the board is restored. A separate invariant places the win and block checks before the one-ply loop. Thus the routine picks the best score among the evaluated candidates when neither tactical condition applies. Correctness of the tactical marker itself depends on the assessment recognizing the corresponding winning geometry and on the move being included in C.

For a concrete threat, suppose X occupies row 0, columns 0–3, column 4 is empty, and O is to move. The left endpoint lies outside the board; placing O at (4,0) blocks the only immediate completion along this row. If the same four consecutive X stones are in the interior with both endpoints empty, there are two distinct winning cells. A single O move can occupy one cell, not both. Retaining both choices avoids losing information but does not establish that either saves the game. Similarly, an immediate winning move for O takes priority over blocking if both are available under the algorithm's ordering.

If k candidates are generated, one-ply evaluation costs roughly O(k × evaluation-cost), in addition to candidate generation and ordering. A constant-time local win scan does not make the whole rule-based procedure constant time. A nonterminal board with no generated move requires a defined fallback or no-move response; terminal handling must precede any selection. A short tactical prepass is not a substitute for search through arbitrary combinations.

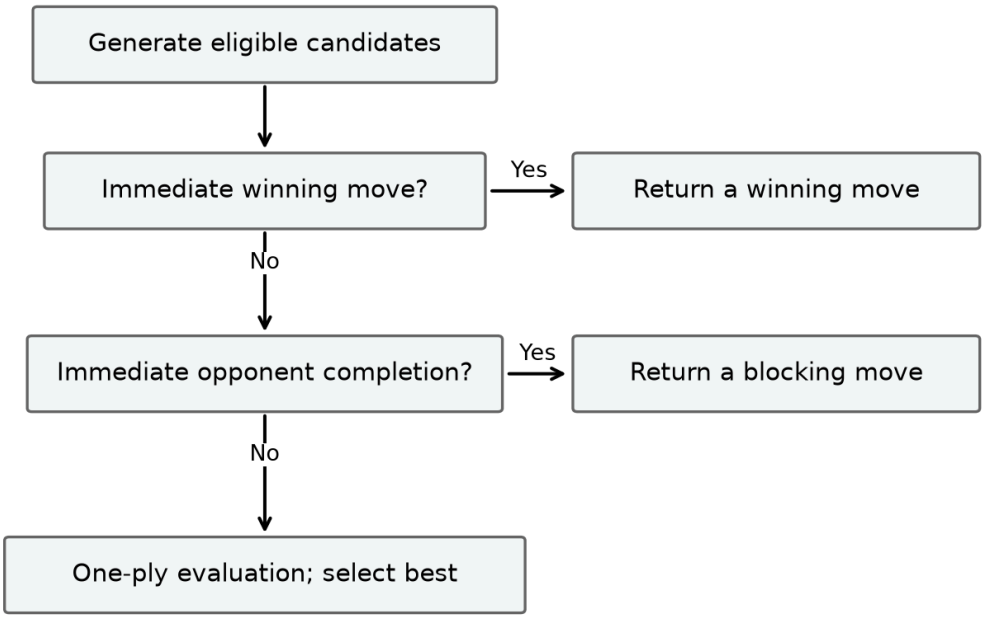


Figure 2.4. Rule-based tactical priorities

Author-drawn explanation from the cited source. Source: [7].

2.5 Static evaluation and terminal utility

2.5.1 Evaluation of nonterminal positions

A search with finite depth can stop before a winner exists. Its evaluator assigns an approximate score to that horizon position. In the documented modified Rust snapshot, evaluate scans maximal contiguous runs in four directions, adds the root player's run weights, subtracts the opponent's, and adds or subtracts a center preference. The snapshot counts a run once per axis by skipping a cell when the previous cell on the same axis has the same color. It then clamps the final heuristic to a range below terminal scores [6].

Let R_p(B) be maximal runs of p over the four axes, len(r) their length and open(r) their number of empty endpoints. Let c(x,y)=max(0,14−|x−7|−|y−7|) and C(B,p) be p's sum of c minus the opponent's. The descriptive formula is

$$\mathrm{E_raw}(B,p) = \Sigma[r \in R_p(B)]\ w(\mathrm{len}(r),\mathrm{open}(r)) - \Sigma[r \in R_opp(B)]\ w(\mathrm{len}(r),\mathrm{open}(r)) + C(B,p).$$

The publicly archived Rust baseline defines the following weights for two open ends and one open end, respectively: length at least five, 1,000,000 and 1,000,000; length four, 50,000 and 5,000; length three, 5,000 and 500; and length two, 200 and 50. A run of five or more has the same weight even when both ends are closed; otherwise unmatched cases receive zero from the run-weight function [15]. The separately inspected modified working source has the same weight table; the baseline alone does not establish identical behavior of the vendored binary. These numbers are units of the evaluator, not percentages, probabilities or measured milliseconds.

```
EVALUATE(B,p):
    score ← 0
    for each occupied cell s:
        for d in the four axes:
            if preceding cell along d has the same color: continue
            (length, open_ends) ← scan maximal same-color run
            value ← WEIGHT(length, open_ends)
            if B[s] is p: score ← score + value
            else: score ← score - value
        score ← score + center contribution signed by color
    return CLAMP(score, -H, H)
```

The loop invariant is that score equals the signed contribution of all starts already visited, plus center terms for occupied cells already visited. Every maximal run has exactly one start in each orientation, so the preceding-cell test avoids counting that run repeatedly. This supports equivalence to the stated sum, assuming the run scanner and table implement the stated definitions. It does not prove that the resulting score matches the strategic value of a position.

For an illustrative board with X alone at (7,7), its singleton has no run weight and its center term is 14, so E(B,X)=14. Add X at (8,7) while leaving the remainder empty: the horizontal length-two run has two open ends and contributes 200; center terms are 14 and 13, so the value is 227. This board is an isolated evaluation example and need not represent a legal alternating game history. A blocked endpoint changes the run category and may also add an opponent contribution. Edge cells are not open endpoints.

Scanning all maximal runs across a constant number of directions is $O(N^2)$: although a run is traversed, each occupied cell participates in at most one maximal run per direction. Auxiliary space beyond the board is O(1) in the described scan. The evaluator recognizes contiguous runs; a gap such as XX_XX is not automatically a single contiguous run, even though a separate move test can recognize the winning gap. Forks, broken formations and future threats are not exhaustively classified by this table. More search or a different evaluation function changes the model, not a mathematical certainty about eventual victory.

Illustrative run	Archived Rust weight
.XXXXX.	1,000,000
.XXXX.	50,000
0XXXX.	5,000
.XXX.	5,000
0XXX.	500
.XX.	200
0XX.	50

Figure 2.5. Contiguous-run weights of the archived Rust evaluator

Author-drawn explanation from the cited source. Source: [7].

2.5.2 Terminal states and the score hierarchy

Terminal outcomes must be evaluated before a depth cutoff. The author inspected the modified working source and found M=100,000,000, with q the ply distance from the root capped at 225, and H=M−225−1=99,999,774. The public excerpt reproduces

the terminal formula and records a hash for the working source, but does not include the full constant definitions; the values are therefore a working-source observation, not independently established by that excerpt alone [6]. The result from root perspective is M−q for a root win, −M+q for a root loss, and zero for a draw. An unfinished board has no terminal utility and is evaluated only when the search horizon is reached [6].

```
TERMINAL_UTILITY(winner, full, root, ply):
    q ← min(ply, 225)
    if winner = root: return M - q
    if winner is the opponent: return -M + q
    if full: return 0
    return NONE
```

The clamp ensures every nonterminal heuristic lies between −H and H. Since M−225 = 99,999,775 > H, even the slowest represented win exceeds a nonterminal score; similarly a loss is below every nonterminal score. A win at q=2 receives 99,999,998 and a win at q=4 receives 99,999,996, so maximizing prefers the earlier win. Conversely, if a loss is unavoidable, increasing q makes its negative score less severe and thus delays the loss. This preference follows from the formula; it is conditional on the search actually reaching and correctly identifying those terminal outcomes. The winner must be decided before a full-board draw when the last empty cell completes a line.

2.6 Candidate generation and a soft tactical cap

2.6.1 Spatial neighborhood

Considering every empty cell creates a large branching factor. A candidate policy can instead choose empty cells close to existing stones. For cells a and b, Chebyshev distance is $d_\infty(a,b)=\max(|a_x-b_x|, |a_y-b_y|)$. With radius r, a spatial candidate is an empty cell within distance r of at least one occupied cell. The empty board has a special center choice, and a full board has no move. The exact selection, fallback and ranking below describe the archived modified-engine policy, not the legacy bot's different forcing-move generator [6].

```
RAW_CANDIDATES(B,r):
    if B is full: return []
    if B has no stones: return [center]
    seen ← N × N false array; C ← []
    for each occupied cell s in B:
        for each q in the radius-r square around s clipped to the board:
            if B[q] is empty and not seen[q]:
                seen[q] ← true
                append q to C
    return C
```

For a stone at (7,7), radius one gives a 3 × 3 square with eight empty candidates; radius two gives a 5 × 5 square with 24. With the single stone at (0,0), the radius-two square is clipped to a 3 × 3 corner and yields eight candidates. Multiple stones may nominate the same cell, but seen prevents duplicates. If a radius produces no raw candidates on a nonfull board, the documented selection code falls back to legal moves, avoiding an occupied placeholder. An empty player and a full board return no candidates [6].

The membership invariant says that each appended coordinate is on the board, empty and not previously appended. Conversely, every empty cell in a visited neighborhood is appended on its first encounter. This proves correctness for the defined neighborhood, not equivalence to all legal moves. With m occupied cells, scanning the board and their neighborhoods costs $O(N^2 + m(2r+1)^2)$ before assessment; the seen matrix and candidate list use $O(N^2)$ space. The domain can be clipped at the board edge. Comparing 24 candidates with 224 empty cells is a geometric example of a smaller root move set, not a measured reduction in search nodes or time across games.

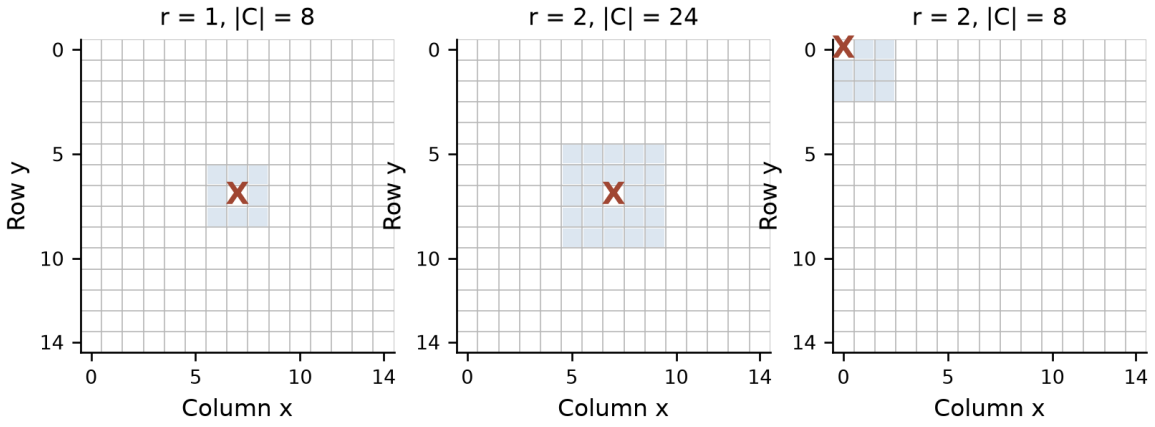


Figure 2.6. Radius and board edges affect candidate membership

Author-drawn explanation from the cited source. Source: [6].

2.6.2 Tactical preservation and membership

For each raw move, the documented select_candidates assesses whether it wins for the moving side, blocks an immediate opponent win, and assigns a quick ranking score. Sorting prioritizes a win, then a block, then descending score, with row-major coordinate used to break ties. If a cap is configured, the ranked list is truncated at max(cap,1,tactical_count), so every assessed immediate winning or blocking coordinate remains even if the number of such coordinates exceeds the cap. After selecting membership, disabling move ordering sorts the retained set by coordinate rather than changing its members [6].

```
SELECT_CANDIDATES(B,p,policy):
    if p is empty or B is full: return empty set
    raw ← RAW_CANDIDATES(B, policy.radius)
    if raw is empty and B is not full: raw ← LEGAL_MOVES(B)
    ranked ← ASSESS each move for win, block, and quick score
    sort ranked by win first, block second, score descending, coordinate
    tactical_count ← count of entries marked win OR block
    if policy.cap exists:
        retain first max(policy.cap, 1, tactical_count) entries
    if ordering is disabled: sort retained entries by coordinate
    return retained entries and assessment metadata
```

Let C be the raw set, T its marked tactical subset, and c the configured cap normalized to at least one. The selected set is the union of T and enough highest-ranked non-tactical members to reach c when possible. If |T|=2 and c=1, both tactical moves remain, so this is a soft cap, not an absolute maximum. The proof depends on tactical entries being ordered before every non-tactical entry and on every relevant move first being in raw. It does not show that all useful multi-ply tactical moves are preserved, or that a move excluded by the spatial policy is later recovered by the cap.

For k raw candidates, assessment takes O(kK) when the relevant directional scan is bounded by K, sorting O(k log k), and storage O(k), in addition to spatial generation. With fixed K=5 the scan term is linear in k. Candidate assessment and full-board evaluation are different operations; a counter named ordering evaluations should be interpreted using its own recorded definition. A historical reference experiment demonstrates the danger of a different, hard cap: in a particular fixture, cap six omitted both winning endpoints while the uncapped radius-one set contained 20 candidates. That recorded counterexample concerns the separate reference policy, not evidence that the modified snapshot's soft tactical cap has the same defect [16].

2.7 Move ordering versus move selection

A search traverses the retained moves in an order. Ordering permutes an already chosen set; filtering or capping changes its membership and therefore the tree itself. In a complete minimax evaluation of the same deterministic tree, any permutation produces the same value, although a tied best move can differ if the first best move is retained. In alpha-beta search, ordering influences how soon useful bounds are discovered and can alter the number of cutoffs, while preserving the root value under the algorithm's assumptions [17].

```
ORDER(retained_moves, rank):
    calculate a rank key for each retained move
    sort by the chosen key with a deterministic tie breaker
    return the same moves in the new order
```

For example, if A, B and C have quick ranking scores 10, 50 and 30, score order is B, C, A, while coordinate order may be A, B, C. Neither order establishes that B is ultimately the best move. If all are still visited under minimax, only traversal changes. Alpha-beta may finish earlier after B supplies a strong bound. Sorting k moves costs O(k log k), plus the cost of computing keys and O(k) storage in a conventional array implementation. A reduction in visited recursive nodes does not by itself establish a reduction in total time, since assessment and sorting have costs.

The documented candidate selection uses fast local information for ranking; the historical reference experiment uses its own evaluation and ordering behavior [6], [8]. Comparisons should hold the board, depth, scoring, candidate membership and terminal rules fixed, changing only ordering. Equal values but different tied moves can both be consistent. A comparison that changes both cap and ordering cannot isolate ordering's effect.

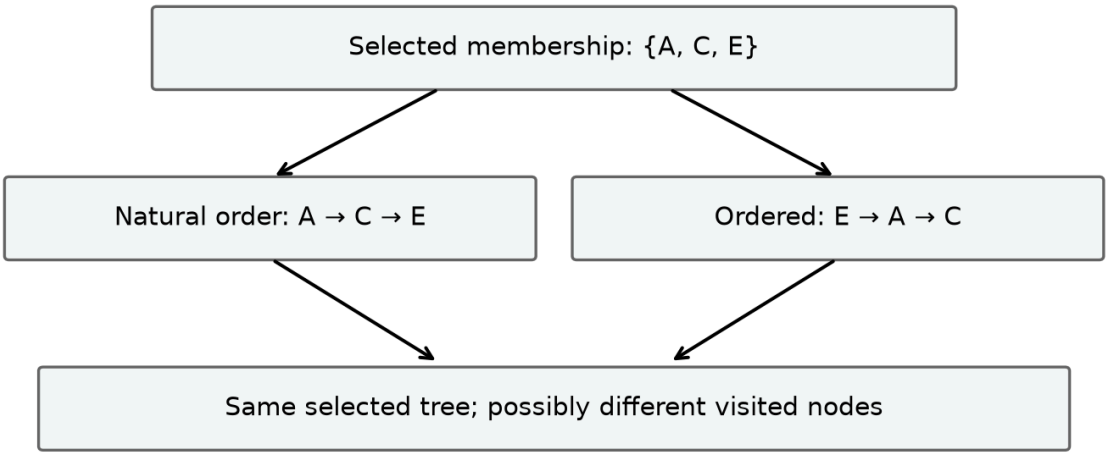


Figure 2.7. Alternative traversal orders on identical membership

Author-drawn explanation from the cited source. Source: [6].

2.8 Minimax search

2.8.1 Recurrence and contract

Minimax assumes two alternating decision-makers. MAX chooses a move to maximize a score from the root player's perspective; MIN chooses to minimize it. Its input is the current board, the side to move, a nonnegative depth d in plies, the last move when available, a candidate policy and an evaluator. The output is a value and, at the root, a selected move, or no move for an already finished game. State includes the mutable working board, remaining depth, root perspective and recursion stack. Search must begin from a board compatible with the chosen rules, or validate it separately.

Define U(s,R) for terminal states, E(s,R) for a nonterminal horizon, and transition T(s,a). With a fixed deterministic candidate function A, the depth-limited value is

V(s,d) = U(s,R) if s is terminal;

V(s,0) = E(s,R) if s is not terminal;

V(s,d) = max[a∈A(s)] V(T(s,a),d−1) if MAX moves;

V(s,d) = min[a∈A(s)] V(T(s,a),d−1) if MIN moves.

An empty generated set for a nonterminal board must have a specified behavior. The documented value routine returns zero for an empty list, while candidate fallback is intended to avoid this on a nonfull valid position. That fallback convention should not

be silently reinterpreted as a win or as a comprehensive validation of arbitrary positions [6].

```
MINIMAX(B, player, root, depth, last, ply):
    t ← TERMINAL_UTILITY(WIN_AT(B,last), IS_FULL(B), root, ply)
    if t exists: return t
    if depth = 0: return EVALUATE(B,root)
    moves ← SELECT_CANDIDATES(B,player)
    if moves is empty: return 0   // documented convention for this case
    best ← -infinity if player = root else +infinity
    for m in moves:
        require APPLY(B,m,player)
        try: v ← MINIMAX(B,opp(player),root,depth-1,m,ply+1)
        finally: require UNDO(B,m)
        if player = root: best ← max(best,v)
        else: best ← min(best,v)
    return best
```

At the root, test each allowed root move, remember an action that achieves the greatest value, and restore the board after every branch. Initial root validation may need to scan for a pre-existing winner, whereas recursive checks can use the just-played move under the earlier precondition. The archived code shows a shared recursive value function configurable as minimax or alpha-beta; it should be described as the snapshot's documented behavior and not as a verified complete current source revision [6].

2.8.2 Proof, example and complexity

Use induction on d. For terminal nodes or d=0 the procedure returns exactly the corresponding definition. Assume recursive results equal V for depth d−1 and that each apply/undo restores the parent. At a MAX node the loop's best value after j children is the maximum of the first j child values; at completion it is the maximum over A. The MIN case is symmetric. Each recursive call reduces d, so a finite candidate list guarantees termination. This proves evaluation of the specified finite tree if transition, terminal detection, evaluation and candidate generation satisfy their contracts; it does not prove the evaluator predicts the outcome of an unrestricted game.

In an abstract depth-two tree, MAX has actions A and B; MIN's leaves under A have values 3 and 5, and under B have values 2 and 9. MIN returns 3 for A and 2 for B; MAX chooses A with 3. These leaf values are invented to explain the recurrence. A depth-one search on the same state could yield another move because it evaluates different horizon positions.

If the maximum number of generated moves is b at each level, the full tree has O(b^d) nodes for b>1; actual branching may vary by state, cap and terminal events. Depth-first exploration uses O(d) stack frames plus the live candidate lists, conservatively O(db) move entries and a working board of $O(N^2)$, not storage for the whole tree. Each node also incurs candidate-generation or evaluation costs, which can be substantial. If terminal is reached sooner, recursion stops early. A search that filters away a legal move computes minimax only over its remaining tree, and no optimal-play claim for the full game follows.

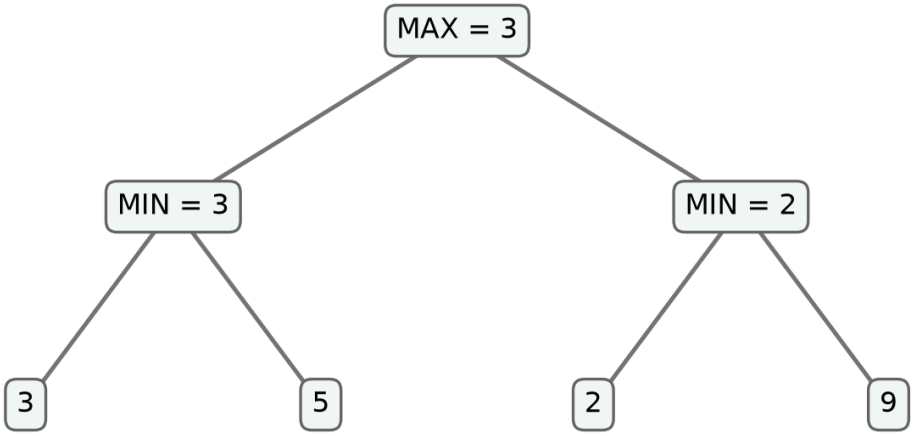


Figure 2.8. A two-level Minimax example

Author-drawn explanation from the cited source. Source: [12].

2.9 Alpha-beta pruning

2.9.1 Bounds and pseudocode

Alpha-beta pruning computes the same root value as minimax on the same finite deterministic tree under a consistent utility and candidate policy, while sometimes skipping subtrees. alpha is the best lower bound MAX can already secure in the current ancestor context; beta is the best upper bound MIN can already secure. At a MAX node, update alpha after observing a child; at a MIN node, update beta. When alpha ≥ beta, the remaining siblings cannot improve the relevant ancestor's decision. Knuth and Moore analyze alpha-beta and the effects of move ordering under stated assumptions [17].

```
ALPHABETA(B, player, root, depth, last, ply, alpha, beta):
    t ← TERMINAL_UTILITY(WIN_AT(B,last), IS_FULL(B), root, ply)
    if t exists: return t
    if depth = 0: return EVALUATE(B,root)
    moves ← SELECT_CANDIDATES(B,player)
    if moves is empty: return 0
    best ← -infinity if player = root else +infinity
    for m in moves:
        require APPLY(B,m,player)
        try: v ← ALPHABETA(B,opp(player),root,depth-1,m,ply+1,alpha,beta)
        finally: require UNDO(B,m)
        if player = root:
            best ← max(best,v); alpha ← max(alpha,best)
        else:
            best ← min(best,v); beta ← min(beta,best)
        if alpha ≥ beta: break
    return best
```

Here restoration occurs before a cutoff is tested. The caller starts with an unbounded window when it needs a root result. A value returned from an internally cut-off node may represent a bound rather than an exact independent value. This distinction becomes essential when such results are cached. The code excerpt increments a cutoff counter at a break; that event count is not the total number of theoretical descendant nodes omitted [6].

2.9.2 Worked cutoff and correctness boundary

Use the same abstract tree. Search A first: its MIN node sees 3 and 5, returns 3, and MAX updates alpha to 3. At B, MIN first sees 2, so beta becomes 2. Because beta ≤ alpha, B cannot supply MAX a value exceeding the already available 3: MIN can choose the 2 branch. The remaining B child, whose illustration value is 9, is unnecessary for choosing the root action. The root answer remains A, 3. Visiting the children in another order may produce fewer or no cutoffs without changing the full-tree result.

The invariant is that alpha and beta summarize bounds supplied by already considered alternatives and ancestors. Once they cross, further siblings at the current node cannot alter the ancestor choice under the minimax ordering. Combined with the minimax induction and the restoration invariant, pruning preserves the root value on the same tree. A practical check compares minimax and alpha-beta with identical positions, turns, depth, terminal rule, evaluator, candidate set and preferably deterministic tie behavior. If the set of candidate moves differs, the comparison tests two different games rather than pruning alone. A corpus of passing cases is bounded empirical evidence, not proof for every state.

Worst-case O(b^d) remains possible, for instance with unhelpful ordering. Idealized ordering in a uniform tree can reduce the effective leaf growth toward O(b^(d/2)); this is a model, not a promise of a particular elapsed time. The recursion stack and live candidate lists have the same conservative O(db) bound as depth-first minimax, in addition to the board. Cutoff at equality is valid for the value, though tie selection and visited-node counts can depend on ordering. The algorithm alone does not address deadline, collision safety, illegal input or selective candidate loss.

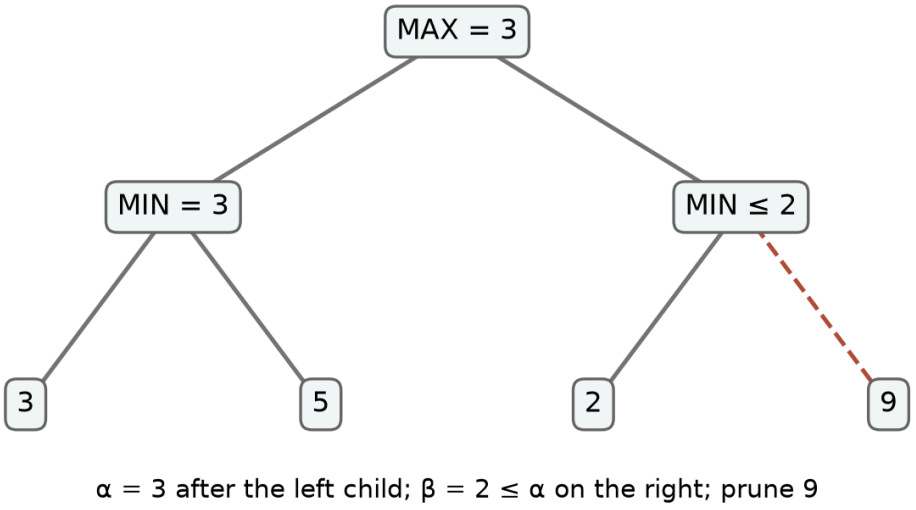


Figure 2.9. Alpha-Beta cutoff on an unchanged tree

Author-drawn explanation from the cited source. Source: [17].

2.10 Iterative deepening and budgets

Iterative deepening performs complete searches at depths 1, 2, … until reaching a configured maximum or a budget interrupts an iteration. Its input is a root position, a maximum depth and a budget. Its output should distinguish the best result from the most recently completed iteration, the completed depth, and the reason for stopping. The working board, search counters, and any table or ordering hints must obey the respective implementation's lifecycle. An incomplete iteration's partially explored best move is not the same as a completed minimax result for that depth.

```
ITERATIVE(B, maximum_depth, budget):
    last_complete ← NONE
    for depth from 1 to maximum_depth:
        if budget is exhausted: break
        result ← SEARCH_AT_DEPTH(B, depth, budget)
        if result was interrupted: break
        last_complete ← result
    return last_complete, completed depth, stop reason
```

The invariant after iteration d is that last_complete corresponds to a whole completed search at d under that iteration's search contract. It follows inductively because the result is replaced only after the iteration returns normally. A separate abort-safe restoration guarantee is required: interruption in a child must unwind all applied moves. If depth one returns A, depth two returns B and depth three aborts, the completed result is B at depth two. If no iteration completes, a caller must explicitly handle the absence of a completed result; a fallback candidate is useful but should be labeled as such.

Repeated shallow searches consume work, although the deepest full iteration can dominate for a sufficiently large, regular branching factor. Each iteration has the complexity of its selected search; total time is the sum over attempted depths plus ordering and bookkeeping. No hard real-time guarantee follows from this scheme. A search interrupted only at periodic checks can exceed its nominal wall-clock deadline before the next check, especially if individual operations are expensive.

The legacy TypeScript findBestMove uses performance.now(), a deadline and a periodic node check, and catches its search-abort sentinel. It initializes a fallback from the first candidate before any depth is completed; its completed-depth field remains zero if the first iteration aborts. Its recursive search uses try/finally around move recursion to undo on abort, but this does not prove that every possible exceptional failure in all unrelated helpers is safe [11]. The historical Rust reference instead used a node budget, and its report states that its aggregate omits work inside an aborted iteration. Those semantics should not be combined into one universal time-budget claim [8]. The inspected local game engine specifies fixed depth for its tiers; archived runs separately record each configured depth [18]. The available evidence does not establish that its Master tier uses this legacy or reference iterative-deepening implementation.

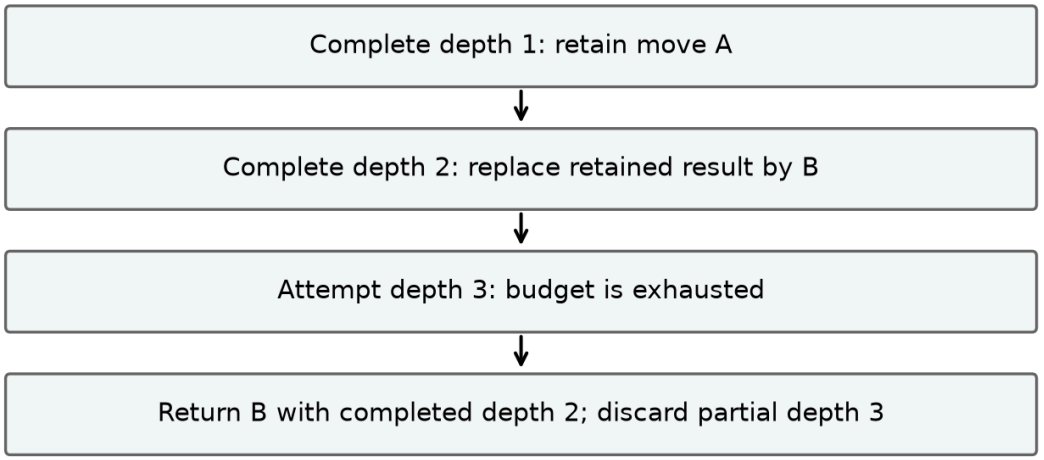


Figure 2.10. Iterative deepening and incomplete work

Author-drawn explanation from the cited source. Source: [8].

2.11 Zobrist hashing and incremental state

2.11.1 Definition and update

A transposition can arise when different sequences of moves lead to the same board and relevant side to move. Comparing all cells at each cache access costs $O(N^2)$, so a compact hash can serve as an index. Zobrist's method assigns a pseudorandom word to each (cell, player) pair and combines the keys for occupied cells with exclusive OR (XOR) [19]. One also needs a side-to-move key if the same occupancy can be evaluated with different players to move. The hash is a compact fingerprint, not mathematical proof that two states are identical.

Define Z[x,y,p] as a fixed random word for a stone, and S as a fixed side key. Under one convention,

$$H(B,p) = \text{XOR } Z[x,y,\text{color}] \text{ over occupied cells, XOR } S \text{ if } p \text{ is the designated second side.}$$

All keys must remain fixed for the lifetime of the table. Applying a move of p at m adds its key with XOR. If the transition changes the side, XOR S as well. Undo repeats the same XOR operations in reverse. The essential identity is (h XOR z) XOR z = h.

```
FULL_HASH(B, player):
    h ← 0
    for each occupied cell m with stone color c: h ← h XOR Z[m,c]
    if player is designated second side: h ← h XOR S
    return h

UPDATE_HASH(h, m, p, side_changed):
    h ← h XOR Z[m,p]
    if side_changed: h ← h XOR S
    return h
```

For an illustrative four-bit example, let h=1010 and the inserted stone's key be 0110, ignoring side for the moment. Apply gives 1100; undoing XOR with 0110 restores 1010. These are demonstration values, not the project keys. If side changes, use the side key on both forward and reverse transitions. If a terminal transition preserves the side to move, toggling S unconditionally would be incorrect for the full-hash definition.

The hash invariant is incremental_hash = FULL_HASH(current board, current side). It holds initially if computed from the board; XORing the stone's key exactly accounts for adding that feature, and XORing the side key exactly accounts for a side change. Repeating those operations removes the features on undo. This proof assumes the same keys, actual transition semantics, and no omitted state features relevant to the hash contract. It does not eliminate collisions.

The legacy TypeScript board maintains two 32-bit halves, a seeded key table and a side key, and updates them with make and undo. Its pattern counts and stone count are also incrementally updated; restoring only the hash would not restore the whole board state [20]. Historical reference reports document incremental hash checks and note that a terminal winning move may leave to_move unchanged in that separate implementation, so its side-key update must follow the actual state transition [8]. This distinction cannot be transferred automatically between implementations.

A full recomputation scans $O(N^2)$ cells; one incremental XOR update uses $O(1)$ time. The key table has $O(N^2)$ entries for a fixed number of players; the board and any pattern caches have their own space costs. A finite hash collision remains possible even when incremental and recomputed hashes agree on every tested position. A table may verify a full key or full state, or otherwise accept a defined collision risk. A player key alone also does not encode rules, search depth, evaluator, root perspective or a candidate policy.

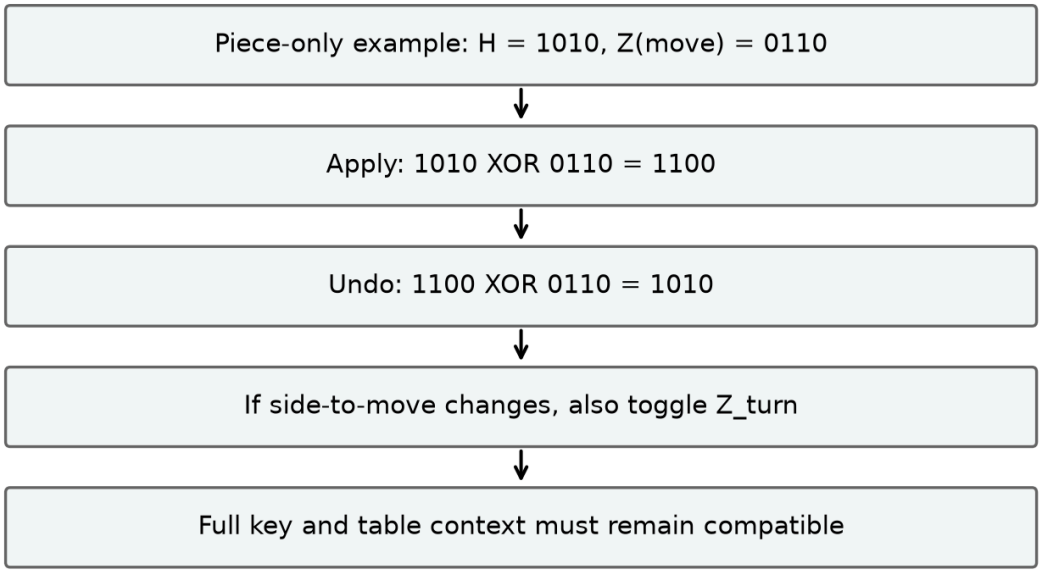


Figure 2.11. Incremental XOR and turn-key handling

Author-drawn explanation from the cited source. Source: [20].

2.12 Transposition table: entries, bounds and context

2.12.1 Meaning of stored values

A transposition table (TT) maps a position fingerprint to a previously computed search result. A typical entry stores a hash or verification key, remaining search depth, value, bound type and a suggested best move. The move may help order later search even if the stored score is unusable. A cache lookup is sound only when its represented search problem is compatible with the current one, its depth is sufficient under the selected contract, and its bound justifies the requested cutoff or exact result.

For an original search window $[\alpha_0,\beta_0]$, a common classification is: result $v \le \alpha_0$ is an upper bound; $v \ge \beta_0$ is a lower bound; and $\alpha_0 < v < \beta_0$ is exact, subject to the search's assumptions. Equality is treated conservatively at the boundary. For instance, with window [3,8], a returned 2 is an upper bound, 5 is exact and 9 is a lower bound. These values illustrate a window; they are not recorded project measurements. A lower bound is usable for a beta cutoff when it is at least β, and an upper bound for an alpha cutoff when it is at most α.

```
PROBE(key, required_depth, alpha, beta, context):
    entry ← table.lookup(key)
    if entry missing or key/context is incompatible: return MISS
    hint ← entry.best_move if it is legal in this position else NONE
    if entry.depth < required_depth: return HINT_ONLY(hint)
    if entry.bound = EXACT: return VALUE(entry.score, hint)
    if entry.bound = LOWER and entry.score ≥ beta:
        return VALUE(entry.score, hint)
    if entry.bound = UPPER and entry.score ≤ alpha:
        return VALUE(entry.score, hint)
    return HINT_ONLY(hint)

STORE(key, depth, score, alpha_original, beta_original, best_move):
    if score ≤ alpha_original: bound ← UPPER
    else if score ≥ beta_original: bound ← LOWER
    else: bound ← EXACT
    store key, depth, score, bound, best_move according to replacement policy
```

The pseudo-probe explicitly tests context and hint legality as an engineering contract. It is a safe conceptual design, not a claim that every project implementation includes all such checks. The classification uses the original search window; using a window altered by a prior lookup without carefully defining semantics can mislabel a bound as exact. Cache storage and retrieval must agree about whether a returned cutoff value is fail-hard or fail-soft.

2.12.2 Why reuse can be wrong

A useful invariant is: whenever a TT score is reused, it is valid for the current position, depth interpretation, perspective, rule set, evaluation and move-generation context, and its bound permits the intended conclusion. An exact compatible entry may answer a query; a lower or upper bound is usable only in its permitted direction. If any premise is false, the stored number can change the decision. A full hash comparison narrows ordinary index collisions but still does not guarantee that different game states cannot share a fingerprint.

The historical reference report documents a concrete context hazard. It records one same-root warm reuse, then a probe after playing a move and changing root player: a fresh table returned score 0, whereas reuse of the prior table returned 19,800. Its hash omitted several contextual factors, including root perspective, and the report does not claim a patch. These are results of a particular native reference probe, not a measurement or defect proven in the current browser engine [21]. The report also notes that mate scores depend on ply from the root; a cached value reused at another ply may require normalization or a sufficiently narrow reuse scope. Rules, evaluation weights, side, candidate radius/cap, ordering semantics, terminal

metadata and requested depth can likewise matter. A board fingerprint by itself does not certify those conditions.

The legacy TypeScript uses parallel fixed-size arrays indexed by a modulo operation on one hash half and compares both halves before using a slot. It stores depth, score, flag and best move. This is a different storage and replacement design from the reference's hash map [11], [8]. The existence of a depth check does not resolve every context hazard; in particular, project-wide claims require inspecting lifecycle, perspective and score conventions. A cached move hint should be checked against current legal candidates before application. A new game, changed rules, new evaluator or changed root is a reason to reassess or clear a cache unless compatibility is proved.

Expected lookup can be O(1) for an appropriate hash table, but collisions, table implementation and replacement behavior affect actual costs. Space is proportional to the allocated slots or stored entries plus keys, buckets and overhead. Initial hash-map capacity is not a strict maximum, and the native size of one entry is not the total memory of the container. A TT can reduce repeated work on suitable workloads, but a finite test does not prove collision freedom, global optimality or a universal speedup.

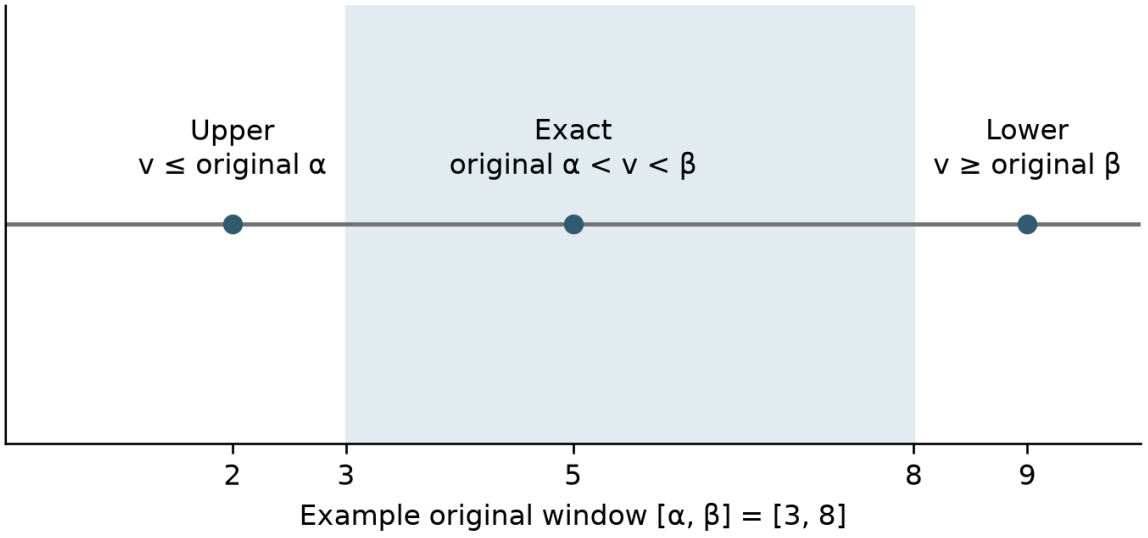


Figure 2.12. Bound semantics in a transposition table

Author-drawn explanation from the cited source. Source: [8].

2.13 Quiescence search in the legacy TypeScript bot

A fixed-depth search can stop at an unstable position: a static evaluator may prefer a move just before an immediate tactical reply. Quiescence extends selected tactical

continuations beyond the ordinary horizon so that the evaluator is applied after a limited forcing sequence. This is a selective model and has its own horizon; it cannot eliminate every tactical error. The relevant implementation is the legacy TypeScript bot, not a feature established for the current WASM engine [11].

Inputs and state. The legacy routine receives the board, alpha and beta bounds, current ply and a quiescence depth. It works from the perspective of the side to move, uses negamax sign reversal, increments a node counter and checks for abort. Its state includes board cells, side, hash and cached pattern counts. Its output is a bounded score for the selected forcing continuation, or an abort propagated to the caller. It first checks whether either side has a five pattern. It then calculates standPat, applies a beta cutoff if that static score is at least beta, raises alpha if appropriate, and stops at quiescence depth five. Otherwise it generates forcing candidates, orders them and recurses after a move with (-beta,-alpha); each recursive branch uses try/finally to undo [11].

```
QUIESCENCE(B, alpha, beta, ply, qDepth):
    count node; check budget or deadline
    if previous player has a five: return -MATE + ply
    if side to move has a five: return MATE - ply
    standPat ← EVALUATE_FROM_SIDE_TO_MOVE(B)
    if standPat ≥ beta: return beta
    alpha ← max(alpha, standPat)
    if qDepth ≥ 5: return alpha
    moves ← GENERATE_FORCING_CANDIDATES(B)
    order moves by their quick score
    for m in moves:
        MAKE(B,m)
        try: v ← -QUIESCENCE(B,-beta,-alpha,ply+1,qDepth+1)
        finally: UNMAKE(B,m)
        if v ≥ beta: return beta
        alpha ← max(alpha,v)
    return alpha
```

The sign change reflects the negamax convention for an evaluation whose perspective changes with the side to move. The procedure returns beta on a fail-high and alpha otherwise, so its return can be a bound rather than an exact unrestricted game value. Its loop invariant is that alpha is at least the stand-pat score and all examined forcing continuations, subject to the window and fail-hard convention; the board must equal the incoming state before every sibling. The depth limit guarantees termination even when forcing candidates continue to exist. It also creates a new horizon.

For a simplified numeric example, suppose the window is [−10,10], stand-pat is 2, and the first searched forcing move returns 6 after sign reversal. Alpha rises to 6. If

another move returns 12, the routine returns beta, 10, as a fail-high bound. This does not assert that 10 is the exact minimax value. In a real position, the forcing generator's definition and threat patterns decide which moves are included. The legacy code considers outcomes such as five, open four and four for either side when identifying forcing moves. These categories arise from its own bitmask pattern evaluator, which differs from the snapshot's contiguous-run evaluator [11], [20].

The stand-pat assumption deserves care in a game where a player must move: static evaluation does not literally grant a legal pass. In a forced-defense position an immediate static cutoff may not establish the same value as examining all legal replies. The finite depth, selective forcing set and candidate implementation further limit the result. A rigorous claim requires a specified tactical contract and tests that compare against an appropriate full or deeper reference on representative positions. With at most f forcing moves at each quiescence level and maximum extra depth q, a coarse upper bound is O(f^q) nodes plus candidate generation, ordering and evaluation at each node; actual cost may differ sharply. The stack and stored move lists take O(qf) space under the same conventional assumption, in addition to board and caches.

2.14 Legacy bitmask patterns and incremental line counts

The older TypeScript bot represents a line segment available to one player as a bitmask: a one denotes the player's stone and zero denotes an empty cell; an opposing stone divides the line into separate segments. It precomputes a classification for eligible lengths and masks using completion tests. Categories include a five, an open four, a four, an open three and a broken three. The terminology is the implementation's category scheme; it must not be silently equated to the maximal contiguous runs and weights of the documented Rust snapshot [20], [11].

```
CLASSIFY_SEGMENT(mask, length):
    if mask contains five consecutive 1 bits: return FIVE
    wins ← number of zero positions which become a five when set
    if wins ≥ 2: return OPEN_FOUR
    if wins = 1: return FOUR
    threats ← number of zero positions which, when set,
              leave at least two immediate winning positions
    if threats ≥ 2: return OPEN_THREE
    if threats = 1: return BROKEN_THREE
    return NONE

COUNT_LINE(B, p, start, step, length):
    split the line wherever an opponent stone occurs
    encode p and empty cells of each segment as a mask
    for each segment of length at least 5:
        increment the bucket returned by CLASSIFY_SEGMENT
    return counts
```

For a hypothetical five-cell segment XX_XX, setting the middle zero creates five consecutive bits, so the segment has at least one immediate winning completion under this rule. A segment with two distinct empty completion cells is classified above a one-completion segment, assuming no five is already present. The precise category depends on the whole segment and the code's precedence, not only on a local substring. Opponent cells split the segment; a board boundary is also a boundary. Short segments cannot contain five and do not contribute a five-capable pattern.

The board's incremental update subtracts old counts for the four lines crossing a changed cell, makes the move, then adds counts for the new versions of those lines. Undo performs the inverse around clearing the cell. The invariant is that accumulated per-player counts equal a fresh sum of classifications for the board, assuming the untouched lines are unchanged and the four affected lines are correctly identified. Each changed cell lies on exactly one line of each axis, so only those lines need recomputation. This is a proof sketch for the update scheme, not a proof that the classifier expresses every strategically meaningful formation.

The legacy position evaluator shown in the repository scans lines and sums category weights from the selected perspective; cached counts are also used by other routines. A full evaluation over a constant number of axes is $O(N^2)$ for fixed-time table lookup per segment; recalculating four affected lines is $O(N)$ in board width per player, in addition to the cost of table lookup. Precomputing a pattern table trades initialization work and memory for later lookup; its precise limits, size and supported board lengths are implementation constraints. The actual code initializes for line lengths up to 15, so changing board size without reviewing this bound is unsafe. A

cached count is only trustworthy when every mutation follows the update discipline and every aborted sequence restores it.

2.15 Difficulty configurations and the active request path

The application exposes six AI difficulties. An archived experiment records Easy as Random and Normal as Rule-based and the higher four tiers as alpha-beta, with configured depth and soft cap shown below [18]. The modified working Rust source was inspected as a cross-check; the radius values are working-source observations and are not columns in that CSV. The archived excerpts also show the strategy dispatch [6]. Neither the experiment record nor the working-source inspection alone guarantees that the vendored binary was rebuilt from those exact files; adapter and binary provenance require separate verification [9].

Table 2.1. Difficulty configurations and recorded search behaviour

Difficulty	Recorded strategy	Depth	Radius / soft cap	Interpretation
Easy	Random	0	All legal empty cells	Selects without look-ahead; seed controls pseudo-random sequence.
Normal	Rule-based	1	Neighborhood, no fixed cap in default policy	Immediate win, block, then one-ply heuristic.
Hard	Alpha-beta	2	2 / 15	May be handled first by the frontend tactical resolver.
Very Hard	Alpha-beta	3	2 / 12	The user-facing tier name is not itself an API numbering rule.
Expert	Alpha-beta	4	2 / 10	The cap is soft for marked immediate tactics.
Master	Alpha-beta	5	2 / 8	Fixed-depth snapshot configuration; no inference of legacy TT or iterative deepening.

The radius entries come from the author’s working-source inspection; the recorded CSV directly supplies strategy, configured depth and cap. Depth is measured in plies including the root move under the documented search convention. Increasing depth while decreasing the candidate cap changes two parameters at once. Therefore the table alone cannot establish that the last tier is stronger in all positions or quantify a strength difference. A controlled evaluation would need a specified collection of positions or games, equal conditions, alternated colors where appropriate, seeds, scoring and uncertainty. A completed-depth field describes an API result; a

configured depth or an uninstrumented zero node count is not automatically a measurement of how much actual engine work occurred.

The current frontend controller's input is an application state, round identity and difficulty. It increments a request identifier. For Easy and Normal it proceeds to the WASM engine adapter. For the four higher tiers it first asks the TypeScript tactical resolver for a forced move for O; if one is returned, it creates a tactical result without requesting engine search. Otherwise it copies move history into a request to the engine. The frontend prepass and the Normal Rust rule-based policy are different decision stages [10].

The archived adapter excerpt shows a new game instance being populated from the request's move history, rejecting an unsuccessful replay, asking for an AI move, freeing the game in finally, converting Rust (x,y) into frontend (row,col), and attaching request/round identity and statistics. Its elapsed scope is named worker-hydrate-and-search, which includes history reconstruction and the search call, rather than being an isolated inner-loop benchmark [6]. The public current application adapter should be checked independently against this historical excerpt when documenting a specific revision. A request identifier and round identifier support detecting outdated work only when the receiving controller actually validates them; their existence alone does not prove every stale-result race is handled.

2.16 Synthesis and limits of the algorithmic claims

The algorithms in this chapter have different responsibilities. Win detection decides the stated terminal rule; board mutation maintains a position; random and rule-based choices provide baselines; the evaluator assigns an approximate horizon value; candidate generation restricts the moves explored; ordering permutes the retained moves; minimax evaluates the resulting tree; alpha-beta skips provably irrelevant branches of that same tree; iterative deepening manages progressive search; and hashing plus a transposition table aim to reuse compatible earlier work. Quiescence extends a selected class of tactical moves at a horizon, but its own assumptions and depth limit remain.

A proof about one layer cannot be substituted for a proof about another. For example, alpha-beta/minimax equivalence does not show that the candidate policy retains every decisive move. Correct XOR undo does not prove that a cache entry was computed using the same root perspective. An accurate win scan does not

validate an arbitrary imported history. A worker-based adapter does not, by itself, establish responsiveness or correctness of cancellation. Likewise, a compiled binary, a baseline Rust file, an archived modified-source excerpt and a native reference report are different evidence objects, each with its own provenance.

The recorded research tests and experiments are useful because they specify fixtures, configurations and counters, including negative cases and restoration checks. They remain finite. The historical reference timing procedure records sequential repetitions on a local native environment and does not establish browser latency, general device performance or statistical significance [8]. For a reproducible claim about a particular application build, the report must identify the source revision or snapshot, binary provenance, rule and evaluation configuration, player and position, candidate set, depth, platform, and meaning of each reported counter. The following chapters can then describe the system integration and evidence without turning theory into an unmeasured product outcome.

CHAPTER 3. SYSTEM ANALYSIS AND DESIGN

3.1 Actors, use cases and functional requirements

The primary actor is a person using the web application. The person can play against another person on the same board or select a computer opponent. The AI service is a system component rather than an independent user: it receives a snapshot and returns a proposed move. A hosting integration may supply lifecycle information, but it does not own the board or bypass the controller's move checks. This actor model keeps responsibility for accepted state transitions inside the application. [2] [22]

Table 3.1. Functional requirements F01–F09

Requirement	Trigger and input	Required behaviour and observable result
F01 Select mode and difficulty	A person selects local two-player or AI mode, or an available AI difficulty.	Invalidate the previous AI turn, reset board and search feedback, establish the selected mode and setting in a ready match, and carry the selected difficulty in subsequent AI requests.
F02 Validate and place a stone	A pointer gesture resolves to a board coordinate or another attempted move occurs.	Accept one legal current-player move to an in-bounds empty cell, update history and evaluate the outcome. Reject occupied or out-of-bounds moves, paused or ended games, a locked AI turn and wrong-player moves without changing the board.
F03 Determine the round outcome	An accepted move completes a Freestyle winning line or fills the board without a win.	Check the four axes, record and display a win or draw, and prevent further moves in the ended round.
F04 Maintain history, undo and reset	A move is accepted, or a person invokes Undo, Replay or New Game.	Maintain the move history. Undo one ply in two-player mode or the human move and a completed AI reply where applicable, restoring the correct turn and phase. Replay and New Game reset board and feedback, invalidate the old AI request and establish the phase intended by each action.
F05 Request and validate an AI move	An accepted nonterminal human move changes the turn to the computer in AI mode.	Request one proposed move, lock board input while the computer is thinking, and commit only a still-current legal result; invalidate responses belonging to an earlier round or request.
F06 Show AI feedback and retry	An AI request is pending, succeeds, or fails.	Display thinking state and, when supplied, strategy, source, counters, elapsed scope or an error. Permit retry when provided by the application; do not invent Rust statistics for a route that did not run Rust.
F07 Render and resize the board	Board state changes or the available drawing region changes size.	Render the board and stones from the current state and update the board view for the changed drawing region.

Requirement	Trigger and input	Required behaviour and observable result
F08 Host integration and standalone operation	A host supplies locale or changes pause state, or the game is opened standalone.	Apply supported host locale and pause signals when integrated, remain usable in standalone mode, pause the implemented game-owned input and rendering activities and invalidate stale AI work, and resume a needed AI turn from current state.
F09 Inspect presentation material	A reader uses the separate presentation site.	Provide explanatory material, figures and the Python examples available in that site, keeping instructional demonstrations separate from engine benchmarks.

The requirements are derived from controller actions and their guards, not merely from the presence of buttons. For example, replay and new game both clear stones, but they have different phase semantics. A reset-like visual action must be traced to the method it calls before its behaviour is described. Similarly, removing search feedback after Undo is part of keeping displayed evidence associated with the current board. [2] [23]

Completion and verification criteria. The following criteria define the expected result for each functional group; they are acceptance targets, not a claim that every scenario has already passed. The detailed evidence and its environment are given in Chapter 5.

F01: Switching between two-player and AI modes or choosing a difficulty starts the intended match configuration and invalidates an earlier AI request; the selected tier is carried into subsequent requests. Verify each selection and the board/turn state before and after the change. F02: A valid move has integer row and column in 0–14, targets an empty cell and is made by the permitted player in an active, unlocked round. Accept it once and change one cell; reject occupied, out-of-range, fractional, paused, ended, wrong-turn and AI-locked inputs without mutating the board or history. The current core bounds helper checks range but not integrality, so rejection of direct fractional coordinates is a known open criterion; the pointer and WASM adapter checks do not establish that every direct caller is protected.

F03: In each of the four axes, test a five-stone line and an overline of six or more, including a boundary example; both must win under Freestyle. A full board without a winner must draw, while a win on the last cell must take precedence over a draw. F04: Verify the documented Undo behavior for two-player and AI play, restoring cells, history, turn and phase; Replay/New Game must reset the intended state and invalidate old work. F05: During an AI turn, lock input and accept only a legal reply

for the active request and round; test late replies after Undo, Reset, mode change and cancellation. F06: Show pending, result and error/retry states as applicable; attribute strategy, counters and elapsed scope to the route that produced them, leaving unavailable measurements absent.

F07: On a state update or viewport resize, render the current board and keep coordinate mapping correct. F08: Check locale and pause/resume behavior in an actual supported host and standalone mode, including stale-response handling and a needed AI restart; do not infer host acceptance from unit tests. F09: Open the presentation figures and Python examples independently, and label demonstrations as teaching material rather than live-engine performance results.

For algorithm verification, compare Minimax and Alpha-Beta on identical positions, rules, root perspective, depth, evaluation and candidate membership; equal root scores are required, with tied moves allowed to differ. The archived native corpus reports 408 comparison pairs including repeated configurations, which is finite evidence, not 408 independent boards or a proof of all positions. Separately verify the actual vendored WebAssembly and adapter with identified binary and fixtures; native search results alone do not prove browser/WASM behavior or binary provenance. Recorded local checks and headless browser observations establish only their stated environments. Physical-device and real-host testing, unresolved fractional input, and human acceptance remain pending until separately recorded.

3.2 Non-functional requirements and acceptance boundaries

Correctness means preserving legal state transitions, the agreed rule and the relation between board cells and history. Maintainability means keeping rule evaluation, rendering, input conversion and asynchronous search behind identifiable interfaces. Responsiveness motivates the Worker boundary for substantial search, while correct cancellation requires more than moving work off the main thread. Reproducibility requires a source revision, binary identity, configuration and measurement procedure rather than a screenshot containing a duration.

The design does not promise a fixed frame rate, a maximum response time on every device or an optimal move at every difficulty. Such promises require representative measurements and a definition of acceptable behaviour. A finite suite may verify a particular invariant on selected inputs; physical touch, host integration and deployment acceptance require their own environments. These boundaries guide the

evaluation in Chapter 5 and prevent an application build from being presented as a complete acceptance decision.

3.3 Architecture and ownership

The system has five principal responsibilities. React owns interface composition and subscribes to a controller snapshot. GameController owns the game-facing action boundary, current-player choice, AI-turn lifecycle and notification. The core owns board cells, legal-move history and rule evaluation. Pixi owns the visual board and converts gestures through an input/coordinate layer. The AI adapter owns search dispatch and Worker resources; it proposes a move but does not directly mutate the authoritative application board. [22] [2] [24] [14]

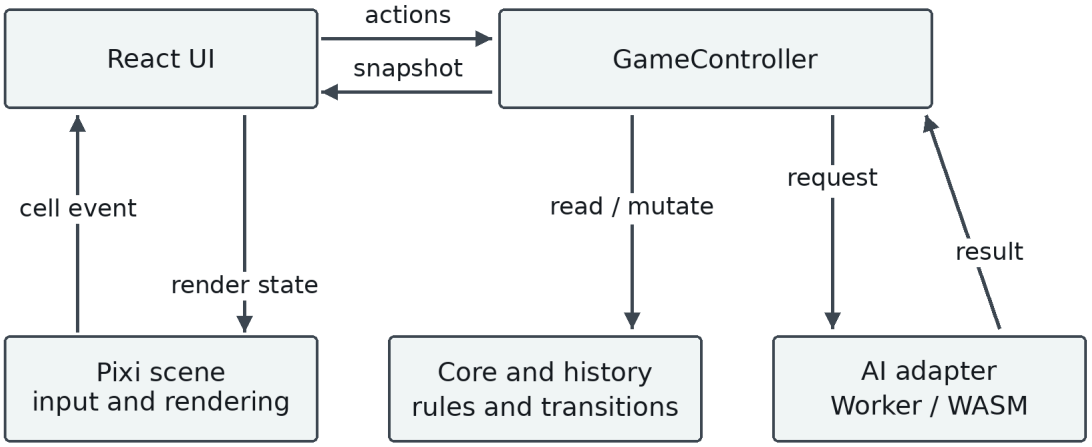


Figure 3.1. Responsibilities in the playable application

Author-drawn explanation from the cited source. Source: [2].

This ownership arrangement gives a clear dependency direction. The scene reports a cell action to the application/controller. The controller changes domain state and publishes a snapshot. The scene receives a list of stones, last move and winning cells for rendering. A visual object is not a second authoritative board. The AI path receives copied history and returns a result through the controller, which repeats the relevant validity checks before committing it.

A snapshot contains the match record, current player, thinking and pause flags, board stones, history, last move, last AI result and error. The controller copies move records and their cell coordinates when creating it. Subscribers read a stable cached

snapshot until a notification creates a new one. This matters for React subscriptions: returning a newly constructed object on every read could produce unnecessary updates or unstable identity, whereas mutating a previously delivered object could conceal a change. The observed implementation uses an explicit cached value and listener set. [2]

Web Workers provide a standard way to execute scripts independently from interface scripts, but that standard does not automatically establish this project's lifecycle safety. Here the Worker wrapper is responsible for transport and pending promises, while the game controller is responsible for deciding whether a result remains valid for the current round. These are complementary responsibilities. Terminating a worker can stop obsolete work; checking the current token prevents an already-resolved obsolete response from being committed. [25] [26]

3.4 Board state, history and legal transition contract

The frontend GameState stores one value per cell and an ordered move history. A move records its player, cell and index; history is also the input used to reconstruct a fresh Rust game for a search. These two uses require consistency. If a board cell changed without a corresponding history update, the display could show one position while the engine searched another. The transition contract therefore relates the occupied-cell set to the history sequence rather than treating them as independent data. [14]

The board has N = 15 and N^2 = 225 cells. A frontend cell is (row, col); Rust entry points use (x,y) with x = col and y = row. The row-major linear index is row × N + col. The normal pointer route applies floor, producing integer cell coordinates, and checks that they lie in 0 through 14. The WASM result adapter separately validates both integer-ness and bounds before mapping a result to a frontend cell. The core isBoardCell function itself checks numeric bounds but does not test Number.isInteger; direct fractional inputs therefore require an additional guard. A public API should not rely exclusively on the renderer to supply integers, because tests or other callers can enter through other routes. [27]

The playMove behaviour rejects an occupied cell before mutation, records the accepted move and evaluates the supplied rule. The higher controller supplies the current player and enforces pause, phase and AI-turn guards. This division is significant: a low-level board operation may not know which UI phase is active, while

the controller should not duplicate the geometric win scan. The report therefore describes the legal move as a composed contract rather than attributing every guard to one function. [28] [2]

The history sequence is not merely a move counter. It preserves alternation and the exact order needed for hydration. A full board without a winner is declared a draw after the win test. That order matters on the last empty cell: a winning final placement must remain a win rather than being classified as a draw merely because the board is now full. The controller's finishMove uses this ordering. [2]

3.5 Match phases and state transitions

MatchController defines setup, ready, playing and game-over. Its initial state is ready, local two-player mode, normal difficulty and round identifier zero. Calling playCell from setup or ready invokes startRound, which increments the round identifier, clears the old outcome and enters playing. An accepted win or full-board draw changes the match to game-over. The thinking flag belongs to the AI request lifecycle; it is not a fifth value in the phase enumeration. [23] [2]

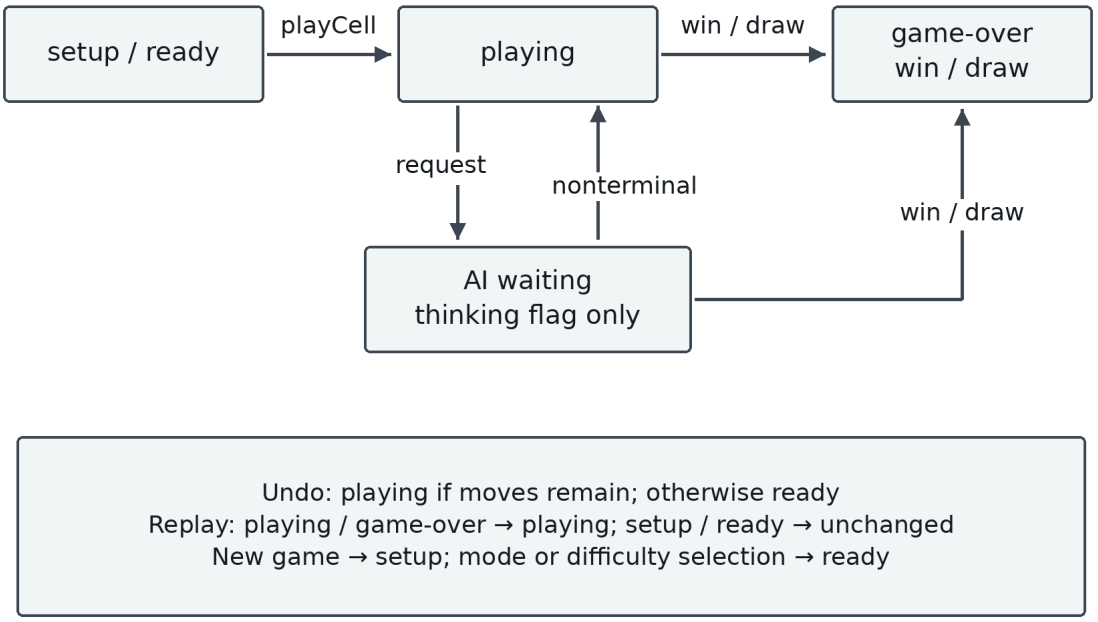


Figure 3.2. Match phases and the separate AI waiting flag

Author-drawn explanation from the cited source. Source: [23].

New game increments the round identifier and enters setup. Selecting a mode or difficulty first clears old game work and then places the match in ready. Replay may enter playing directly when the previous phase is playing or game-over. Undo calls

resumeAfterUndo, increases the round identifier and chooses playing if stones remain, otherwise ready. These distinctions explain why a state diagram must name actual events instead of using a single unspecified Reset arrow.

Pause is orthogonal to the match phase. A paused playing round remains a playing round whose input and current AI work are suspended or invalidated. On resume, the controller checks whether it is still an AI match, still playing and still O's turn before requesting again. If pause were represented as a new board with an implicit player reset, the resumed game could diverge from its history. The source keeps the board and phase while handling the lifecycle separately. [2]

A state-machine invariant can now be stated. A result may commit only when its request belongs to the active AI turn and round, the application is unpaused, the phase is playing, the mode is AI and O is the current player. None of these predicates is replaced by the fact that a promise resolved successfully. Success describes the completed computation; the predicates describe whether that computation is still applicable.

3.6 Asynchronous request protocol and cancellation

An AI action captures a new turn token, the round identifier and difficulty before setting thinking state. The request contains a copied move history. The transport wrapper adds its own message identifier so the correct pending promise receives the corresponding Worker response. The returned domain result carries its request/round identity, difficulty, move and statistics. The message identifier, AI request identifier and round identifier serve related but different scopes; they should not be collapsed into a single undocumented counter. [2] [3] [26]

On completion, the controller compares the captured token with its current token, checks pause and the returned round identifier, and repeats the phase, mode and current-player conditions. Only then does it clear thinking state, save feedback and pass the proposed O move through the same domain move behaviour used elsewhere. A rejected returned move produces an error instead of being painted directly. A missing move in an active round is also an error. This keeps the authoritative board inside the domain path. [2]

A concrete race illustrates the mechanism. Suppose request A is searching round 8. The person invokes New game before A finishes. The controller increments its token, clears state and changes the round. Even if A's response was already queued, its

captured token no longer matches. The promise resolution cannot insert A's move into the new board. Meanwhile, cancelPending rejects pending transport promises and terminates the old Worker, so future requests construct a new one. The token guard and resource cancellation address different parts of the race. [2] [26]

The rejection path is also scoped. A caught error changes feedback only when the captured token still equals the current token. Thus cancellation of an old request should not overwrite the error display of a newer turn. The Worker wrapper ignores messages from a worker object that is no longer its current worker, and ignores message identifiers absent from its pending map. These guards are source mechanisms; whether all possible races are covered remains a test question.

3.7 Coordinate transformation and gesture interpretation

The scene works with a camera translation (cx,cy), a positive zoom z and cell size S = 56. For a screen position expressed in the scene's coordinate system, the inverse transform is worldX = (screenX − cx)/z and worldY = (screenY − cy)/z; the selected column and row are floor(worldX/S) and floor(worldY/S). Conversion back to a cell's world origin multiplies its column and row by S. A centre point for drawing a stone is obtained by adding half a cell in the renderer, rather than changing the indexing contract. [29]

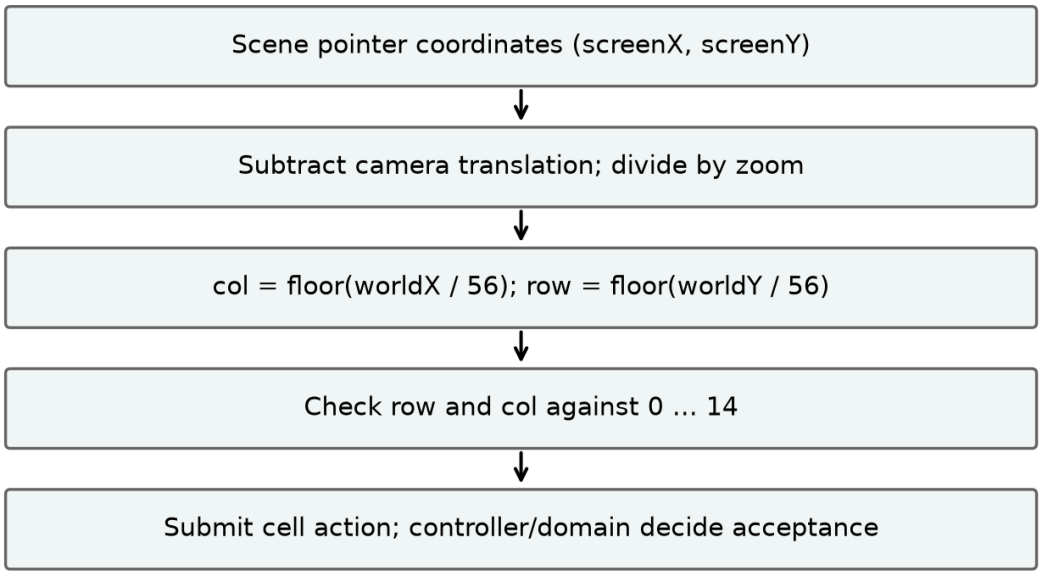


Figure 3.3. Inverse camera transform before board bounds checking

Author-drawn explanation from the cited source. Source: [29].

The floor operation is important near boundaries. A negative world coordinate must remain out of bounds; rounding it toward a nearby valid cell could turn a margin tap into a move. For example, with camera x = 20, zoom = 0.5 and screen x = 19, the world x is −2 and the column is −1. The subsequent bounds check rejects it. With screen x = 230, world x is 420 and the column is 7. The same calculation on y yields the row. The report uses scene screen coordinates; client-page coordinates must first be converted by the event/rendering system that supplies them.

The input controller has idle, pressed and dragging states and a configured drag threshold of six screen units. A press stores the last pointer. Each move computes displacement from the last stored pointer; a sufficiently large displacement while pressed changes the state to dragging, and dragging produces a pan delta. On release, a valid cell tap is emitted only if the gesture remained pressed. The last-pointer update means the implemented threshold is checked against the last event displacement, not a guaranteed accumulated distance from the initial press. This detail is preserved here so the explanation does not promise a different gesture recognizer from the source. [30]

The scene treats pointer-up outside through the same handler and suppresses event processing while paused. It fits the board to the viewport using the minimum of available width and height ratios, with different padding for a narrow viewport. The camera's translation and scale are then applied to the world container. This separation lets the same rule and history logic work under different viewport sizes, although a desktop viewport simulation does not establish physical-touch behaviour. [24]

3.8 Search-result contract and presentation of telemetry

The Rust result validator expects integer x/y coordinates in bounds, a finite score, the requested difficulty, one of the known strategy names and nonnegative safe-integer counters. Those counters include visited nodes, leaf evaluations, cutoffs, ordering evaluations, generated candidates and completed depth. An older binary returning only a coordinate pair fails this contract. This is preferable to silently displaying a nominal depth beside a result whose engine work was never instrumented. [27] [9]

The adapter maps the fields into application terminology and explicitly labels the source as Rust. Its elapsed timer starts before construction and history hydration, includes ai_move and free, and ends after cleanup. It excludes module loading and

message delivery. A displayed duration therefore has a defined scope: it is not a complete user-perceived round trip and is not a measurement of only the recursive search. The tactical route instead labels its source as tactical, reports zero Rust search counters and a null score. Zero counters on that route mean Rust search was bypassed, not that a nontrivial Rust tree required no work. [3] [27]

Completed depth is the depth reported for the finished computation. A configuration table is a requested policy, while completed depth is output telemetry. Neither alone proves that a higher depth is supported or that every node at that depth was examined under a full-legal move set. Candidate pruning, terminal leaves and the chosen strategy all affect the actual search. The interface should preserve these distinctions when showing details to a player or evaluator.

3.9 Verification scenarios derived from the design

Table 3.2. Verification scenarios and expected outcomes

Scenario	Input/event sequence	Required observation
S01 Legal placement	Tap an empty in-bounds cell while the current turn is available	One history item and stone added, correct outcome and next player.
S02 Rejected placement	Tap an occupied or out-of-bounds cell	No board/history mutation.
S03 Last-cell win	Place the final empty stone so it completes a winning line	Win is recorded before a possible draw condition.
S04 Delayed AI response	Request AI, change the round, then deliver the old response	Old result cannot commit a move or overwrite current feedback.
S05 Undo before reply	Human move followed by Undo while a request is pending	Request invalidated and board/turn restored consistently.
S06 Undo after reply	Completed human-plus-AI pair followed by Undo	Correct number of plies removed and feedback cleared.
S07 Worker failure	Worker raises an error, followed by a valid retry	Pending work rejects; error visible; a new Worker can serve a new request.
S08 Pause/resume	Pause during O's AI turn, then resume	Input is suppressed; obsolete work discarded; needed current turn can restart.
S09 Resized scene	Change viewport size after board content exists	Transform and visible board update without changing domain cells.
S10 Telemetry mismatch	Use a result missing counters or with invalid coordinates	Adapter rejects the incompatible result rather than inventing fields.

These scenarios are acceptance statements derived from ownership and invariants. Chapter 5 identifies the subset supported by the published test/observation artifacts. A scenario's presence in this table does not assert that a browser, touchscreen or hosting platform has passed it.

CHAPTER 4. IMPLEMENTATION

4.1 Toolchain and project organization

The application package declares React 19, React DOM, Pixi 7.4, TypeScript, Vite, Vitest, i18next and related integration packages. Its scripts provide development serving, production build, preview, formatting, static typechecking and a test run. These are package declarations at the pinned revision; the exact resolved dependency versions are governed by the lockfile. The report does not infer an installed global tool version from a version range in package.json. [31]

The source folders reflect the design boundaries. app contains the game controller. game/core supplies state, bounds, coordinates and rules. game/behaviors implements move and match transitions. game/ai defines the service and tactical route, while game/wasm provides the worker, adapter, typed contract and generated engine package. board/input, board/spatial and board/render separate gestures, transforms and drawing. pixi integrates them into a scene; React components provide controls and feedback around the canvas. [1]

This structure must be distinguished from the separate DOM-based CaroGame project preserved in the older local layout and historical source revision. Similar filenames do not establish that two folders run the same renderer or call the same engine. For the public revision used here, the executable application package named 10-caro is located under CaroGame, and its scene imports Pixi. Historical diagrams showing a DOM board or hooks are identified as that design's explanatory material rather than silently substituted for the active scene. [31] [24] [32]

4.2 Accepted move processing

At the action boundary, GameController.playCell first rejects a paused state. It starts a round when appropriate, rejects an ended match and prevents a human action while AI is thinking or while O owns the turn in AI mode. The controller then calls playMove with the current player and proposed cell. A nonaccepted result returns immediately without modifying board or history. However, from setup or ready, startRound runs before cell validation: a rejected direct call can advance the internal phase and round identifier without a notification. Normal pointer input filters out-of-bounds cells before this call; stronger atomic rejection at the controller boundary remains a verification and improvement item. For an accepted result, it

processes a win or draw, otherwise alternates the player, notifies subscribers and starts AI only when the match is still playing in AI mode with O to move. [2] [28]

The sequence is deliberate. Notification occurs after the accepted domain change, so the interface receives a coherent snapshot. The AI request is based on the updated history, not on a speculative screen state. Rule evaluation occurs through the supplied FreestyleGomokuWinRule. The scene never paints a permanent stone merely because a pointer event arrived; it renders the state returned through the controller. This permits the same move contract to reject an occupied cell consistently in the UI and in tests.

Undo is implemented by history reversal. In local two-player mode it removes one ply. In AI mode, if the last recorded player is O, it removes two plies; otherwise it removes one. Before reversal it invalidates AI work and clears search feedback. It then determines the current player from the remaining last move, asks the match controller to resume after Undo and notifies. The two-ply case describes the current human-plus-reply convention; it is not a generic time-travel algorithm that chooses an arbitrary earlier state. [2]

4.3 AI dispatch, Worker execution and WASM hydration

AIController.requestMove assigns a request identifier and inspects the difficulty. Easy and Normal bypass the TypeScript tactical shortcut so they actually execute the Rust Random and Rule-based policies. Higher tiers ask the tactical resolver for an immediate forced action for O. If one exists, the controller resolves a typed result with tactical source, null score and explicitly zero Rust counters. Otherwise it copies the entire accepted history and delegates to the WASM engine wrapper. This distinction is visible in the result contract and should remain visible in experimental interpretation. [3] [10]

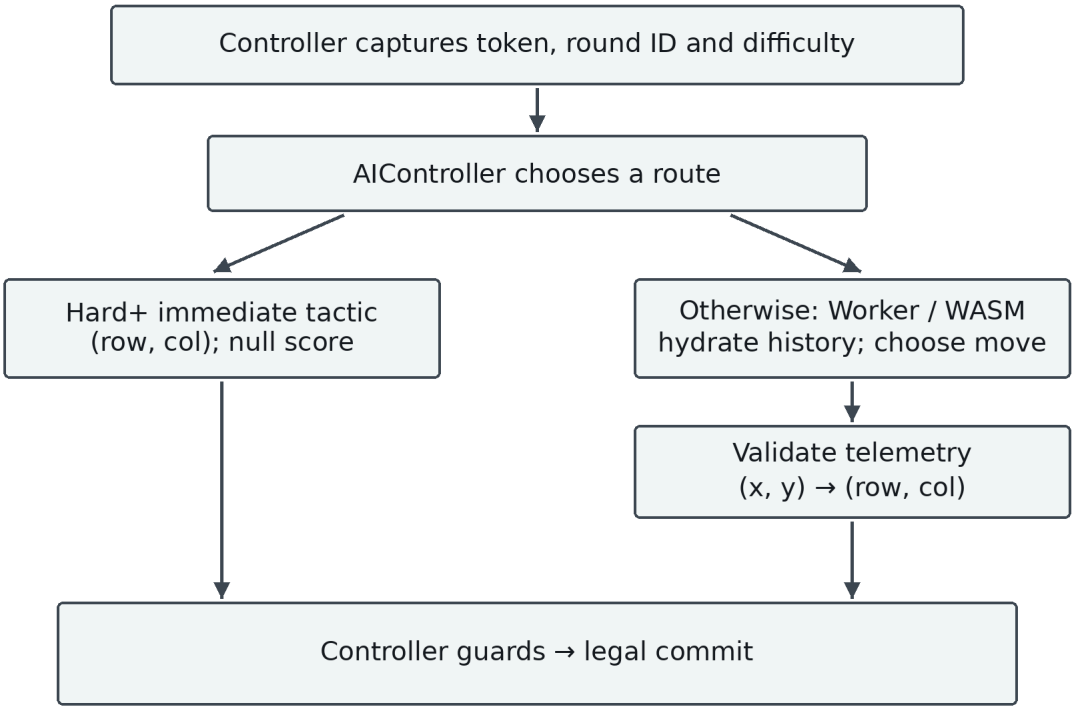


Figure 4.1. Tactical and Rust routes converge at the guarded commit

Author-drawn explanation from the cited source. Source: [3].

The wrapper creates a module Worker lazily, registers a pending promise under a transport identifier and posts a search message. worker.ts loads the generated engine module and invokes the shared executeWasmSearch function. Sharing this adapter with non-Worker tests reduces the chance that a test exercises a different coordinate or telemetry mapping from the browser route; it does not replace a real-Worker test. When a result message returns, the wrapper resolves the matching pending entry. A structured Worker error rejects it. [33] [27]

Hydration creates a fresh Rust Game object and replays each history move as (col,row). Each replay call must succeed. If any move cannot be applied, the adapter throws an error identifying the move rather than searching a partially reconstructed board. After hydration, it calls ai_move with the mapped difficulty identifier, validates the returned object and maps (x,y) back to (row,col). A finally block frees the game even if hydration, search or result validation fails. This is an explicit resource lifecycle, not an assumption that JavaScript garbage collection releases every Rust allocation promptly. [27]

The generated package includes JavaScript glue, TypeScript declarations and the .wasm binary. The Rust implementation is therefore a real part of the application route. The JavaScript file is a binding layer rather than a readable replacement for all

Rust search code. The archived Rust source and source excerpts explain the relevant algorithms, while the binary hash and adapter records connect the tested package to the recorded experiments. A complete rebuild claim additionally needs the exact modified Rust source and build inputs. [34] [7] [6]

4.4 Board rendering and interface synchronization

CaroScene creates a transparent, antialiased Pixi application associated with the supplied canvas. It builds a world container with separate grid, highlight/effect and stone layers. The scene stores the latest render stones, last move and winning-cell set supplied by syncState; these are rendering inputs derived from the controller, not independent move authority. The camera subscription reapplies translation/scale and refreshes the visible board. A ticker callback checks whether the viewport dimensions have changed and refits when necessary. [24]

The separation of layers explains the update path. A grid renderer draws board geometry. A stone renderer maps player/cell records to visual pieces. A highlight renderer displays the last move and winning cells. Input goes through the gesture controller and coordinate transform before onCellClick reports row and column. This organization permits tests to reason about bounds and camera math without requiring every rule test to instantiate Pixi graphics. Conversely, a core test cannot demonstrate that a visual stone is correctly positioned in a real browser.

The React application subscribes to controller state and synchronizes the scene when relevant snapshot fields change. It wires user controls to controller actions, supplies selection values and renders feedback/error states. It also coordinates hosting lifecycle inputs where configured. This layer is responsible for component effects and teardown; the scene's destroy path releases camera subscription, listeners, render layers and the Pixi application. The AI service's destroy method separately terminates its Worker resources. Explicit ownership avoids making a canvas object responsible for unrelated background computation. [22] [24] [26]

The interface includes translation resources for its supported labels and search details. Those resources map keys to displayed text; they do not change difficulty semantics or engine telemetry. An elapsed label, a completed-depth label and a strategy/source label should all refer to the current AI result. Undo, reset or a mode change clears stale feedback in the controller, ensuring that an old score is not displayed as though it belonged to the new board. [35] [2]

4.5 Presentation website and interactive explanation

present is a separate React application for explaining the project. Its application composes overview, roadmap, weekly algorithm material, benchmark sections, an interactive board and figure inspection components. Its purpose is pedagogical: readers can inspect an algorithm, a diagram or an example without treating the presentation as the authoritative running game engine. The source contains a browser-side demonstration engine and explanatory sections for search, candidates and advanced methods. [32] [36]

A presentation component can reveal a design decision or teach a recurrence, but the surrounding description must be checked against the executable source. An animation that follows a Minimax tree is not itself a measurement of the production engine. A figure registry's reproduction of an old result is not a new experiment. The report therefore traces quantitative claims back to their original JSON/CSV and distinguishes hand-entered examples from generated output. This is especially relevant when different weights or timing constants appear in teaching snippets.

The interactive board uses a presentation utility implementation with its own state and decision logic. It should be specified and tested within that application, but it must not be used to infer the internal performance of the Rust/WASM route. Similarly, the historical component hierarchy image describes a particular frontend arrangement; it is valuable evidence of work completed but is not automatically the hierarchy of the active Pixi application. The inventory preserves these artifacts and their roles rather than deleting their distinctions.

4.6 Python/Pyodide execution and four instructional programs

The presentation's Python runner loads or reuses a Pyodide instance. It captures stdout and stderr, loads packages inferred from imports when supported and executes code through runPythonAsync. The returned string is displayed to the reader, while caught errors become visible output. Pyodide executes Python in a browser environment based on WebAssembly; its presence does not mean the game AI is implemented in Python. The game engine's Rust/WASM path and the teaching runner are separate consumers of web execution technology. [37] [38] [39]

The runner caches an instance and queues callbacks for simultaneous loading. These mechanisms explain how its intended reuse works. They do not establish robust recovery from every loading failure: the code paths and pending callbacks

require their own review and tests. In particular, an educational execution panel should report loading/execution failure clearly and should not transform a failure into a fabricated successful numerical result. The report describes the observed implementation without silently claiming a more complete runtime manager.

4.6.1 Pattern scanner

The first program constructs a 15 × 15 NumPy board with four X stones horizontally and three O stones vertically. Its detection function inspects contiguous horizontal runs and classifies five, open four, closed four, open three and closed three according to endpoint availability. Therefore, the printed horizontal open-four result is meaningful for that example, while the mere presence of a vertical O line does not demonstrate that this function scans it. The code does not implement the full four-axis production win scan or a complete seven-pattern taxonomy. [38]

The input contract is a small integer matrix and a selected player. Its output is a dictionary of counts. A run should be counted once by identifying its beginning, extending to its end and checking endpoints without leaving the matrix. The example illustrates how an algorithm can transform board data into features; it does not prove that every printed instructional label accurately describes its scope. Broken threats such as XX.XX require additional pattern logic beyond counting contiguous occupied runs.

4.6.2 Heuristic calculator

The second program calculates a score from manually supplied pattern counts and a demonstration weight dictionary. The instructional attack/defence cases yield 94,000 and −7,000 under those inputs and its defensive factor. These are arithmetic outcomes of the supplied feature counts, not results obtained by asking the production Rust evaluator to scan a live board. Its weights must not be substituted for the game's different published baseline weights. [38] [15]

This example is useful for explaining why a favourable attacking feature and an opponent threat contribute with different signs. It should specify the feature vector, weights, opponent factor and score convention together. A negative value means the particular weighted formula favours the opponent under that convention; it does not independently establish a forced loss. Terminal utility and tactical completion checks remain separate mechanisms, as specified in Chapter 2.

4.6.3 Chebyshev candidate example

The third program places five stones and compares empty cells with candidates within Chebyshev radius two. It obtains 36 selected cells from 220 empty cells, so the reduction for that constructed board is (220 − 36)/220 × 100 = 83.64%. The denominator excludes occupied cells and the candidate mask excludes those cells as well. Replacing this per-board calculation with an average from a different experiment would change the meaning of the number. [38]

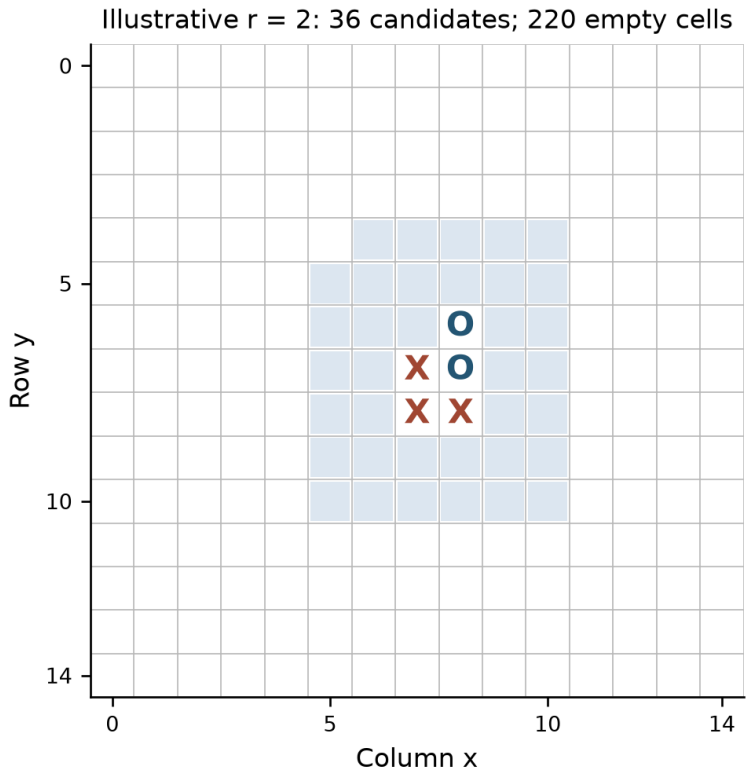


Figure 4.2. Union of clipped Chebyshev neighbourhoods in a five-stone example

Author-drawn geometry; exact snippet outcome is stated in the text. Source: [38].

The implementation demonstrates set membership and deduplication around multiple stones. Overlapping neighbourhoods contribute one candidate per cell. Board edges clip the neighbourhood, so a theoretical square size cannot be multiplied by the number of stones to obtain the unique candidate count. The example does not measure move quality, tactical retention or search speed; those require the policies and experiments described in Chapters 2 and 5.

4.6.4 Empirical-analysis snippet

The fourth program formats statistics that are already embedded in the snippet, including a timing value of 42.24 microseconds. It does not run one hundred games or load the published battle JSON during that execution. The archived battle dataset

reports a different average decision time, and the report discusses this discrepancy explicitly. Executing a print-based analysis demonstrates the formatting and arithmetic of its constants, not the provenance of the constants. [38] [40]

Dividing an example duration by 16.67 milliseconds computes a fraction of a nominal 60 Hz frame budget. It does not measure frame presentation, scheduling, input latency, hydration, message transfer or the rest of a game frame. A conclusion about frame drops needs an actual browser measurement of the relevant rendering/interaction workload. This distinction preserves the educational value of the example while avoiding a stronger empirical claim than its code supports.

4.7 Implementation traceability and limits

The implementation can be traced from a controller action through domain state and into display, or from a search request through copied history, a Worker, a Rust/WASM object and a validated response. The critical invariants are legal board/history correspondence, move restoration during search, consistent rule interpretation, correct coordinate conversion and current-turn applicability at commit. Each has a responsible module and can be tested at its boundary rather than inferred from a single large integration result.

The published tree makes substantial work inspectable: the application source and contracts, generated Rust/WASM package, Rust baselines and harnesses, research artifacts and presentation programs. It also leaves specific reproducibility limits for the exact modified engine and reference dependency used in some historical measurements. These limits do not remove the algorithms from the report. They determine how confidently an implementation-specific claim can be made and which source or evidence type should accompany it. Chapter 5 applies that discipline to every numerical comparison.

CHAPTER 5. RESULTS AND EVALUATION

5.1 Evaluation questions, evidence and boundaries

The evaluation addresses four separate questions. First, do the game and the recorded Rust engine preserve basic board and search behaviour on the tested positions? Second, on the same selected search tree, does Alpha-Beta return the same value as Minimax and how much work does it avoid? Third, what do move selection, ordering, hashing and a transposition table contribute, and where do they create risks? Fourth, what can be observed through the actual browser Worker and user interface? The answers come from different artifacts and cannot be merged into a single performance or correctness claim.

The repository snapshot examined for this report is commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc of tducn110/basicproject. The current-engine evidence was recorded on 23 October 2026; the separate reference experiments were recorded on 17 October. The published research directory contains measurements, a harness, provenance and selected source excerpts. The 23 October provenance identifies the measured engine and application as a local, uncommitted implementation. The archived excerpts preserve selected symbols from that snapshot; the source and reproducibility boundaries are listed in Appendix B. The 27 October local follow-up is identified separately as unpublished evidence and is not represented as a GitHub-backed result. [41] [42] [1]

The evidence has four levels. Native experiments exercise algorithms without a browser. Node-based WebAssembly parity checks exercise a copied binary and adapter on a finite corpus. A headless browser experiment exercises a real Worker and a browser UI experiment exercises selected interactions. Weekly reports and presentation components explain the project but can contain illustrative or historical material. A source hash identifies the files recorded at measurement time; it does not publish their contents or prove that an unrelated binary was built from them. Result files should therefore be read with their configuration, date and provenance rather than as claims about every revision or environment.

5.2 Native board and search checks of the 23 October engine

The native verification record reports four controlled positions and sixteen seeded positions, with three timing repeats for the controlled configurations. It records 816

search measurement rows, 408 Minimax/Alpha-Beta pairs, 408 ordering-value pairs and 820 root-restoration checks. It also records 84 candidate configurations, 12 tactical-completion retention checks, four terminal checks and 225 native address-offset checks. Its status is passed, but its own scope explicitly excludes browser and WebAssembly execution. The count of checks is not the number of independent players or games, and a finite fixture corpus cannot prove all possible boards correct. [43] [44]

The restoration check matters because recursive search temporarily places a stone and then undoes it. A returned score alone could conceal mutation of the caller's board. The archived source excerpt shows a search branch placing a candidate, recursively computing the child value and undoing the move. The published verification record says that the tested roots were restored. Together these give a traceable explanation of what was checked; the excerpt and result do not reproduce the complete modified project by themselves. A board geometry or address check is likewise not a measurement of WebAssembly memory size or browser throughput. [6] [43]

An illustrative comparison can be reconstructed directly from the published summary. For the center position at requested depth two, radius two and soft cap eight, all three configurations below return score 64 and the same best coordinate (x=6, y=6). Their search counters include the root. Their time values are the medians of three sequential native measurements, reported in nanoseconds; the timer excludes the root clone. [42]

Table 5.1. Native engine search on the centre fixture at depth two

Configuration	Nodes	Leaves	Cutoffs	Median time (ns)
Minimax, unordered	73	64	0	56,635
Alpha-Beta, unordered	47	38	7	41,723
Alpha-Beta, ordered	26	17	7	31,167

Relative to Minimax, unordered Alpha-Beta visits (73 − 47) / 73 = 35.62% fewer nodes; ordered Alpha-Beta visits (73 − 26) / 73 = 64.38% fewer. For this position and configuration, ordering also reduces the count from 47 to 26, or 44.68% relative to unordered Alpha-Beta. Both Alpha-Beta cases record 264 ranking evaluations, so leaf and node counts do not represent all work. This example illustrates pruning and ordering on one selected tree, not a general speedup guarantee. Measurement order

was not randomized and the CPU was not pinned; three observations are insufficient for a confidence interval or a general statement about machine-independent latency.

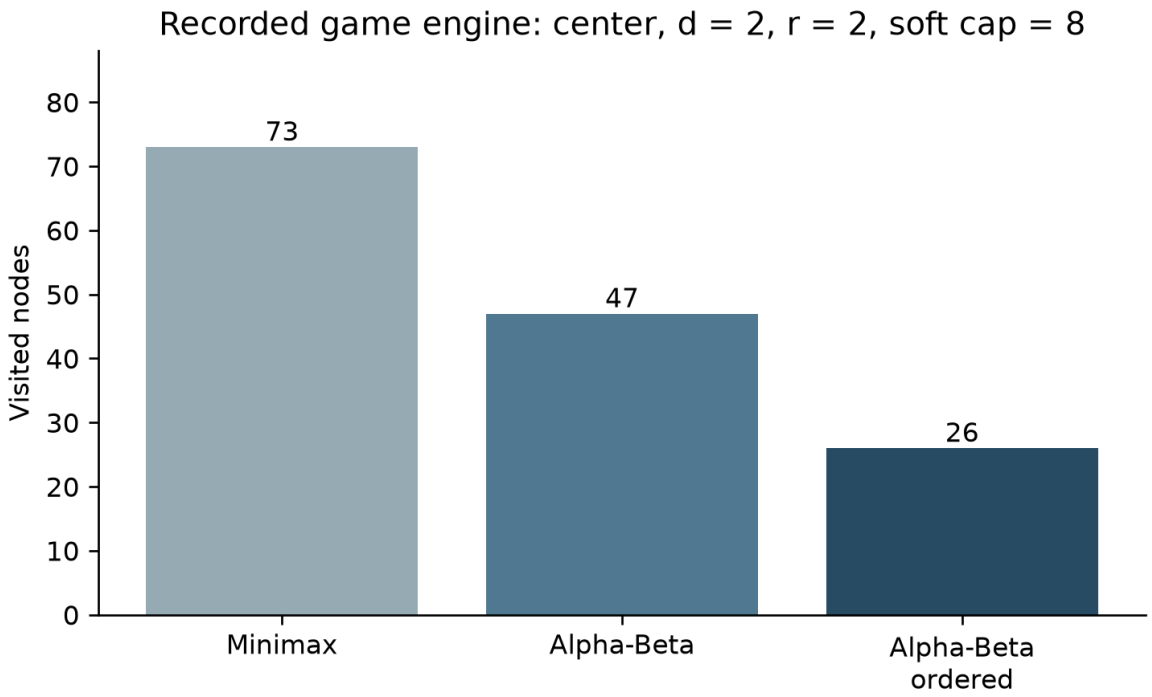


Figure 5.1. Recorded native engine nodes on the centre fixture

Chart reconstructed from the cited GitHub CSV. Source: [42].

The equality checks also have a defined meaning. When Minimax and Alpha-Beta use the same legal candidate membership, rules, evaluation and depth, agreement checks the two implementations on that selected tree. Ordering may change which equally valued move is encountered first, so a comparison that permits tied moves must compare values rather than insist on identical coordinates. A selective candidate set is a different tree from the all-legal tree; equal values on selected fixtures cannot establish that discarded moves are harmless on every board. Tactical retention checks test known immediate completions, not every future tactical sequence.

5.3 WebAssembly parity and browser observations

The parity artifact describes eight fixtures executed through Node's synchronous initialization of the copied WebAssembly binary and the actual adapter. Coordinates use row and column indices 0–14, with the Rust convention x = column, y = row, and the row-major index row × 15 + column. It records board hashes, accepted moves and outcomes. The terminal-turn convention is explicitly different: after a terminal move the recorded Rust state flips to the next player while the TypeScript state

retains the last mover. The parity assertion for next player is therefore limited to ongoing states. This distinction prevents a real representation difference from being misreported as full state identity. The binary hash in the record is 1ff854e6493b39727d07cfcf69b0af06e349a748249d85d688741a4f85017989. Eight fixtures are targeted evidence, not proof of all-state parity. [45]

A separate Worker record describes Headless Chrome 155 on Linux, using the fetched binary with the same recorded SHA-256. Its history contains four stones around the centre: X at (row 7, column 7) and (8,7); O at (7,8) and (8,8). For the six difficulty requests, the artifact reports legal returned moves and source: rust. Hard, Very Hard, Expert and Master report depth 2, 3, 4 and 5 respectively, with node counts 47, 252, 551 and 1,418 for this fixture. The Worker elapsed scope is hydration plus search, whereas the reported round trip includes message and scheduling overhead. For example, the Master row reports 3.0 ms Worker elapsed and about 3.3 ms round trip in that single run. Easy's 84 ms round trip and 1.8 ms Worker elapsed show why these scopes must not be conflated. The figures are individual observations, not latency percentiles, a benchmark distribution or a guarantee on another browser or device. [46]

The UI artifact separately records a 1280 × 800 headless browser run using actual React, Pixi input, synthetic DOM-dispatched pointer events and the Worker. It lists six difficulty labels, records a depth-two result with 61 nodes and 13 cutoffs before Undo, and records the details panel removed and Undo disabled after the action. No alert is recorded in those observations. This gives evidence for that specific interaction flow. The artifact itself excludes a physical touchscreen and Wink host; it does not demonstrate real-device input, host integration or deployment acceptance. [47]

The earlier gate ledger lists G3 as pending, G4 as partial and G5 as pending. Later evidence records a passed native G3 verification on its stated finite corpus [43], six legal Rust Worker results for the recorded browser fixture under G4 [46], and a passed synthetic-input UI/Undo observation under G5 [47]. These are verified observations with the stated scopes; the older ledger was not updated to consolidate them, and no overall human ACCEPTED decision is recorded [41].

5.3.1 Follow-up local application validation, 27 October

A later local validation was performed on the application copy corresponding to the project work. Its verification record reports nine Vitest files with 59 tests passed, a

successful TypeScript --noEmit check, and a successful Vite production build under Node 24.22.1. The build record includes a warning about the native configuration loader and a JavaScript chunk larger than 500 kB; it records a 111.58 kB WebAssembly output and 34.14 kB gzip size. These figures describe the recorded build output, not decoded memory consumption or a performance benchmark. They are local observations recorded on 27 October and are not represented as a GitHub-published log. The current 10_caro checkout also contains project work that must be analysed as the current local implementation even where its exact source revision has not yet been published. (Author's local observation, 27 Oct. 2026; unpublished run record.)

The same local record describes one browser observation at 1365 × 900 and 390 × 844. A synthetic pointer event at the canvas centre was followed by a Hard AI response and visible telemetry. This demonstrates the observed path for that local run, but is not a comprehensive interaction suite, physical touchscreen result, tournament or Wink host test. The test, typecheck and build checks exercise different boundaries; none alone establishes the accuracy of every AI move or production readiness. A future publication should preserve the test output, source revision and browser procedure under an immutable reference before treating this local run as independently reproducible. (Author's local observation, 27 Oct. 2026; unpublished run record.)

5.4 Separate native reference experiments

The 17 October reference experiment investigates Minimax, Alpha-Beta, candidate generation, ordering, Zobrist hashing, a transposition table and a node budget in a Rust reference library. It does not execute the game Worker, the game's vendored WebAssembly, or the separate game engine. The published harness uses a separate caro-core-reference dependency. The experiment is auditable as a published dataset and harness specification; Appendix B documents the source needed for independent reproduction. [8]

The controlled fixtures use a 15 × 15 board, freestyle rules and default evaluation weights. center starts with X at (7,7) and O to move. clustered has X at (7,7) and (8,7), O at (7,8) and (8,8), and X to move. In win, X occupies (7,3) through (7,6) while O occupies (5,5) and (5,6); X can complete a five at either end. The block fixture reverses the colours, making two immediate opponent endpoints. The tactical fixtures are synthetic and need not be reachable through ordinary alternating play. Sixteen

additional positions were generated through play transitions with the recorded random seed. [8]

The matrix compared all legal moves and uncapped radius-one candidates at depths one and two; radius-one candidates capped at twelve at depths one through three; and radius one capped at eight at depth four. Configurations included Minimax, plain Alpha-Beta with and without ordering, and an optimized search with a fresh table. The CSV contains 544 measurement rows and 224 summary rows. The verification file reports 546 search/assertion calls: the 544 recorded rows plus two cap-loss calls, so the two numbers do not conflict. It also records 551 restoration checks, 112 Minimax/Alpha-Beta pairs, 112 ordering-value pairs and 208 cold-table value pairs, with browser_or_wasm_exercised: false. [8]

5.4.1 A paired Minimax and Alpha-Beta example

For the clustered fixture, all-legal moves, depth two and natural order, both algorithms report score 0 and best move (row 6, column 7). Minimax records 48,842 nodes, 48,620 leaf evaluations and a median 69,252 µs. Alpha-Beta records 10,939 nodes, 10,717 leaves, 218 cutoffs and median 16,442 µs, from three repeats. The observed node reduction is (48,842 − 10,939) / 48,842 = 77.60%; the ratio of the two medians is approximately 69,252 / 16,442 = 4.21. These are properties of this particular fixture, depth and environment, not a universal multiplier. [48]

Table 5.2. Native reference search on the clustered fixture at depth two

Clustered, all-legal, depth 2	Nodes	Leaves	Cutoffs	Median (µs)
Minimax, natural order	48,842	48,620	0	69,252
Alpha-Beta, natural order	10,939	10,717	218	16,442
Alpha-Beta, ordered	2,164	1,942	220	68,457

The ordered search visits only 2,164 nodes, but performs 48,841 extra ordering evaluations and has a median of 68,457 µs. The counterexample is important: fewer recursively visited nodes do not necessarily mean lower elapsed time, because ranking candidates also costs work. At depth one, ordering cannot prune a lower level and can add overhead. The run uses native std::time::Instant, excludes state cloning and table/Zobrist initialization from the timer, has only three sequential controlled repeats, and does not randomize order or pin a CPU. Values must not be presented as browser timings. [48]

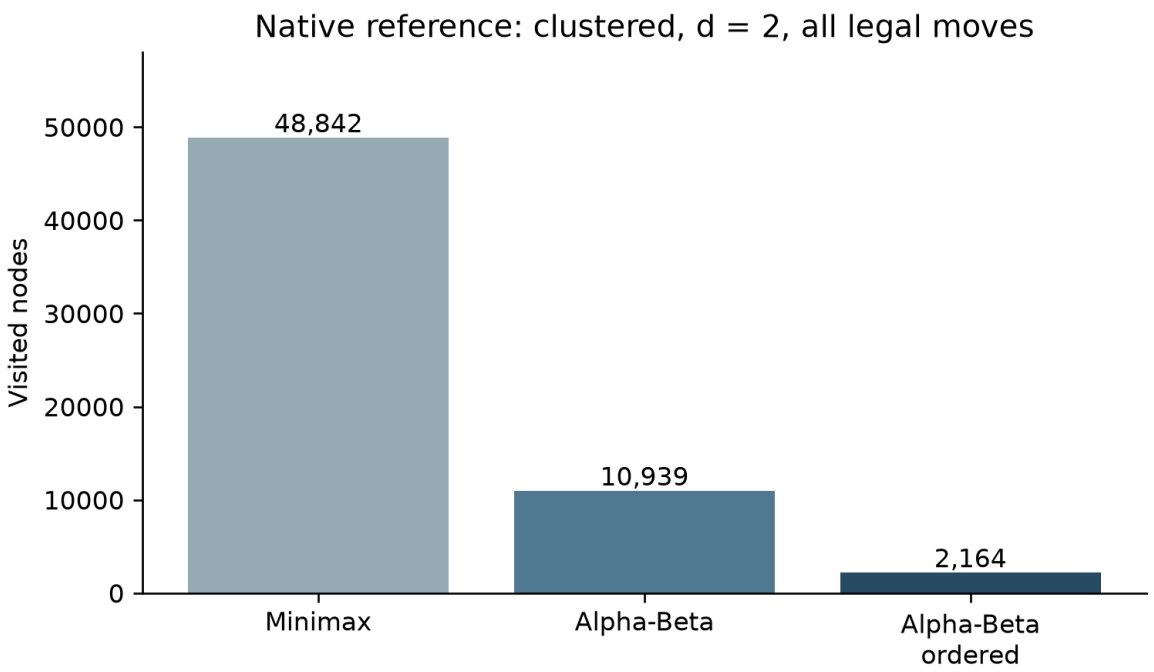


Figure 5.2. Native reference node counts, with ranking cost excluded from bars

Chart reconstructed from the cited GitHub CSV. Source: [48].

5.4.2 Candidate caps, ordering and tactical completeness

Candidate generation reduces the set of considered moves by selecting empty cells near existing stones. This can reduce branching but changes the search problem. The reference cap_loss.json supplies a concrete risk: in the synthetic win fixture at radius one, 20 uncapped candidates include two immediate winning endpoints (7,2) and (7,7), but a cap of six omits both. At depth one, the uncapped score is 9,999,999 and the capped score is 99,990. This is not an Alpha-Beta error: it is a difference in which moves are admitted before the search. It demonstrates why an ordering or cap policy must be evaluated against tactical membership, and why a value equality proof applies only after the move set is fixed. [16]

The later 23 October engine record reports twelve tactical-completion retention checks for its own candidate policy. These results concern another implementation and cannot erase the reference counterexample. Conversely, a failure of that reference cap does not establish that the later engine's soft cap fails. A useful future experiment would separate candidate generation, tactical retention, ranking and pruning in an ablation, using the same positions and reporting both selected move quality and total work.

5.4.3 Hash, table scope and interrupted search

The reference verification record reports 3,998 forward and 3,998 undo hash checks, plus four terminal hash-axis checks. Its table semantics record reports three bound variants and observed exact, lower and upper entries. These checks concern the inspected corpus and implementation. A native entry size of 12 bytes is recorded, but the underlying hash map is not a fixed 8 MB table; reservation or native object size alone does not establish total memory consumption. The record states that rule configuration and root perspective are not included in the key and mate-ply normalization is absent. Those facts constrain safe reuse. [49]

The cache-scope probe makes the constraint tangible. For a clustered fixture at radius one and cap eight, a warm table at the same root returns the same score of 0 as the initial search, with nine nodes and eight hits in the probe. After a move changes the root context, the recorded cold score is 0 and the warm score is 19,800. The record characterizes cross-root reuse of root-relative entries as unsupported. A conservative implementation must start with a fresh table or identify all evaluation and rule context in its namespace; the example does not prove that every warm reuse is unsafe. [21]

The budget artifact tests restoration during interruption. A direct budget of ten is exhausted and restores the root. In the iterative case, a budget of fifty is exhausted after a completed depth of two; 40 aggregate nodes are reported, and the record explicitly says that aggregate excludes the aborted iteration. It is therefore incorrect to describe 40 as the complete work consumed by all attempted iterations. The last completed result remains meaningful only when incomplete iteration state is not presented as a completed deeper result. [50]

5.5 Historical cross-language and Week 4 datasets

The repository also publishes a historical benchmark comparing a native Rust reference harness with a TypeScript harness running in Node. Its JSON has one named midgame fixture and depths one through four. At depth two, both rows report score 19,990, 609 nodes and 39 cutoffs; native Rust records 0.3125 ms and TypeScript 5.5795 ms. At depth four, both report score 9,999,997, 496,790 nodes and 16,350 cutoffs; the times are 285.4517 ms and 5,083.4518 ms respectively. Those matching counters show agreement for these recorded rows. They do not establish equivalence on all positions. [51]

Table 5.3. Historical native Rust and TypeScript/Node benchmark

Depth	Nodes, each row	Rust native (ms)	TypeScript/Node (ms)
1	45	0.0221	0.5383
2	609	0.3125	5.5795
3	16,184	9.0248	156.7846
4	496,790	285.4517	5,083.4518

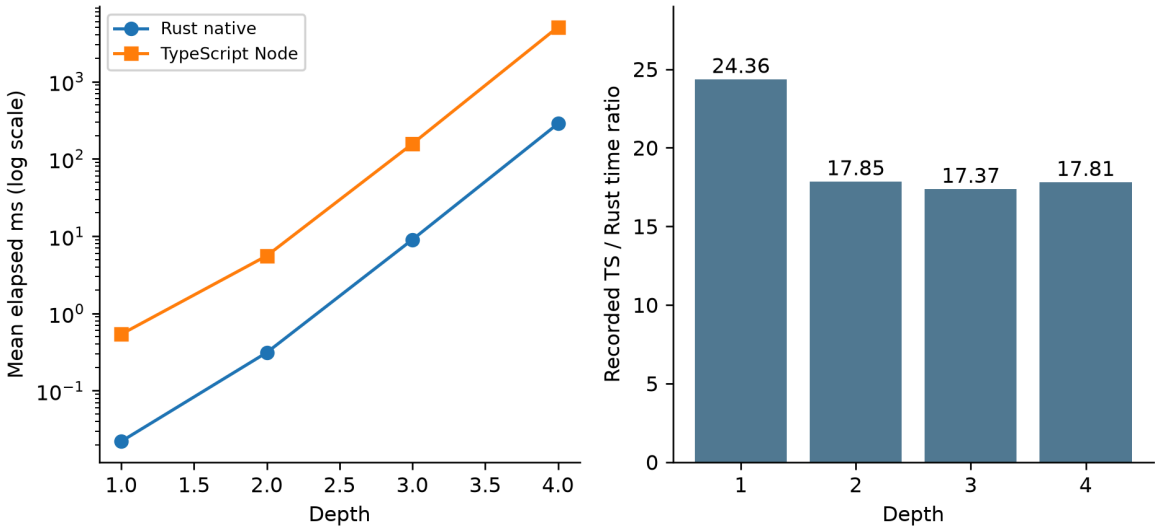


Figure 5.3. Historical native-versus-Node benchmark on one midgame fixture

Chart reconstructed from the cited GitHub JSON. Source: [51].

The Rust and TypeScript harnesses use different runtimes and iteration procedures, and the public reference dependency is missing. The JSON lacks a full independent uncertainty analysis. Any per-depth ratio is a descriptive ratio of those recorded values only; it is not evidence that the game Worker is a fixed multiple faster than a browser implementation, that the visual presentation runs at 60 frames per second, or that all devices have the same result. The data also cannot independently isolate the effects of language, runtime, warm-up and hardware.

The Week 4 battle JSON reports 100 native games of a Rule-Based bot against a Random bot: 100 Rule-Based wins, zero Random wins and zero draws, with mean 9.90 plies per game. It records a mean decision time of 251.26 µs, mean 220.21 empty cells and 60.80 candidates. From the recorded means, 1 − 60.80 / 220.21 is approximately 72.39% fewer candidates than empty cells. This is a ratio of aggregate means in the recorded experiment, not a guarantee on every position. The JSON also lists seven pattern fixture statuses as pass. The numbers describe the given opponent, setup and generator; they are not an estimated win rate against a

population of human players. The JSON is a summary rather than a per-game move history, so seat balance, seed sensitivity and independent game-by-game replay cannot be established from it alone. [40]

5.6 Presentation examples and evidence quality

The present application combines explanatory components, assets, a figure inspector and a Python code visualizer. It should be treated as a teaching and presentation artifact, separate from the game's actual Worker. In the pattern example, the code iterates over rows, even though its description mentions four axes; it is consequently a horizontal demonstration, not proof that the example scans vertical and diagonal lines. The heuristic example uses its own fixed weights and defensive multiplier. The Chebyshev example calculates candidates for an embedded toy board. These examples help explain a concept but do not automatically reproduce the current Rust engine's results. [38]

The fourth Python example embeds historical statistics rather than reading a fresh run. A comparison between a decision time and a theoretical 16.67 ms frame interval cannot establish a real browser frame rate: rendering, scheduling, Worker communication and measurement context also matter. Descriptions and hard-coded figures in presentation components should be traced back to actual data before being reused in a technical conclusion. Where an asset has no traceable dataset, its visual content is illustrative rather than an additional measurement. [38] [51] [40]

5.7 Threats to validity and interpretation

Internal validity. Search comparisons require the same position, depth, terminal rules, evaluation perspective and candidate set. Otherwise an apparent improvement can arise from searching a different tree. A table that ignores rule or perspective context can return a stale value. An aborted iteration must be counted separately from completed work. These issues are directly motivated by the cap-loss, cache-scope and budget probes.

Measurement validity. The native experiments distinguish recursive nodes, leaf evaluations, ranking evaluations, cutoffs and time. Treating one counter as a substitute for all computation can reverse the apparent conclusion, as the clustered ordered example shows. Three sequential timing repeats and lack of randomized order, CPU isolation or uncertainty intervals support descriptive comparisons only.

Native timer scope, Worker hydration and search, and browser round-trip duration are different quantities.

External validity. The fixtures cover controlled and seeded boards but not the full space of legal games. Tactical test positions may be synthetic. A 100-game Rule-Based versus Random result is narrow, and it does not calibrate six difficulty levels against human skill. Node execution and headless-browser synthetic input do not establish physical touchscreen, Wink host, deployment or production behaviour.

Reproducibility and provenance. The public repository includes raw records, harnesses, source excerpts and cryptographic hashes. Appendix B records the exact source gaps that affect independent builds. The stale gate table and later individual artifacts must be cited separately. A reproducible follow-up should archive exact source, build configuration, dependencies, tool versions, random seed, raw runs and a measurement protocol together under one immutable revision.

CHAPTER 6. CONCLUSION AND FUTURE WORK

6.1 Work completed and contribution

The project combines a playable Caro application with an educational sequence on representation, rules, heuristic evaluation and adversarial search. The application repository provides the user interface and game-side code; the weekly material documents the development sequence from initial proposal to search and advanced techniques. The presentation project supplies visual and executable teaching examples. Separate research artifacts provide controlled search data and record experimental boundaries. These products have different purposes and should not be treated as one executable engine. [1] [52]

The technical contribution is the mapping from game decisions to data structures and algorithms. A board state supports legal placement, directional line checks, history and rollback. A heuristic gives a bounded estimate for nonterminal positions, while terminal outcomes take precedence. Minimax specifies alternating decisions; Alpha-Beta can discard branches that cannot alter the selected-tree value. Candidate generation restricts the search space, and ordering changes traversal order. The reference experiments illustrate further techniques—incremental hashing, table bounds and iterative budgets—but do not prove that every technique is active in the current game. The application and reference implementations must each be described through their own source and evidence. [44] [8]

The 23 October native record reports agreement and restoration on its finite corpus, and the separate Node and browser artifacts exercise selected integration paths. The reference data shows a substantial node reduction in one paired example and also shows that ordering can consume enough additional evaluation work to remove the elapsed-time benefit. The cap-loss and cross-root cache probes expose concrete correctness risks. These findings are more useful than a single headline speedup because they identify both the assumptions under which the method works and the conditions that require more care.

6.2 Limitations

The local application validation adds a later test, build and browser observation, but its verification record is not published at the cited repository snapshot. It must remain identified as local evidence rather than converted into a fabricated GitHub citation.

The source and reproducibility gaps in Appendix B constrain verification of the historical experiments. The stale gate ledger is not a completed acceptance record, even though later artifacts support individual native, Worker and UI observations. No participant study, physical device acceptance, production deployment, comprehensive skill calibration or statistically powered performance comparison is supplied. The historical 100-game summary is against a Random bot, and its per-game traces are not available in the published JSON. [41] [44] [8] [40]

The evaluation therefore answers whether particular checks were recorded and what those runs reported. It does not prove game-theoretic optimality, all-state parity, universal performance superiority, all-device responsiveness or that the application is ready for every host environment. As the report is based on an immutable repository snapshot, later code or local checks require their own provenance before they can update these conclusions.

6.3 Future development

1. Publish a reproducible source bundle. Include the exact current Rust engine, reference library, manifests, lockfiles, harnesses, recorded hashes and build instructions under immutable revisions. Record any local modifications instead of presenting an earlier baseline as the measured source. Provide a script that compares binary hashes and states the platform and toolchain.

2. Expand rule and tactical testing. Add board-edge and long-line cases, occupied and out-of-range inputs, full-board draws, simultaneous threats, undo after terminal states and independently checked randomized move sequences. Compare TypeScript and WebAssembly outcomes under an explicit terminal-turn convention. Each new rule variant needs its own contract and oracle.

3. Measure candidate and ordering policies separately. Use a fixed corpus of reachable positions, include known immediate wins and blocks, compare all-legal and selective search, and report missed tactics, decision quality, ranking evaluations, nodes and elapsed time. Separate soft-cap tactical retention from a hard cap; record ties and ordering cost.

4. Harden and test table reuse. Specify the key namespace for rules, evaluation weights, root perspective and mate distance; validate exact/lower/upper entry semantics at varying depths and windows. Exercise warm and cold tables, collisions

and root changes, and measure actual memory rather than infer a fixed budget from entry size.

5. Improve interruption and telemetry. Define whether counters include aborted iterations and whether a depth means requested or completed depth. Test cancellation, reset, stale replies, Worker failure and restoration, with an explicit last-completed-result policy. Propagate real measured statistics consistently through the UI.

6. Evaluate user-facing behaviour. Run repeatable browser tests with real input, more viewport sizes and target devices; test the actual host integration separately. Measure end-to-end latency and frame timing in the intended environment, with enough repetitions and disclosed methodology. A human usability or difficulty study requires participants, protocol and consent before results can be claimed.

7. Synchronize the educational presentation. Mark illustrative code and figures, link measured charts to their source data and update outdated labels. Ensure that a four-direction description corresponds to code that actually scans four directions, or label a horizontal example accurately. Keep narrative claims distinct from executed results.

REFERENCES

[1] tducn110, “Repository snapshot,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/tree/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc. [Accessed: Oct. 27, 2026].

[2] tducn110, “CaroGame/src/app/GameController.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/app/GameController.ts. [Accessed: Oct. 27, 2026].

[3] tducn110, “CaroGame/src/game/ai/AIController.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/game/ai/AIController.ts. [Accessed: Oct. 27, 2026].

[4] tducn110, “CaroGame/src/game/core/WinRules.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/game/core/WinRules.ts. [Accessed: Oct. 27, 2026].

[5] tducn110, “Repository snapshot,” tducn110/caro, GitHub, commit c1e7447532e4c5ff5590f2c5562dd21cd0acf7b9, 2026. [Online]. Available: https://github.com/tducn110/caro/tree/c1e7447532e4c5ff5590f2c5562dd21cd0acf7b9. [Accessed: Oct. 27, 2026].

[6] tducn110, “research/current_2026_10_23/SOURCE_EXCERPTS.md,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/current_2026_10_23/SOURCE_EXCERPTS.md. [Accessed: Oct. 27, 2026].

[7] tducn110, “research/current_2026_10_23/baseline_engine/src/ai.rs,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/current_2026_10_23/baseline_engine/src/ai.rs. [Accessed: Oct. 27, 2026].

[8] tducn110, “research/week_05_09/README.md,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/week_05_09/README.md. [Accessed: Oct. 27, 2026].

[9] tducn110, “CaroGame/src/game/wasm/contracts.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/game/wasm/contracts.ts. [Accessed: Oct. 27, 2026].

[10] tducn110, “CaroGame/src/game/ai/TacticalMoveResolver.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/game/ai/TacticalMoveResolver.ts. [Accessed: Oct. 27, 2026].

[11] tducn110, “CaroGame/src/game/bot/search.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/game/bot/search.ts. [Accessed: Oct. 27, 2026].

[12] C. E. Shannon, “Programming a computer for playing chess,” The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science, ser. 7, vol. 41, no. 314, pp. 256–275, Mar. 1950, doi: 10.1080/14786445008521796. [Online]. Available: https://doi.org/10.1080/14786445008521796. [Accessed: Oct. 27, 2026].

[13] The Rust Project Developers, “Type layout,” The Rust Reference. [Online]. Available: https://doc.rust-lang.org/reference/type-layout.html. [Accessed: Oct. 27, 2026].

[14] tducn110, “CaroGame/src/game/core/GameState.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/game/core/GameState.ts. [Accessed: Oct. 27, 2026].

[15] tducn110, “research/current_2026_10_23/baseline_engine/src/eval.rs,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/current_2026_10_23/baseline_engine/src/eval.rs. [Accessed: Oct. 27, 2026].

[16] tducn110, “research/week_05_09/results/cap_loss.json,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/week_05_09/results/cap_loss.json. [Accessed: Oct. 27, 2026].

[17] D. E. Knuth and R. W. Moore, “An analysis of alpha-beta pruning,” Artificial Intelligence, vol. 6, no. 4, pp. 293–326, 1975, doi: 10.1016/0004-3702(75)90019-3. [Online]. Available: https://doi.org/10.1016/0004-3702(75)90019-3. [Accessed: Oct. 27, 2026].

[18] tducn110, “research/current_2026_10_23/engine_difficulty_runs.csv,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/current_2026_10_23/engine_difficulty_runs.csv. [Accessed: Oct. 27, 2026].

[19] A. L. Zobrist, “A new hashing method with application for game playing,” Computer Sciences Department, University of Wisconsin–Madison, Madison, WI, USA, Tech. Rep. 88, Apr. 1970. [Online]. Available: https://research.cs.wisc.edu/techreports/1970/TR88.pdf. [Accessed: Oct. 27, 2026].

[20] tducn110, “CaroGame/src/game/bot/board.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/game/bot/board.ts. [Accessed: Oct. 27, 2026].

[21] tducn110, “research/week_05_09/results/cache_scope_probe.json,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/week_05_09/results/cache_scope_probe.json. [Accessed: Oct. 27, 2026].

[22] tducn110, “CaroGame/src/App.tsx,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/App.tsx. [Accessed: Oct. 27, 2026].

[23] tducn110, “CaroGame/src/game/behaviors/match/MatchController.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/game/behaviors/match/MatchController.ts. [Accessed: Oct. 27, 2026].

[24] tducn110, “CaroGame/src/pixi/CaroScene.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/pixi/CaroScene.ts. [Accessed: Oct. 27, 2026].

[25] WHATWG, “Web workers,” HTML Living Standard. [Online]. Available: https://html.spec.whatwg.org/multipage/workers.html. [Accessed: Oct. 27, 2026].

[26] tducn110, “CaroGame/src/game/wasm/GomokuWasmEngine.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/game/wasm/GomokuWasmEngine.ts. [Accessed: Oct. 27, 2026].

[27] tducn110, “CaroGame/src/game/wasm/executeSearch.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/game/wasm/executeSearch.ts. [Accessed: Oct. 27, 2026].

[28] tducn110, “CaroGame/src/game/behaviors/move/PlayMove.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/game/behaviors/move/PlayMove.ts. [Accessed: Oct. 27, 2026].

[29] tducn110, “CaroGame/src/board/spatial/CoordinateTransform.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/board/spatial/CoordinateTransform.ts. [Accessed: Oct. 27, 2026].

[30] tducn110, “CaroGame/src/board/input/BoardInputController.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/board/input/BoardInputController.ts. [Accessed: Oct. 27, 2026].

[31] tducn110, “CaroGame/package.json,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/package.json. [Accessed: Oct. 27, 2026].

[32] tducn110, “present/src/App.tsx,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/present/src/App.tsx. [Accessed: Oct. 27, 2026].

[33] tducn110, “CaroGame/src/game/wasm/worker.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/game/wasm/worker.ts. [Accessed: Oct. 27, 2026].

[34] tducn110, “CaroGame/src/game/wasm/engine,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/tree/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/game/wasm/engine. [Accessed: Oct. 27, 2026].

[35] tducn110, “CaroGame/src/i18n.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/i18n.ts. [Accessed: Oct. 27, 2026].

[36] tducn110, “present/src/utils/caroEngine.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/present/src/utils/caroEngine.ts. [Accessed: Oct. 27, 2026].

[37] tducn110, “present/src/utils/pyodideRunner.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/present/src/utils/pyodideRunner.ts. [Accessed: Oct. 27, 2026].

[38] tducn110, “present/src/components/PythonVisualizer.tsx,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/present/src/components/PythonVisualizer.tsx. [Accessed: Oct. 27, 2026].

[39] WebAssembly Community Group, WebAssembly Specification, release 3.0, Oct. 23, 2026. [Online]. Available: https://webassembly.github.io/spec/core/. [Accessed: Oct. 27, 2026].

[40] tducn110, “benchmark/results/week4_bot_battle.json,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/benchmark/results/week4_bot_battle.json. [Accessed: Oct. 27, 2026].

[41] tducn110, “research/current_2026_10_23/GATES.md,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/current_2026_10_23/GATES.md. [Accessed: Oct. 27, 2026].

[42] tducn110, “research/current_2026_10_23/engine_search_summary.csv,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/current_2026_10_23/engine_search_summary.csv. [Accessed: Oct. 27, 2026].

[43] tducn110, “research/current_2026_10_23/engine_verification.json,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/current_2026_10_23/engine_verification.json. [Accessed: Oct. 27, 2026].

[44] tducn110, “research/current_2026_10_23/README.md,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/current_2026_10_23/README.md. [Accessed: Oct. 27, 2026].

[45] tducn110, “research/current_2026_10_23/wasm_ts_parity.json,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/current_2026_10_23/wasm_ts_parity.json. [Accessed: Oct. 27, 2026].

[46] tducn110, “research/current_2026_10_23/G4_real_browser_worker.json,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/current_2026_10_23/G4_real_browser_worker.json. [Accessed: Oct. 27, 2026].

[47] tducn110, “research/current_2026_10_23/G5_browser_ui.json,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available:

https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/current_2026_10_23/G5_browser_ui.json. [Accessed: Oct. 27, 2026].

[48] tducn110, “research/week_05_09/results/search_summary.csv,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/week_05_09/results/search_summary.csv. [Accessed: Oct. 27, 2026].

[49] tducn110, “research/week_05_09/results/tt_semantics.json,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/week_05_09/results/tt_semantics.json. [Accessed: Oct. 27, 2026].

[50] tducn110, “research/week_05_09/results/budget_abort.json,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/week_05_09/results/budget_abort.json. [Accessed: Oct. 27, 2026].

[51] tducn110, “benchmark/results/benchmark_data.json,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/benchmark/results/benchmark_data.json. [Accessed: Oct. 27, 2026].

[52] tducn110, “Week_05_09_Report_Index.md,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/Week_05_09_Report_Index.md. [Accessed: Oct. 27, 2026].

APPENDIX A. REPOSITORY INVENTORY AND EVIDENCE MAP

The following inventory covers all 20 top-level directories in the pinned Git tree and adds one row for top-level files, giving 21 inventory roles. Git does not track empty directories, and ignored or locally untracked materials are outside this snapshot. A weekly document is a historical project record; its statements should be checked against dated code and raw data before describing present runtime behaviour. [1] [52]

Table A.1. Repository inventory and roles

Item	Role and interpretation
.gemini	Repository-contained configuration/context materials; not an experimental result.
.obsidian	Vault configuration and navigation context; not a source of runtime acceptance.
CaroGame	Game application code, frontend assets, packages and tests. Inspect the pinned tree for the version actually being described.
Week1	Proposal and early scope/diagrams; establishes planned motivation and organisation.
Week2	Board and rule foundations; separates game-state concepts from later search.
Week3	UI and component architecture material and diagrams.
Week4	Pattern, heuristic, Random and Rule-Based material, including historical figures.
Week5	Minimax and search-tree analysis documents.
Week6	Alpha-Beta and table-related research documents; scope must be distinguished from active game wiring.
Week7	Candidate-generation documentation and its selected-move trade-off.
Week8	Move-ordering documentation and search-order trade-off.
Week9	Zobrist hashing and transposition-table documentation.
Week10	Iterative deepening, budgets and evaluation documentation.
archived	Earlier drafts retained for history, not automatic current authority.

Item	Role and interpretation
assets	Shared images and charts; datasets and generation code are needed to classify each figure.
benchmark	Rust and TypeScript benchmark harnesses and historical JSON summaries; the reference dependency is missing from this tree.
present	Presentation application, components, assets, static data and Python examples.
research	Dated engine evidence, baseline/excerpts, native research harness, raw CSV and JSON artifacts.
rules	Rule-related documentation; check its version and relation to the actual implementation.
scripts	Data/diagram-generation and benchmark orchestration scripts.
Top-level files	README, layouts, plans, review, benchmark discussion, report index, configuration and DOCX template; individually dated documents are not interchangeable.

The historical chronology is organised by the folders rather than by an unsupported claim that every later algorithm was integrated into the shipped game. Weeks 1–3 frame the problem, board and UI; Week 4 examines local patterns and baseline bots; Weeks 5–6 develop adversarial search and pruning; Weeks 7–8 focus on candidate selection and traversal order; Weeks 9–10 discuss hashing, table reuse and interrupted iterative search. The dated evidence in research must be used to establish which experiment actually ran and when. [52] [8]

Weekly deliverables and completion targets. These relative milestones identify the expected handoff, not a signed record that each week passed. Calendar start, end and review dates remain to be confirmed. W09–W10 describe reference research; game-engine integration requires its own evidence.

W01: Define board, actors, Freestyle scope and source boundaries; deliver the problem model and stated assumptions.

W02: Specify coordinate and state operations; demonstrate valid/invalid moves, win and draw checks on named fixtures.

W03: Describe UI ownership and match lifecycle; trace move, Undo, Reset and stale-request handling through their owners.

W04: Describe run patterns, evaluation and Random/Rule-based baselines; give a reproducible worked evaluation and identify the implementation.

W05: Specify Minimax recurrence, depth, apply/undo invariant and a traced example; identify the tested candidate set.

W06: Specify Alpha-Beta bounds and cutoff; compare root scores to Minimax under the same fixed search context.

W07: Specify radius, soft cap, immediate-tactic retention and six configurations; distinguish selection from filtering limits.

W08: Compare ordering with identical membership; record value, node and ranking-work definitions rather than assuming a speedup.

W09: Describe Zobrist updates and TT bounds; verify restoration and disclose collision, root-context and mate-score reuse hazards.

W10: Specify iterative deepening and budget interruption; show last completed result and restoration, and mark integration or tournament evaluation pending without execution evidence.

APPENDIX B. PUBLIC SOURCE GAPS AND CLAIM REGISTER

Table B.1. Public evidence and remaining verification gaps

Evidence or claim	What the snapshot contains	What remains open
23 October modified engine	Selected source excerpts, hashes, raw measurements and verification records.	Complete modified engine and exact build are not published; the earlier baseline_engine is a different state.
23 October app modifications	Provenance and selected excerpts plus separately committed game tree.	Establish byte-for-byte correspondence before equating the measured uncommitted app with a committed version.
Native reference library	Harness, results, detailed source hashes and configuration.	caro-core-reference dependency is absent; hash alone cannot reconstruct source.
Binary and adapter parity	Finite Node parity record and copied WASM hash; separate real browser Worker record.	All-state equivalence, full source-to-binary reproducibility and broad browser coverage are unproven.
Browser UI	One headless run and synthetic pointer interaction.	Physical input, Wink host, accessibility, other devices and production deployment need separate checks.
27 October follow-up	Local record of 59 tests, typecheck, build and two-viewport browser observation.	The run record is unpublished; archive exact source, logs, toolchain and procedure before external reproduction.
Gate status	G3 native verification passed on the recorded corpus; G4 real-Worker and G5 UI/Undo observations are recorded. The older ledger predates these observations.	A consolidated gate-by-gate review and acceptance decision are not provided.
Benchmarks	Raw or summary files and stated fixture/measurement scope.	Unpublished dependencies, small repeats, timing confounders, absent per-game traces and no universal performance claim.
Presentation	Source code, figures and embedded statistics.	Some descriptions or numbers require correction or attribution before use as experimental evidence.

APPENDIX C. READING AND REPRODUCIBILITY PROCEDURE

The commands below use repository-relative paths. They are instructions for a future controlled run, not a claim that the missing Rust dependencies are present or that these commands were executed for this report. Save existing result files before running any generator: the documented research runners overwrite their result directories. Use a clean checkout of the pinned revision and record the operating system, versions, exact command, hardware and any modifications with the output. [8] [44]

```
git clone https://github.com/tducn110/basicproject.git
cd basicproject
git checkout 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc
git status --short
```

For frontend inspection, run the scripts declared by the corresponding package after installing its locked dependencies in a suitable Node environment. A test or build only establishes the checks actually executed; it is not a device or host acceptance test.

```
cd CaroGame
npm ci
npm run typecheck
npm test
npm run build
```

For the presentation project, start a separate shell from the repository root. Its dev server serves a teaching site; starting it is not a benchmark.

```
cd present
npm ci
npm run build
npm run dev
```

Open another terminal at the basicproject repository root for the following commands. The native reference runner is documented below, but the pinned public tree lacks caro-core-reference, so it cannot run in a clean checkout until the exact dependency is supplied and verified against the published hashes. The Rust benchmark has its own local path dependency and is similarly incomplete in a clean checkout. Do not substitute an arbitrary library and label the result a reproduction of the historical experiment.

```
python3 research/week_05_09/run.py
cargo test --manifest-path caro-core-reference/Cargo.toml --locked
cargo run --release --locked --manifest-path research/week_05_09/Cargo.toml
-- research/week_05_09/results
```

The 23 October run_checks.py documents native, experiment, WebAssembly and frontend actions, but its referenced local measured project layout and modified source are not all in the published tree. Use the exact measured snapshot rather than silently substituting the archived baseline. Read the script and restore the exact source snapshot before considering reproduction; collect new outputs in a separate dated location. A future run should compare results and hashes without overwriting the historical records. [44]